



Este libro está pensado para aquellos que necesitan aprender rápidamente las herramientas matemáticas de los tres softwares. Nada de complejas descripciones. Este libro busca que el lector, en pocas líneas, entienda como realizar el ajuste de funciones, resolver sistemas de ecuaciones u operar con matrices, por ejemplo.

Alternativas de Matlab: Scilab y Octave

Aprendiendo los tres programas en un solo libro

Juan Pedro Messiga

Índice

Capítulo 0: Las interfaces de Matlab, Scilab y Octave	4
0.1 <i>Distintas partes de la interfase</i>	<i>6</i>
0.2 <i>Empezando a usar Matlab, Scilab y Octave.....</i>	<i>9</i>
0.3 <i>Agilizar el uso de Matlab, Scilab y Octave con el historial de comandos</i>	<i>11</i>
0.4 <i>Definición de variables.....</i>	<i>12</i>
0.5 <i>Octave y Scilab como una calculadora:</i>	<i>12</i>
0.6 <i>Como usar un script</i>	<i>13</i>
Capítulo 1: Operaciones matemáticas básicas.....	14
Capítulo 2: Operaciones básicas.....	18
2.1 <i>El operador colon y notación científica</i>	<i>19</i>
2.2 <i>Los comandos clear y clc y la función class</i>	<i>20</i>
2.3 <i>Interrumpir un cálculo.....</i>	<i>23</i>
2.4 <i>Los comandos help, error, exist y como introducir comentarios.....</i>	<i>23</i>
2.5 <i>Las funciones isscalar, isvector e ismatrix.....</i>	<i>24</i>
2.6 <i>Uso de paréntesis</i>	<i>25</i>
2.7 <i>Cambio de tipo de variable</i>	<i>25</i>
Capítulo 3: Operadores lógicos y de comparación	28
Capítulo 4: Funciones especiales.....	32
4.1 <i>Funciones trigonométricas</i>	<i>33</i>
4.2 <i>Otras funciones matemáticas</i>	<i>34</i>
Capítulo 5: Vectores y Matrices	37
5.1 <i>Definición, índices y el operador colon con matrices y vectores</i>	<i>38</i>
5.2 <i>Operaciones de matrices (suma, resta, multiplicación y potencia).....</i>	<i>40</i>
5.3 <i>División de matrices</i>	<i>42</i>
5.4 <i>Funciones especiales propias de matrices y vectores.....</i>	<i>45</i>
Capítulo 6: Precedencia	51
6.1 <i>Precedencia:</i>	<i>52</i>
Capítulo 7: Graficación	54
7.1 <i>Introducción a la graficación</i>	<i>55</i>
7.2 <i>Personalizado de curvas</i>	<i>61</i>
7.3 <i>Personalizado de la leyenda, el título y las etiquetas de los ejes</i>	<i>62</i>
7.4 <i>Creación de varios gráficos.....</i>	<i>65</i>
Capítulo 8: Funciones creadas por el usuario.....	69
8.1 <i>Funciones creadas por el usuario.....</i>	<i>70</i>
8.2 <i>Variables globales y locales.....</i>	<i>71</i>
8.3 <i>Definir una variable como global dentro de una función</i>	<i>76</i>

8.4	<i>Variables persistentes</i>	78
Capítulo 9: Estadística y números complejos		81
9.1.	<i>Generación de números aleatorios</i>	82
9.2.	<i>Promedio, mediana y desviación estandar</i>	83
9.3.	<i>Permutación de enteros aleatorios</i>	85
9.4.	<i>Rango, desviación absoluta promedio y covarianza</i>	85
9.5.	<i>Operaciones con números complejos</i>	88
Capítulo 10: Strings		91
10.1.	<i>Introducción a strings</i>	92
10.2.	<i>Formato de strings</i>	93
10.3.	<i>Recurrencia de impresión de strings</i>	95
10.3.1	<i>Recurrencia de impresión de strings en Octave</i>	95
10.3.2	<i>Recurrencia de impresión de strings en Scilab</i>	97
10.4.	<i>Funciones de strings</i>	98
Capítulo 11: Loops for y while		102
11.1.	<i>Loops while</i>	103
11.2.	<i>El comando continue</i>	104
11.3.	<i>El comando break</i>	105
11.4.	<i>El loop for</i>	106
Capítulo 12: Logical indexing		109
Capítulo 13: Operaciones con Word y Excel		112
13.1.	<i>Uso de paquetes de Octave</i>	113
13.2.	<i>Importar información de Excel a Octave</i>	115
13.3.	<i>Importar información de Excel a Scilab</i>	118
13.4.	<i>Exportar información de Octave y Scilab a Excel</i>	119
13.5.	<i>Exportar información de Octave y Scilab a un archivo de texto</i>	121
13.5.	<i>Las funciones “fgets” y “mgetl”</i>	125
Capítulo 14:		127
Resolución de sistemas de ecuaciones diferenciales, algebraicas y cálculo diferencial e integral		
14.1.	<i>Las funciones simbólicas y la función “subs”</i>	128
14.2.	<i>Resolución de integrales</i>	129
14.2.1	<i>Resolución de integrales definidas</i>	130
14.3	<i>Cálculo de derivadas</i>	132
14.4	<i>Resolución de ecuaciones diferenciales</i>	135
14.5	<i>Resolución numérica de ecuaciones diferenciales</i>	139
14.6	<i>Resolución de sistemas de ecuaciones lineales</i>	144
14.7	<i>Resolución de sistemas de ecuaciones no lineales</i>	145

Capítulo 15: Herramientas de control automático de procesos	149
15.1 Definición de funciones de transferencia	150
15.2 Respuestas de lazo abierto	151
15.3 Definición de un PID y la respuesta a lazo cerrado	155
15.4 Lugar de las raíces y diagrama de bode.....	156
Capítulo 16: Ajuste de funciones	159
16.1 Ajuste de funciones: Introducción	160
16.2 Ajuste de funciones: Octave	160
16.3 Ajuste de funciones: Matlab	161
16.4 Ajuste de funciones: Scilab	163

Capítulo 0

Las interfaces de Matlab, Scilab y Octave



¿Qué es Matlab?

MATLAB es un software diseñado para el cálculo matemático. Provee al usuario de un entorno de desarrollo integrado (IDE) y un lenguaje de programación propio. Al ser un software rico en herramientas matemáticas y el cálculo numérico, permite la manipulación de matrices, la representación de datos y funciones por medio de graficos en hasta tres dimensiones, la representación estadística de información y muestras y el desarrollo de algoritmos para cualquiera de las ramas de la matemática.

Al tratarse de un programa comercial, propiedad de Mathworks, se requiere de una licencia paga para su uso. Además de permitir el desarrollo de algoritmos, cuenta con una interfaz gráfica llamada Simulink, la cual actúa como un paquete adicional (y está fuera del alcance de este libro) pero que para usarse óptimamente se debe conocer el lenguaje de Matlab y sus funciones fuera de Simulink. Además del contenido básico del software, Mathworks desarrolló distintos *toolboxes*. Es decir, contenido adicional para usuarios interesados en ramas específicas, como el ajuste de curvas, el desarrollo de base de datos o la ciencia de datos.

El programa es sumamente versátil y permite su uso en diversas ramas de la ciencia y la ingeniería. Es por ello que cuenta con millones de usuarios en el mundo, siendo probablemente el más utilizado de los softwares de este tipo. Sin embargo, al ser un software pago, fue llevando al desarrollo de otros softwares. En este libro se enseña cómo usar dos de ellos, llamados Scilab y GNU Octave.

Vale la pena aclarar que el lenguaje utilizado en Matlab es el mismo que el de GNU Octave, por lo que aprender uno de ellos permite saber utilizar el otro. Por eso, al terminar de leer este libro sabrá programar en los tres softwares: Matlab, Scilab y Octave.

Hablemos de Octave

Octave es considerado la alternativa libre de Matlab. Su lenguaje y las funciones que utiliza son prácticamente las mismas. Sus entornos de programación son, además, muy similar. Es por eso que ambos softwares pueden enseñarse de forma paralela. De hecho, en este libro, todo lo que se explique de Octave se puede usar idénticamente en Matlab (salvo que se indique lo contrario, lo cual ocurre en solo dos veces en las más 160 páginas que tiene este libro). Es por eso que con lo que acá se aprenda de Octave también se aprende en Matlab.

Otra de las grandes ventajas de Octave es que su licencia es gratuita, por lo que puede ser usado por cualquiera. Por otro lado, la gran desventaja de este software es que no cuenta con una interfaz gráfica como Simulink.

Al ser un programa de código abierto, los usuarios pueden desarrollar sus propios paquetes de funciones y subirlas a internet para compartirlas con otros miembros de la vasta comunidad de Octave. De este modo la comunidad crece y se enriquece mes a mes.

Conociendo Scilab

Scilab es otro software similar a Matlab y Octave y, como este último, es de licencia gratuita. Fue desarrollado por el INRIA (Institut National de Recherche en Informatique et en Automatique). Su lenguaje es parecido al de estos dos, pero con algunas diferencias un poco más notorias. La principal desventaja que presenta este software es que muchas veces las funciones que utiliza tienen nombres distintos a los de Matlab y Octave. Por ejemplo, la función de Scilab que permite conocer la clase de una variable es “typeof” mientras que en Matlab y Octave es “class” (tienen el mismo nombre en ambos softwares). Así, el usuario debe conocer la equivalencia entre funciones análogas.

En este punto, el lector debe estar planteándose cuál es la razón para aprender a usar Scilab. La principal razón es que este software tiene una interfaz gráfica similar Simulink llamada Xcos. Si bien la misma no será estudiada en este libro, saber usar correctamente Scilab permite usar Xcos. Además, la versión actual de Scilab permite la activación de paquetes de funciones, como Octave.

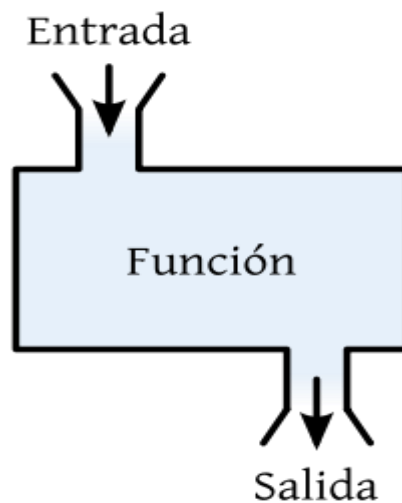
Resumiendo...

En definitiva, considero sumamente útil que los usuarios que trabajen con algoritmos relacionados con temas de la ciencia y la ingeniería aprendan a usar los tres softwares. Cada uno de ellos tienen ventajas y desventajas, lo cual determina las siguientes razones para usar uno u otro:

- Matlab: Es el programa de cálculo matemático más utilizado por las empresas y universidades alrededor del mundo. Además, permite el uso de Simulink. La gran desventaja es que es un software pago.
- Octave: Es considerado la alternativa libre a Matlab, por lo que muchas de las funcionalidades de uno están disponibles en otros. Su problema principal es que no tiene Simulink o una versión alternativa.
- Scilab: Es un programa gratuito, pero su lenguaje y funciones pueden ser distintos al de los otros dos softwares. Sin embargo, compensa esta desventaja introduciendo la interfaz gráfica Xcos, similar a Simulink.

0.1 Distintas partes de la interfase

Matlab, Scilab y Octave son softwares que utilizan comandos y funciones, las cuales son introducidas en cada programa y se obtienen resultados. Los comandos son sentencias que escribe el usuario y el ordenador da una respuesta. Por ejemplo, que se borre el contenido de las variables. Por su parte, las funciones son subrutinas o métodos que toman una determinada cantidad de inputs (entradas) y da un número determinado de outputs (salidas). La siguiente figura muestra un esquema de una función, donde se tiene una entrada y una salida.



Matlab, Scilab y Octave tienen interfaces similares. En cada una de ellas se encuentran las siguientes subinterfaces, todas ellas tienen las mismas funciones en cada uno de los tres programas. A continuación se presentan las funciones de cada subinterface:

1. Ventana de comandos o **Consola** (Command Window o **Console**): Se usa para
 - introducir comandos o llamar funciones.

```
>> sind(90)  
ans = 1
```
 - También se definen variables.

```
>> x=2  
x = 2
```

```
>> y=1+2
y = 3
>> z=x+y
z = 5
```

2. Explorador de archivos (Current directory o **File browser**): Muestra la carpeta en donde se guardan los scripts (ver sección 6 de este capítulo) y funciones creadas por el usuario (ver capítulo 8) y otros archivos, como tablas.
3. Editor: Es el espacio donde se escriben los scripts y las funciones. Se crean en el editor y se llaman desde el editor o desde la ventana de comandos.
4. Espacio de trabajo (Workspace o **Variable browser**): Presenta las variables que se encuentran definidas como globales.
5. Historial de comandos (Command History): Quedan guardados todos las funciones y comandos utilizados por el usuario. En la ventana de comandos se puede utilizar la flecha para arriba del teclado para ver que cosas van saliendo del command history. También se ejecutar o correr los comandos y funciones del historial de comandos haciendo doble click en ellos.

En la siguiente figura se presenta una captura de pantalla de la interfaz de los tres softwares. Se observa que todos tienen interfaces muy similares. Cada una de las cinco subinterfaces arriba listadas se encuentra recuadrada con un color.

1. Ventana de comandos o **Consola** (Command Window o **Console**): Se encuentra recuadrada en **violeta**.
2. Explorador de archivos (Current directory o **File browser**): Se encuentra recuadrado en **rojo**.
3. Editor: Se encuentra recuadrado en negro.
4. Espacio de trabajo (Workspace o **Variable browser**) Se encuentra recuadrado en **marrón**.
5. Historial de comandos (Command History): Se encuentra recuadrado en **celeste**.

En la primera imagen se compara a Matlab con Scilab. Mirando en detalle se observa que en Matlab se encuentran cinco subinterfaces, mientras que en Scilab solo cuatro. La subinterfaz faltante es el editor y para acceder a la misma se debe hacer click en alguno de los dos íconos que se encuentran recuadrados en negro, al lado de la flecha. El ícono de la izquierda crea un nuevo archivo de texto, el cual pertenece al editor. Esto es asimilable al editor de Matlab, el cual se encuentra recuadrado en negro.

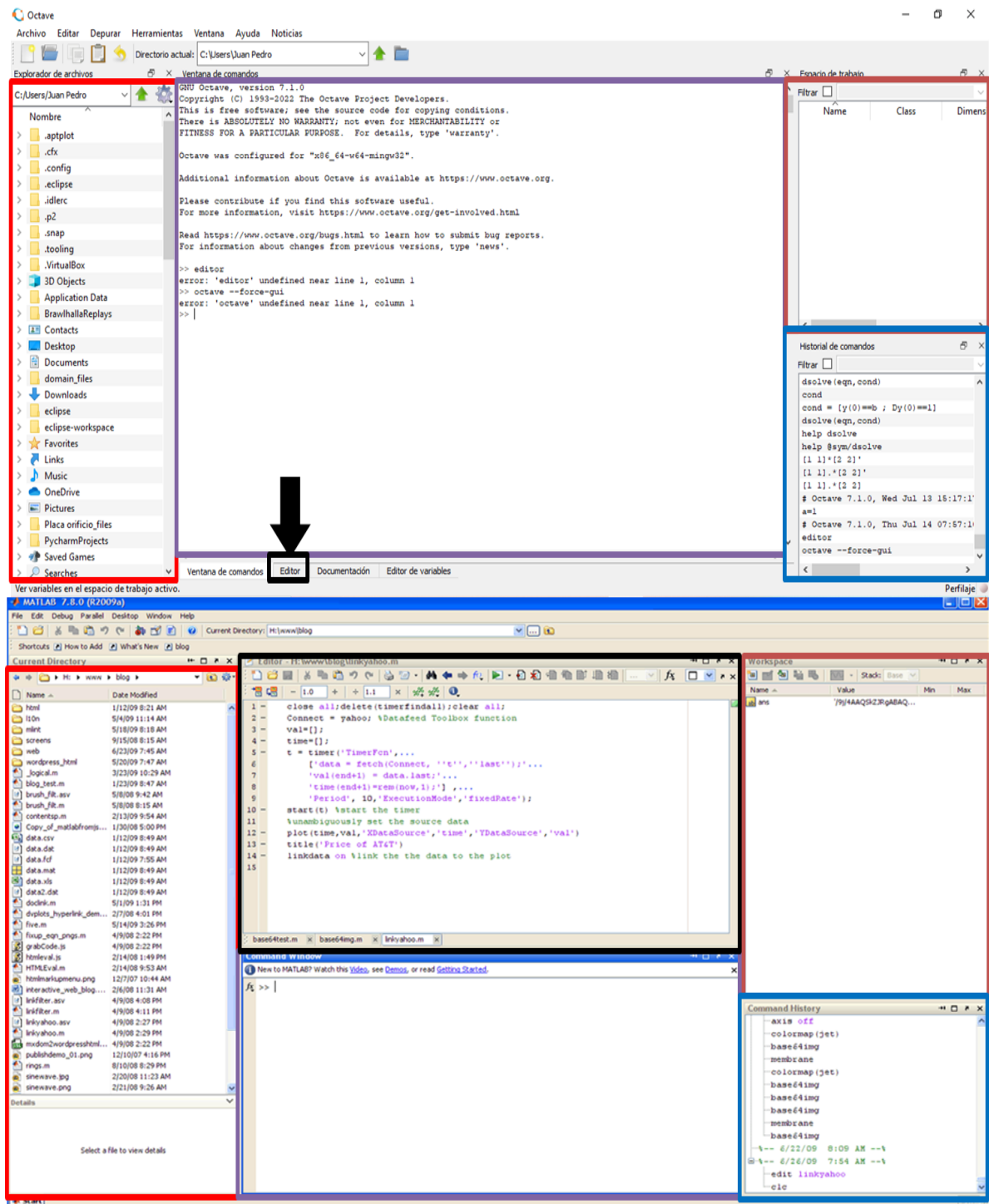
En la segunda imagen se compara a Matlab con Octave. Una vez más se observa que las subinterfaces son iguales. La única diferencia, al igual que con Scilab, está en el editor. Mientras que en Matlab está visible (y recuadrado en negro), en Octave se encuentra oculto tras la viñeta “editor”. Para acceder al mismo se debe hacer click en ella. Luego de eso el usuario se encontrará con un editor de texto plano en el que puede escribir sus comandos, funciones o sentencias.

Es posible que la primera vez que el usuario abra Scilab u Octave se encuentre con que la disposición de las interfaces no es la misma que la presentada en estas imágenes. Para obtener una disposición como esta se puede hacer lo siguiente:



En Scilab se debe hacer click en la ventana de comandos. Luego, se hace click en el ícono de “Scilab Preferences” (o “Preferencias de Scilab”). Después, se hace click en el “+” a la izquierda “General” y se selecciona “Desktop layout”. Finalmente, se selecciona un layout “Integrated”. Se hace click en “OK” y se reinicia el programa

En Octave simplemente se hace click en el menú “Ventana” y se habilita la ventana de comandos, el explorador de archivos, el editor, el espacio de trabajo y el historial de comandos, dependiendo de si los mismos están activados o no.



0.2 Empezando a usar Matlab, Scilab y Octave

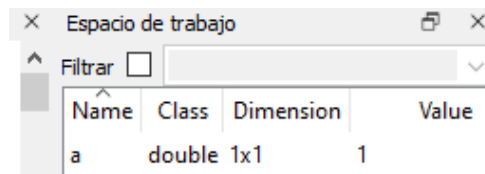
Se puede mostrar el valor de una variable simplemente escribiéndola en la ventana de comandos. Además, si se utiliza “;” al final de una sentencia, el resultado no aparece en la ventana de comandos.

```
>> a=1
a = 1
>> a=1;
```

El símbolo “>>” (en scilab es “-->”) representa la línea de la ventana de comandos donde el usuario introduce las instrucciones o sentencias que se le da al programa. Se debe escribir la sentencia que el

usuario desea que se ejecute y luego se pulsa la tecla “ENTER”. Si el comando o sentencia se ejecutó correctamente el programa mostrará el resultado de la operación (como ocurre en “a=1”) u ocultará el resultado (como en “a=1;”). En caso que la sentencia no se haya podido ejecutar se obtendrá un mensaje de error describiendo el mismo.

Se debe notar que luego de introducir cualquiera de las dos líneas introducidas más arriba, en el espacio de trabajo aparece la variable “a”, mostrando que la misma se creó correctamente. Además, en el historial de comandos apareció una nueva línea, la cual representa la creación de la variable “a”.



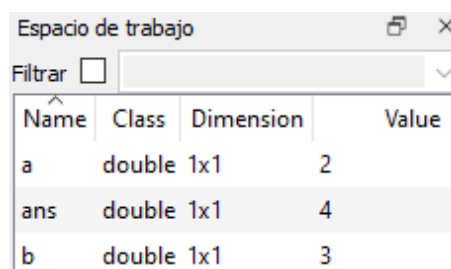
Name	Class	Dimension	Value
a	double	1x1	1

Para definir una variable Se procede como sigue a continuación. Además, las variables definidas pueden realizar las operaciones matemáticas definidas en cada software. Notar que estas dos últimas operaciones también se agregan al historial de comandos.

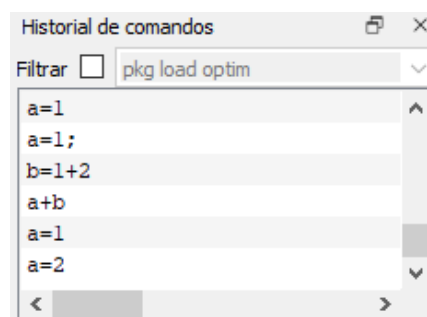
```
>> b=1+2
b = 3
>> a+b
ans = 4
```

Si una variable ya se encuentra definida pero el usuario vuelve a “redefinirla” entonces el valor de la misma se sobrescribe. Por ejemplo, el resultado de escribir las dos líneas siguientes es que “a” tome el valor 2. Además, en la ventana de trabajo la variable “a=1” será reemplazada por la variable “a=2”. Por el contrario, en el historial de comandos aparecerán las dos líneas.

```
>> a=1
a = 1
>> a=2
a = 2
```



Name	Class	Dimension	Value
a	double	1x1	2
ans	double	1x1	4
b	double	1x1	3



Command
a=1
a=1;
b=1+2
a+b
a=1
a=2

Hay que tener en cuenta que para poder utilizar una variable la misma debe estar previamente definida. Esto puede verse en la siguiente línea, donde al no estar “x” definida entonces el software da error. Para comprobar que la misma no existe puede observarse que la misma no se encuentra en el espacio de trabajo.

```
>> a+x
Error: 'x' undefined near line 1, column 3
```



Lo mismo pasa si se llama una función o comando que no existe. Por ejemplo:

```
>> rayo_a_trueno
Error: 'rayo_a_trueno' undefined near line 1, column 3
```

Después de cada operación en la ventana de comandos en que no se defina una variable se crea una variable “ans”, que sería la respuesta o el resultado de la última operación. “ans” puede usarse como una variable más y sumarse a otras variables o números. Sin embargo, debe tenerse en cuenta que luego de cada operación en que no se defina o sobrescriba el valor de una variable, “ans” cambia de valor.

```
>> 1+1
ans = 2
>> ans
ans = 2
>> 1+2
ans = 3
>> ans
ans = 3
>> ans+1
ans = 4
>> ans+a
ans = 5
```

0.3 Agilizar el uso de Matlab, Scilab y Octave con el historial de comandos

El historial de comandos puede agilizar el uso de Matlab, Scilab y Octave de dos modos distintos.

El primero es que cuando el usuario está utilizando la ventana de comandos y desea ejecutar un comando, función o sentencia que sabe que ejecutó hace poco (y por lo tanto está guardado en el historial de comandos) puede apretar la flecha hacia arriba del teclado, lo que ocasionará que cada vez que lo haga aparecerá una de las funciones, comandos o sentencias que ejecutó anteriormente. Lógicamente, el orden en que aparecen es el que tienen en dicho historial, desde abajo hacia arriba. Para ilustrar esto se puede tomar la última imagen, en la cual se presentan distintas sentencias. El orden en que aparecerán estas es el siguiente:

```
>> a=2
>> a=1
>> a+b
>> b=1+2
>> a=1;
>> a=1
```



El lector puede notar que el orden en que aparecen los comandos es el que corresponde a ir de abajo hacia arriba en la figura del historial de comandos de la sección anterior.

Otra forma de usar el historial de comandos es hacer doble click en cualquiera de las funciones, comandos o sentencias que aparecen en él. Por ejemplo, si se hace doble click en la sentencia “a+b” y luego en “b=3”, entonces en la ventana de comandos se observará lo siguiente:

```
>> a+b  
  
ans = 5  
>> b=1+2  
  
b = 3
```

Usando estas dos herramientas, vinculadas al historial de comandos, el usuario puede hacer un uso más ágil de Matlab, Scilab y Octave.

0.4 Definición de variables

Los nombres de las variables no pueden empezar con determinados caracteres (números, %, por citar algunos). El símbolo de porcentaje “%” en Octave, por ejemplo, se utiliza para hacer comentarios, por lo que lo que se escriba después de este (dentro de la misma línea) el programa lo considera como no estuviese. **Lo mismo aplica a las dos barras “/” en Scilab, las cuales también sirven para comentar una sentencia.** Notar también que una variable no puede empezar con un número, pero sí puede usarse números en el nombre de la variable. Por eso, “1a” no puede ser el nombre de una variable, pero “a1” sí.

```
>> %a=1  
>> 1a=1  
error: parse error:  
  
syntax error  
  
>>> 1a=1  
  
>> a1=1  
a1=1
```

0.5 Octave y Scilab como una calculadora:

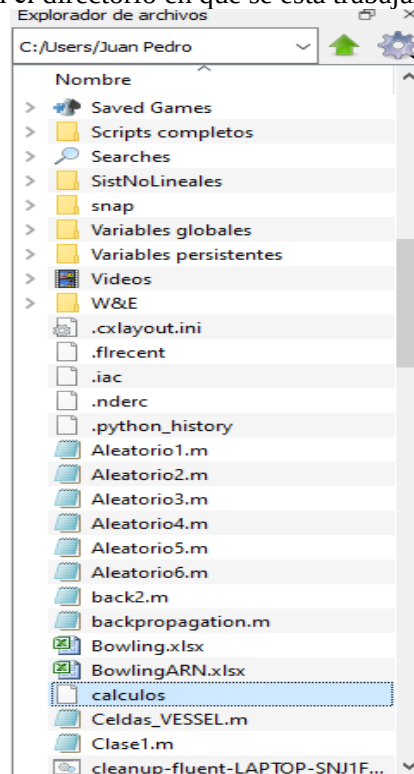
¿Qué es una calculadora? Es un dispositivo que permite realizar cálculos, pero una vez obtenido el número el valor no queda guardado en una variable (a excepción de “ans”, que se sobrescribe después de cada cálculo). Es decir, no se guardan los resultados.

A Matlab, Scilab y Octave se los puede usar de un modo similar. Se puede introducir operaciones, obtener resultados, definir variables y ejecutar funciones. Sin embargo, si no se guardan en variables usando el comando “save” o en un script (archivo de texto) los resultados se pierden. A continuación hay una serie de sentencias.

```
>> a=0  
a = 0  
>> x=90  
x = 90  
>> y=x+8
```

```
y = 98
>> save calculos
```

El archivo “cálculos” se guarda en el directorio en que se está trabajando actualmente.

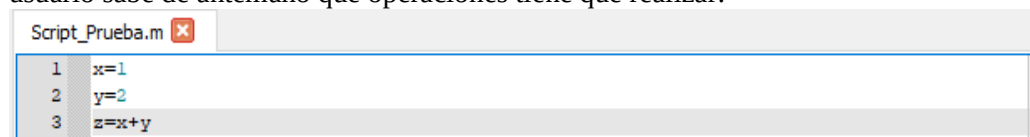


Si el usuario borra las variables (como se verá más adelante, el usuario puede hacer esto usando el comando “clear” o saliendo de Octave o Scilab), puede recuperar las mismas introduciendo en la ventana de comandos lo siguiente. De este modo, las variables guardadas en “calculos” aparecerán en el workspace con sus respectivos valores.

```
>> load calculos
```

0.6 Como usar un script

Un script es un archivo de texto con extensión “.m” donde el usuario escribe una serie de comandos y funciones. Las sentencias se van ejecutando una a una en orden, de izquierda a derecha, y cuando llega al final de la línea, se pasa a la de abajo. Luego de escribir un script, el usuario debe darle al botón de “play”. Así, Octave y Scilab chequea que no haya ningún error de sintaxis y, de ser correcto, ejecuta una sentencia después de otra, como fue descripto anteriormente. Los scripts son usados cuando el usuario sabe de antemano que operaciones tiene que realizar.



En este script se define a “x” como 1, a “y” como 2 y a “z” como la suma de las dos anteriores. Luego de ejecutar el símbolo de “play” en el editor, en la ventana de comandos aparecerá lo siguiente:

```
>> Script_Prueba
x = 1
y = 2
z = 3
```

Capítulo 1

Operaciones matemáticas básicas



Octave y Scilab nos permiten realizar las siguientes operaciones matemáticas:

Operación	Símbolo	Función
Suma	+	-
Resta	-	-
Multiplicación	*	-
Multiplicación (punto)	.*	-
División	/	-
División (punto)	./	-
División invertida	\	-
División invertida (punto)	.\	-
Potencia	^	-
Potencia (punto)	.^	-
Factorial	-	factorial()
Redondeo	-	round()
Redondeo (cero)	-	fix()
Redondeo ($-\infty$)	-	floor()
Redondeo ($+\infty$)	-	ceil()
Resto	-	rem()
Signo	-	sign()

Como se puede ver, se puede resolver operaciones matemáticas básicas como la suma, resta, multiplicación, división, potencia y raíz. Las operaciones “punto” presentadas en la tabla (multiplicación punto, división punto y potencia punto), para números escalares, son iguales a sus operaciones análogas sin el punto. Las diferencias se verán en el capítulo 5 de vectores y matrices.

```
>> 1+1
ans = 2
>> 1-2
ans = -1
>> 4*6
ans = 24
>> 4.*6
ans = 24
>> 4/2
ans = 2
>> 4./2
ans = 2
>> 4\2
ans = 0.5000
>> 4.\2
ans = 0.5000
```



```
>> 4^2
ans = 16
>> 4.^2
ans = 16
>> factorial(5)
ans = 120
```

Para calcular la raíz se puede introducir exponentes fraccionales. También el exponente puede ser negativo.

```
>> 4^0.5
ans = 2
>> 4.^0.5
ans = 2
>> 4^-0.5
ans = 0.5000
>> 4.^-0.5
ans = 0.5000
```



Recordar que el exponente fraccional es equivalente a la raíz, por lo que para calcular una potencia o una raíz se usa el mismo operador (^).

Octave y Scilab permiten diferentes tipos de redondeo. Los redondeos son:

Redondeo al entero más cercano:

```
>> round(1.1)
ans = 1
>> round(1.49)
ans = 1
>> round(1.51)
ans = 2
>> round(-0.49)
ans = 0
>> round(-0.51)
ans = -1
```

Redondeo hacia el cero:

```
>> fix(1.1)
ans = 1
>> fix(1.49)
ans = 1
>> fix(1.51)
ans = 1
>> fix(-0.49)
ans = 0
>> fix(-0.51)
ans = 0
```

Redondeo al entero más cercano:

```
>> floor(1.1)
ans = 1
>> floor(1.49)
ans = 1
>> floor(1.51)
ans = 1
>> floor(-0.49)
ans = -1
>> floor(-0.51)
ans = -1
```

Redondeo al entero más cercano:

```
>> ceil(1.1)
ans = 2
>> ceil(1.49)
ans = 2
>> ceil(1.51)
ans = 2
>> ceil(-0.49)
ans = 0
>> ceil(-0.51)
ans = 0
```

La función `rem` (que viene de `remainder`, que significa “resto”), permite calcular el resto de la división de dos números.

```
>> rem(1,2)
ans = 1
```

Da 1, que es el resto de la división 1 sobre 2.

```
>> rem(4,2)
ans = 0
```

Da 0, que es el resto de la división 4 sobre 2.

Y ahora aplico la función `sign`, que permite obtener el signo de un número. Se puede utilizar una operación matemática como input.

```
>> sign(-8)
ans = -1
>> sign(47.59)
ans = 1
>> sign(-1+2^2)
ans = 1
```

Capítulo 2

Operaciones básicas



OCTAVE



A continuación, se estudiarán algunos operadores y comandos básicos. Esos que están presentes en casi todos los códigos.

2.1 El operador colon y notación científica

El operador colon (u operador dos puntos) permite crear sucesiones de números (en el capítulo 5 de vectores y matrices se verá que es un vector). Lo que hace es presentar todos los números enteros que le pida en un determinado intervalo.

```
>> 0:10
ans =

     0     1     2     3     4     5     6     7     8     9    10
>> -3:15
ans =

    -3    -2    -1     0     1     2     3     4     5     6     7     8     9
10    11    12    13    14    15
```

Además, puedo informarle cual tiene que ser la diferencia entre dos números sucesivos.

```
>> 0:0.5:10
ans =

Columns 1 through 10:

     0     0.5000     1.0000     1.5000     2.0000     2.5000
3.0000     3.5000     4.0000     4.5000

Columns 11 through 20:

     5.0000     5.5000     6.0000     6.5000     7.0000     7.5000
8.0000     8.5000     9.0000     9.5000

Column 21:

10.0000
```

La sucesión de números puede ir de menor a mayor, como en los ejemplos anteriores, o del mayor al menor.

```
>> 10:-1:-5
ans =

    10     9     8     7     6     5     4     3     2     1     0    -1    -2    -
3    -4    -5
```



Observar que la sucesión tiene tres inputs (el número en el cual inicia la sucesión, el salto entre números sucesivos y el número en la que termina). Si el primero es mayor al tercero, la sucesión va para atrás.

Finalmente, puede ser que no llegue al final del intervalo como está indicado en los límites.

```
>> 1:3:15
ans =

    1     4     7    10    13
```

También hay que tener cuidado con el salto que se elige, ya que si es mayor al intervalo, el mismo resulta vacío. En el ejemplo, como el salto es de 4 pero la sucesión va de 1 a 3, entonces solo muestra el 1.

```
>> 1:4:3
ans = 1
```

Octave y Scilab nos permiten usar la notación científica. Debe notarse que se usa la letra “e” para representar al “10[^]”.

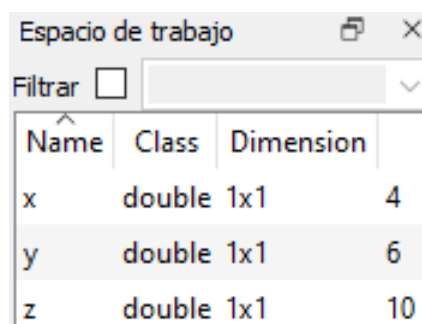
```
>> 1e3
ans = 1000
>> 6e7
ans = 6.0000e+07
>> -49e5
ans = -4900000
>> 0.39e-2
ans = 3.9000e-03
```

2.2 Los comandos clear y clc y la función class

El comando “clear” sirve para borrar determinadas variables. A continuación sigue un ejemplo de como usarlo. Inicialmente, el usuario define las variables “x”, “y” y “z”:

```
>> x=4
x = 4
>> y=6
y = 6
>> z=x+y
z = 10
```

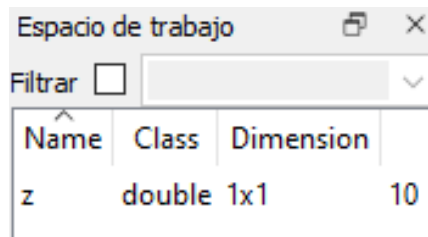
Con esto, el workspace queda como sigue:



Name	Class	Dimension	
x	double	1x1	4
y	double	1x1	6
z	double	1x1	10

Luego, se puede ejecutar la siguiente sentencia, la cual borra las variables “x” e “y”, dejando solo “z”. Esto se observa en el workspace.

```
>> clear x y
```

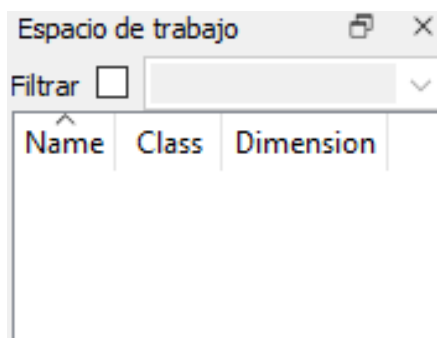


Name	Class	Dimension
z	double	1x1

También se puede usar el comando “clear” sin especificar variables. Así, las borra a todas. Esto es útil cuando tienen que correr un script atrás de otro, para que las variables de un script no se confundan con las de otro. Quizás dos scripts usan los mismos nombres de variables y empiezan a sobrescribirse. Esto produce errores.

A continuación se presentan dos sentencias en las cuales se vuelven a definir las variables “x” e “y”. Así, el usuario volverá a tener en el workspace a las tres variables iniciales. Luego, en la tercer sentencia se ejecuta el comando “clear”. Como puede verse, el workspace queda vacío.

```
>> x=4
x = 4
>> y=6
y = 6
>> clear
```



Name	Class	Dimension
------	-------	-----------

También el usuario puede ejecutar el comando “clc”. Después de muchos cálculos la ventana de comandos puede estar llena de resultados y eso puede ser molesto a la hora de realizar nuevos cálculos. En ese caso, el comando “clc” sirve para borrar la pantalla.

```
>> clc
```

```
Ventana de comandos

Columns 12 through 21:
    5.5000    6.0000    6.5000    7.0000    7.5000    8.0000    8.5000    9.0000    9.5000   10.0000

>> 10:-1:-5
ans =
    10     9     8     7     6     5     4     3     2     1     0    -1    -2    -3    -4    -5

>> 1:3:15
ans =
     1     4     7    10    13

>> 1:4:3
ans = 1
>> 1e3
ans = 1000
>> 6e7
ans = 6.0000e+07
>> -49e5
ans = -4900000
>> 0.39e-2
ans = 3.9000e-03
>> x=4
x = 4
>> y=6
y = 6
>> z=x+y
z = 10
>> clear x y
>> x=4
x = 4
>> y=6
y = 6
>> clear
>>
```

```
Ventana de comandos
>> |
```

En Octave, la función “class” se puede usar para conocer información de la variable que estamos tratando.

```
>> class(4)
ans = double
>> class(1==1)
ans = logical
```

En el primer caso informa que es un double y el segundo da un logical. También se pueden introducir strings, que es un grupo de chars. Los strings se crean entre comillas.

```
>> class("casa")
ans = char
>> class("4")
ans = char
```

En cuanto a Scilab tenemos que usar la función “typeof” en lugar de class. Hubiesen dado “constant”, “boolean”, “string” y “string”. “constant” es análogo a doubles, “boolean” es análogo a logical y “string” significa cadena de “chars”.

2.3 Interrumpir un cálculo

Cuando Octave y Scilab estan calculando tengo la ventana de comandos ocupada y no puedo hacer cálculos. A veces puede pasar que un script tiene algún error y puede demorar mucho el cálculo o seguir calculando infinitamente. También puede pasar que, sin saberlo, le pidamos un cálculo que demande demasiado tiempo. En estos casos es preferible interrumpir el cálculo. Para eso se debe apretar Ctrl + C en el teclado y deja de calcular. Si el usuario quiere probar esto es su computadora debe introducir “rand(1000000,1)” y apretar Ctrl + C.

2.4 Los comandos help, error, exist y como introducir comentarios

El comando “help” sirve para saber información de la función que nos interesa. Debemos introducir help seguido del nombre de la función sobre la que nos interesa informarnos.

```
>> help round
'round' is a built-in function from the file
libinterp/corefcn/mappers.cc

-- round (X)
   Return the integer nearest to X.

   If X is complex, return 'round (real (X)) + round (imag (X)) *
I'.
   If there are two nearest integers, return the one further away
from
   zero.

       round ([-2.7, 2.7])
           => -3      3

   See also: ceil, floor, fix, roundb.
```

Así, se obtiene información sobre la función, como se usa, que resultados se obtiene, que variables acepta, ejemplos de su uso y funciones relacionadas.

En los scripts que me armo también puedo poner un comentario. Eso se hace introduciendo un % en donde quiero escribirlo. En Scilab es exactamente igual, solo reemplazando “//” en lugar de “%”. En el siguiente script de Octave se observa en verde un comentario. Este tiene como función explicar algo sobre alguna función o sentencia, o incluso decir algo del script en general.


```
Script_Prueba.m x myerror.m x
1 a=1
2 b=2
3 c=a+b
4 error('el programa se termina acá') %Esta sentencia interrumpe la corrida
5 d=2*c
```

En este caso explica para qué sirve la función “error”. Al llegar el software a la ejecución de la misma, el programa se interrumpe, emitiendo en la ventana de comandos la línea que se encuentra entre paréntesis. Esto se observa a continuación:

```
>> myerror
a = 1
b = 2
c = 3
error: el programa se termina acá
error: called from
      myerror at line 4 column 1
```



Observe que el mensaje de error en la ventana de comandos indica en que línea se interrumpió la corrida.

Sale el mensaje de error que informa en que línea se cortó la corrida. Se debe notar que todas las líneas que vienen después de la línea de error no se activan.

También puedo chequear la existencia de una variable usando el comando “exist”.

```
>> a=1
a = 1
>> exist a
ans = 1
>> exist d
ans = 0
```

La primera da 1 porque la variable “a” existe y la segunda da 0 porque “d” no existe. Debe notarse que estos son unos y ceros lógicos. En Scilab el comando que hay que usar es “exists” en lugar de “exist”.

2.5 Las funciones isscalar, isvector e ismatrix

También se puede chequear que una variable es escalar, vector o matriz.

```
>> a=10
a = 10
>> isscalar(a)
ans = 1
>> isscalar("casa")
ans = 0
>> isvector(a)
ans = 1
>> ismatrix(a)
ans = 1
```

La primera da 1 porque es escalar y la segunda da 0 porque no lo es. Y ya que Octave y scilab toman a los vectores como matrices con una de las dimensiones 1, y a los escalares como vectores de 1 x 1, entonces un escalar también resulta un vector y una matriz.

Debe tenerse en cuenta que los unos y ceros obtenidos son del tipo lógico. Es decir, son booleanos. En el caso de Scilab se obtendrán “T” o “F”. Una explicación de este tipo de variables se encuentra en el siguiente capítulo.

Se puede operar de un modo similar con las funciones “isvector” e “ismatrix”. Debe tenerse en cuenta que el vector “A” da 1 en “isvector” e “ismatrix” pero da 0 en “isscalar”, por lo que el programa considera a al vector como una matriz en que una de las dos dimensiones es 1. En cambio, un vector no es un escalar.

```
>> A=[1 2]
A =

    1    2

>> isscalar(A)
ans = 0
>> isvector(A)
ans = 1
>> ismatrix(A)
ans = 1
```

Al usar la variable “M”, que es una matriz, solo la función “ismatrix” da 1, mientras que las otras dos dan 0.

```
>> M=[1 2;3 4]
M =

    1    2
    3    4

>> isscalar(M)
ans = 0
>> isvector(M)
ans = 0
>> ismatrix(M)
ans = 1
```

2.6 Uso de paréntesis

El paréntesis se puede usar para darle prioridad a determinadas operaciones respecto a otras, tal cual ocurre en matemática. De no haber paréntesis, Octave y Scilab siguen el orden definido en el capítulo 6, el cual coincide con el de las operaciones del álgebra de la matemática.

```
>> (1+3)*8
ans = 32
>> 1+3*8
ans = 25
```

2.7 Cambio de tipo de variable

Cuando se define una variable, esta se inicializa como double. Sin embargo, se puede convertir en otro tipo usando las siguientes funciones:

Operación	Rango	Función	
		Scilab	Octave
Convertir a integer	$-2^7 - 2^7 - 1$	int8()	
	$-2^{15} - 2^{15} - 1$	int16()	
	$-2^{31} - 2^{31} - 1$	int32()	
	$-2^{63} - 2^{63} - 1$	int64()	
	$0 - 2^8 - 1$	uint8()	
	$0 - 2^{16} - 1$	uint16()	
	$0 - 2^{32} - 1$	uint32()	
	$0 - 2^{64} - 1$	uint64()	
Convertir a single	$-3,4.10^{38} - 3,4.10^{38}$	-	single()
Convertir a double	$-1,79.10^{308} - 1,79.10^{308}$	double()	
Convertir a logical	0 - 1	-	logical()

A continuación se presenta un ejemplo donde se cambia el tipo de la variable “a” de double a integer de 8 bits.

```
>> a=1
a = 1
>> class(a)
ans = double
>> a=int8(a)
a = 1
>> class(a)
ans = int8
```

Pero las variables tienen un cierto rango. Si se convierte un número double en una variable que supera este rango entonces me da el máximo o mínimo valor del rango. En otras palabras, si se tiene la variable “a” igual a 1000 y esta se convierte a integer de ocho bits (cuyo máximo valor puede ser 127) entonces el número que se obtiene en la transición es un integer y vale 127.

```
>> int8(1000)
ans = 127
```

De un modo similar al anterior, en Octave se puede usar la función “logical” para convertir un double en un vector o matriz logical. Debe tenerse en cuenta que esta operación puede ser confusa, ya que todo coeficiente del vector o matriz será convertido en un 0 o 1. El criterio que se toma es que si un número double o integer es distinto de cero entonces se transformará en un 1 logical. El 0 double o integer se convierte en un 0 logical. A continuación se presenta un ejemplo.

```
>> A=[-2:5]
A =

    -2    -1     0     1     2     3     4     5
```

```
>> logical(A)
ans =

1  0  1  1  1  1  1
```

En Scilab se utilizan todos estos comandos de un modo similar pero hay dos diferencias:

- No hay una función que convierta una variable a single. En este caso lo ideal sería usar la función double.
- No hay una función para convertir un vector de doubles en logicals.

Capítulo 3

Operadores lógicos y de comparación



OCTAVE



En las operaciones lógicas o de comparación no se obtienen resultados “numéricos”, sino lógicos. En Octave, los resultados lógicos son 1 y 0, que representan al verdadero y falso, respectivamente. En Scilab, en cambio, se obtienen T y F, por True y False. Las operaciones que se usan en Octave y Scilab van a ser las mismas, solo cambiando en el resultado un 1 por un T y un 0 por un F. A continuación se presentan dichas operaciones:

Operación	Símbolo	Ejemplo	
			Octave
Igual	==	$1 == 1 \rightarrow T$ $2 == 1 \rightarrow F$	$1 == 1 \rightarrow 1$ $2 == 1 \rightarrow 0$
Distinto	~=	$1 \sim 1 \rightarrow F$ $2 \sim 1 \rightarrow T$	$1 \sim 1 \rightarrow 0$ $2 \sim 1 \rightarrow 1$
Mayor	>	$1 > 1 \rightarrow F$ $2 > 1 \rightarrow T$	$1 > 1 \rightarrow 0$ $2 > 1 \rightarrow 1$
Mayor o igual	>=	$1 >= 1 \rightarrow T$ $2 >= 1 \rightarrow T$	$1 >= 1 \rightarrow 1$ $2 >= 1 \rightarrow 1$
Menor	<	$0 < 1 \rightarrow T$ $2 < 1 \rightarrow F$	$0 < 1 \rightarrow 1$ $2 < 1 \rightarrow 0$
Menor o igual	<=	$0 <= 1 \rightarrow T$ $1 <= 1 \rightarrow T$	$0 <= 1 \rightarrow 1$ $1 <= 1 \rightarrow 1$
No	~	$\sim(T) \rightarrow F$ $\sim(F) \rightarrow T$	$\sim(1) \rightarrow 0$ $\sim(0) \rightarrow 1$
And	&	$T \& T \rightarrow T$	$1 \& 1 \rightarrow 1$
Or		$T F \rightarrow T$	$1 0 \rightarrow 1$

La operación igual consiste en comparar dos números. Si dichos números son iguales entonces obtenemos como resultado un 1 lógico o un T. Si los números son distintos entonces el resultado es un 0 lógico o un F.

```
>> (8+1) == (4+5)
ans = 1
>> (8+1) == (4)
ans = 0
```

La operación distinto consiste en comparar dos números y dar 1 lógico o T si son distintos y 0 o F si son iguales.

```
>> (8+1) ~= (4+4)
ans = 1
>> (8+1) ~= (4+4+1)
ans = 0
```

La operación mayor consiste en comparar dos números y dar 1 lógico o T si el de la izquierda es mayor al de la derecha y 0 o F si no es así.

```
>> (8+1) > (4+4)
ans = 1
```

```
>> (8+1) > (4+4+1)
ans = 0
```

La operación menor consiste en comparar dos números y dar 1 lógico o T si el de la izquierda es menor al de la derecha y 0 o F si no es así.

```
>> (8+1) < (4+4)
ans = 0
>> (8+1) < (4+4+2)
ans = 1
```

La operación mayor o igual consiste en comparar dos números y dar 1 lógico o T si el de la izquierda es mayor o igual al de la derecha y 0 o F si no es así.

```
>> (8) >= (4+4)
ans = 1
>> (8+1) >= (4+4+2)
ans = 0
```

La operación menor o igual consiste en comparar dos números y dar 1 lógico o T si el de la izquierda es menor o igual al de la derecha y 0 o F si no es así.

```
>> (8+1) <= (4+4)
ans = 0
>> (8+1) <= (4+4+2)
ans = 1
```

El operador “not” cambia el 1 o T por el 0 lógico o F y al revés.

Finalmente están los operadores “and” y “or”. Estos requieren de dos inputs, al menos. El operador “and” da 1 o T si todos los inputs son 1 o T, mientras que si alguno de ellos es 0 o F entonces el resultado será 0 o F.

```
>> 1==1 & 0==0
ans = 1
>> 1==1 & 1==0
ans = 0
>> 1==0 & 8==0
ans = 0
```

Por otro lado, el operador “or” resulta en un 1 o T si alguno de sus inputs es 1 o T y si todos son 0 o F resulta 0 o F.

```
>> 1==1 | 0==1
ans = 1
>> 1==2 | 0==1
ans = 0
```

El primero da como resultado 1 o T porque uno de los dos argumentos es correcto (1==1) mientras que el segundo da 0 porque ambos argumentos son erróneos. **La operatoria en Scilab es igual y puede realizarla el alumno. La única diferencia es que se van a obtener T y F en lugar de 1 y 0.**

Para mayor completitud, a continuación se presenta la tabla de verdad de los operadores “and” y “or”.

Entrada 1		Entrada 2	Salida	
			Scilab	Octave
1 / T	& (and)	1 / T	T	1
1 / F		0 / F	F	0
0 / F		1 / T	F	0
0 / F		0 / F	F	0
Entrada 1		Entrada 2	Salida	
			Scilab	Octave
1 / T	 (or)	1 / T	T	1
1 / F		0 / F	T	1
0 / F		1 / T	T	1
0 / F		0 / F	F	0

Se debe notar que si a un 1 lógico se le suma un double, automáticamente Octave y Scilab convierten al primero en un double y obtiene un resultado numérico.

```
>> 1==1
ans = 1
>> ans+2
ans = 3
```

Da 3 porque convirtió el 1 lógico en un 1 double. Luego, le sumó 2.

Capítulo 4

Funciones especiales



OCTAVE



4.1 Funciones trigonométricas

Octave y Scilab son softwares de cálculo, por lo que disponen de una amplia gama de funciones para cálculo. Las funciones trigonométricas no son la excepción, estando todas presentes en cada software.

Operación	Símbolo	Función	
		Scilab	Octave
Seno (radianes)	-	sin()	
Seno (grados)	-	sind()	
Coseno (radianes)	-	cos()	
Coseno (grados)	-	cosd()	
Tangente (radianes)	-	tan()	
Tangente (grados)	-	tand()	
Cotangente (radianes)	-	cot()	
Cotangente (grados)	-	cotd()	
Arcoseno (radianes)	-	asin()	
Arcoseno (grados)	-	asind()	
Arcocoseno (radianes)	-	acos()	
Arcocoseno (grados)	-	acosd()	
Arcotangente (radianes)	-	atan()	
Arcotangente (grados)	-	atand()	
Arcocotangente(radianes)	-	acot()	
Arcocotangente (grados)	-	acotd()	

Debe notarse que las funciones pueden usar como argumento (input) los radianes o grados (por ejemplo, el sin o sind, respectivamente) mientras que otras dan el resultado (output) en radianes o grados (por ejemplo, atan o atand, respectivamente).

```
>> sin(90)
ans = 0.8940
>> sin(pi/2)
ans = 1
>> sind(90)
ans = 1
>> sind(pi/2)
ans = 0.027412
```

Notar que sin(90) no da como resultado 1 ya que interpreta al input 90 como 90 radianes. Por su parte, sin(pi/2) es igual a 1 ya que se toma como radianes. En las dos últimas operaciones los resultados input se interpretan en grados y eso explica los resultados.

4.2 Otras funciones matemáticas

Otras funciones, como las mencionadas “atan” y “atand” informan los resultados en radianes o grados, respectivamente. Teniendo en cuenta que matemáticamente el arcotangente de 1 equivale a 45° o 0.785. eso explica los siguientes resultados.

```
>> atan(1)
ans = 0.7854
>> atand(1)
ans = 45
```

Notar que los resultados obtenidos son 0.7853982 y 45.



En Octave, ya está cargada la variable “pi”, que contiene a π . Por otro lado, Scilab también tiene esta variable pero debe introducirse como %pi.

Operación	Función	
	Scilab	Octave
Seno hiperbólico	sinh()	
Coseno hiperbólico	cosh()	
e^x	exp()	
Logaritmo natural (o neperiano)	log()	
Logaritmo en base 2	log2()	
Logaritmo en base 10	log10()	
Raíz cuadrada	sqrt()	
Primera función de bessel modificada	besseli()	
Primera función de bessel	besselj()	
Función error	erf()	
Función error complementaria	erfc()	
Función beta	beta()	
Función gamma	gamma()	
Espacio equiespaciado	linspace()	

Acá tenemos algunas de las funciones. Más allá de las trigonométricas, que tiene cargado Octave y Scilab y pueden ser útiles. No son difíciles de usar. Simplemente hay que introducir como input el valor matemático en que se desea calcular la función. A continuación, algunas observaciones de estas funciones.

- “sinh” y “cosh” calculan el seno y coseno hiperbólico.

- El comando `exp()` seria equivalente a $e^{\cdot}$. En Scilab, para poder usar el número irracional e debemos escribir `%e`.

```
>> exp(2)
ans = 7.3891
>> e^2
ans = 7.3891
```

- Octave y Scilab incluyen funciones para calcular el logaritmo neperiano o natural (`log`), en base 2 (`log2`) o en base 10 (`log10`). Se puede calcular el logaritmo de x en cualquier base b usando la propiedad $\log_b(x) = \ln(x) / \ln(b)$. Es decir, por medio de la línea `log(x)/log(b)`.
- `Sqrt` quiere decir “square root”. Es decir que `sqrt(4)` da 2.

```
>> sqrt(4)
ans = 2
```

- Octave y Scilab incluyen las funciones de Bessel. Las mismas están asociadas a resolución de ecuaciones diferenciales y son complejas de usar, ya que requieren más de un input. El usuario debe conocerlas bien para usarlas correctamente. Además de las dos funciones de Bessel informadas en este trabajo hay otras que tienen cargadas, de distintos órdenes. Se recomienda googlear.
- Las funciones `error` y `errorc` complementarias son funciones matemáticas que también se usan en resolución de ecuaciones diferenciales
- La función `beta` es importante en estadística. En Octave tiene dos inputs. Estos pueden ser dos matrices de iguales dimensiones o una matriz y un escalar. En el primer caso, calcula como output un vector que resulta de combinar uno a uno cada par de elementos de los dos vectores (es decir, usa los dos primeros elementos de las dos matrices como inputs para obtener un output, luego los dos segundos, etc.). En la segunda opción (una matriz y un escalar como inputs) toma al escalar como un vector con todos los elementos iguales. En Scilab solo puede tener como inputs dos matrices de iguales dimensiones y el calculo lo hace como en Octave. Recordar que los escalares en Octave y Scilab son como vectores de dimensión uno y pueden ser usados como inputs de la función `beta` (y en cualquier otra que requiera un vector) y que, además, los vectores son matrices con una de sus dimensiones de valor 1.

```
>> beta(1:3,4:6)
ans =

    2.5000e-01    3.3333e-02    5.9524e-03
>> beta(1:3,4)
ans =

    0.250000    0.050000    0.016667
```

- La función `gamma` también es importante en estadística, pero requiere un solo input. Este tiene que ser un vector. Por eso, a cada elemento del vector le calcula el valor de la función `gamma`.

```
>> gamma(1:3)
ans =

    1    1    2
```

- El comando linspace me permite separar un intervalo en N-1 segmentos. El siguiente ejemplo muestra como se separa el espacio de 0 a 10 en 5 partes (por eso se introduce 6 como tercer output).

```
>> linspace(0,10,21)
```

```
ans =
```

```
Columns 1 through 12:
```

```
          0      0.5000      1.0000      1.5000      2.0000      2.5000
3.0000      3.5000      4.0000      4.5000      5.0000      5.5000
```

```
Columns 13 through 21:
```

```
      6.0000      6.5000      7.0000      7.5000      8.0000      8.5000
9.0000      9.5000     10.0000
```

Capítulo 5

Vectores y matrices



OCTAVE



5.1 Definición, índices y el operador colon con matrices y vectores

Los vectores pueden definirse como cualquier variable. Se pueden usar comas o espacios para separar sus elementos.

```
>> v=[1 2 3]
v =

     1     2     3
```

```
>> v=[1,2,3]
v =

     1     2     3
```

Se generó un vector que tiene tres columnas y una fila. Puedo definir uno que tiene tres filas y una columna escribiendo lo siguiente (se reemplazó el espacio o la coma por punto y coma). El vector es vertical.

```
>> v=[4;5;6]
v =

     4
     5
     6
```

Esto es importante para operaciones entre matrices, porque Octave no transpone un vector si no se lo pide el usuario. Una matriz se define en forma parecida.

```
>> A=[1 2 3;4 5 6;7 8 9]
A =

     1     2     3
     4     5     6
     7     8     9
```

También se puede usar comas para separar entre columnas.

Además, se puede acceder a los coeficientes de cada vector y matriz.

```
>> v(2)
ans = 5
>> A(2,2)
ans = 5
```

De este modo se accede a la coordenada 2 del vector (que es casualmente 2) y a la coordenada (2,2) de la matriz, que es 5.

También se puede acceder a una fila o columna de una matriz o vector usando el símbolo “:”

```
>> A(:,2)
ans =

     2
     5
```

```

      8
>> A(2,:)
ans =

      4      5      6

```

Así se accede a la columna 2 y a la fila 2, respectivamente. Además se puede usar con el comando end (en Scilab, en lugar de “end” se usa \$). “end” significa la última columna o última fila.

```

>> A
A =

      1      2      3
      4      5      6
      7      8      9

```

```

>> A(end,1)
ans = 7

```

Se accedió al último coeficiente de la columna. También se puede usar end como si fuese un cálculo combinado, donde end es el valor de la última fila o columna. En el siguiente caso, “end” equivale a 3.

```

>> A(2*end-4,1)
ans = 4

```

Es decir, se accede a la fila $2*3-4=2$ y a la columna 1. Para vectores aplica lo mismo, pero cuidado que es una matriz con una de sus dimensiones en 1.

El operador colon (o dos puntos) se usa para generar vectores y matrices. Por ejemplo, la siguiente:

```

>> v=1:10
v =

      1      2      3      4      5      6      7      8      9     10

```

Se generó un vector que tiene 10 columnas y en cada uno aparece un número del 1 al 10. Y se puede usar este operador para armar una matriz, como la siguiente, que resulta de 3 por 5.

```

>> A=[1:5;6:10;11:15]
A =

      1      2      3      4      5
      6      7      8      9     10
     11     12     13     14     15

```

Otra forma de usarlo es la siguiente, en la cual salta de a dos.

```

>> B=[1:2:5;6:2:10;11:2:15]
B =

      1      3      5
      6      8     10
     11     13     15

```


Si lo que se busca es transponer una matriz entonces se realiza del siguiente modo:

```
>> B'  
ans =
```

```
1     6     11  
3     8     13  
5    10     15
```

5.2 Operaciones de matrices (suma, resta, multiplicación y potencia)

La suma y resta de matrices en Octave y Scilab es la definida matemáticamente.

```
>> A=[1:2:5;6:2:10;11:2:15]  
A =
```

```
1     3     5  
6     8    10  
11    13    15
```

```
>> B=[1 2 3;4 5 6;7 8 9]  
B =
```

```
1     2     3  
4     5     6  
7     8     9
```

```
>> A+B  
ans =
```

```
2     5     8  
10    13    16  
18    21    24
```

```
>> A-B  
ans =
```

```
0     1     2  
2     3     4  
4     5     6
```

Por otro lado, Octave y Scilab permiten sumar un escalar con una matriz, si bien no está definido matemáticamente. Si a una matriz se le suma 1, por ejemplo, esto equivale a sumarle 1 a cada elemento de ella. Algo análogo aplica a la resta.

```
>> B  
B =
```

```
1     2     3  
4     5     6  
7     8     9
```

```
>> B+1
ans =
```

```
    2    3    4
    5    6    7
    8    9   10
```

Además, Octave permite sumar un vector fila con una matriz, haciendo que cada fila de la matriz sea sumada al vector. Sin embargo, Scilab no permite hacer esto.

```
>> V=[1 2 3];
>> B+V
ans =
```

```
    2    4    6
    5    7    9
    8   10   12
```

Si es un vector columna le suma el vector a cada columna de la matriz.

En Octave y Scilab, la multiplicación de matrices y vectores tiene dos definiciones. Una de ellas es $A*B$ y la otra $A.*B$.

```
>> A*B
ans =
```

```
    48    57    66
   108   132   156
   168   207   246
```

```
>> A.*B
ans =
```

```
    1     6    15
   24    40    60
   77   104   135
```

Cual es la diferencia? La primera es la multiplicación de matrices definida matemáticamente. La conocemos desde el primer año de la facultad, por lo menos. La segunda es la multiplicación coordenada a coordenada o elemento a elemento. Estas dos operaciones son validas para vectores ya que los vectores son matrices con una de las dimensiones en 1. Algo parecido pasa con la potencia. Se puede elevar a cada coeficiente de B al exponente que se desee.

```
>> A.^2
ans =
```

```
    1     9    25
   36    64   100
  121   169   225
```

Se puede también usar la potencia punto, como en el siguiente ejemplo:

```
>> A.^B
ans =

    1.0000e+00    9.0000e+00    1.2500e+02
    1.2960e+03    3.2768e+04    1.0000e+06
    1.9487e+07    8.1573e+08    3.8443e+10
```

Esto implica que cada coeficiente de A es elevado al exponente que resulta de usar el respectivo coeficiente de B.

También se puede hacer multiplicar una matriz por si misma cuantas veces se desee.

```
>> A^2
ans =

    74    92   110
   164   212   260
   254   332   410
```

```
>> A*A
ans =

    74    92   110
   164   212   260
   254   332   410
```

Como puede observarse, A al cuadrado equivale a multiplicar a A por si misma.

Y también se puede calcular A^{-1} . Lo que esta operación permite es calcular la inversa de A. Es una operación análoga a usar la función “inv”.

```
>> A^-1
ans =

    2.8147e+14   -5.6295e+14    2.8147e+14
   -5.6295e+14    1.1259e+15   -5.6295e+14
    2.8147e+14   -5.6295e+14    2.8147e+14
```

```
>> inv(A)
ans =

    2.8147e+14   -5.6295e+14    2.8147e+14
   -5.6295e+14    1.1259e+15   -5.6295e+14
    2.8147e+14   -5.6295e+14    2.8147e+14
```

5.3 División de matrices

En esta sección se estudia la división de matrices. Es importante tener en cuenta que en la matemática la división de matrices no está definida, por lo que acá se verá son solo operaciones computacionales dentro de Octave y Scilab. Aun así, resultan muy útiles para resolver problemas, especialmente en la resolución de sistemas de ecuaciones lineales.

Dependiendo de las columnas o filas de las matrices involucradas puedo tener distintos resultados y hasta realizar operaciones distintas. Vamos a empezar con la operación B/A , que implica hallar “x” tal que $x \cdot A = B$. Dependiendo de la forma de A y B se van a realizar distintos cálculos.

Si A es escalar y B matriz, entonces esto equivale a una matriz B dividido cada componente de A. Es la división de una matriz con un escalar.

```
>> A=2
A = 2
>> B=[1 2;3 4]
B =

     1     2
     3     4

>> B/A
ans =

     0.5000     1.0000
     1.5000     2.0000
```

Si A es de $N \times N$ y B tiene N columnas entonces $B/A = B \cdot \text{inv}(A)$. Esto significa que si A y B son $N \times N$ entonces B/A calcula $B \cdot \text{inv}(A)$. Como puede observarse en la figura de abajo, B/A dio el mismo resultado que $B \cdot \text{inv}(A)$. Además, con la operación $x \cdot A - B$ se comprueba que x cumple que $x \cdot A = B$.

```
>> A=[1 3.3 2;4 6 5;7 9 8];
>> B=[1 1.5 14;3 3 2;-2 0 4];
>> x=B/A
x =

    -85.0000    200.0000   -102.0000
     6.6667   -16.6667     9.0000
    -33.3333     81.3333    -42.0000

>> B*inv(A)
ans =

    -85.0000    200.0000   -102.0000
     6.6667   -16.6667     9.0000
    -33.3333     81.3333    -42.0000

>> x*A-B
ans =

         0         0         0
-7.1054e-15         0         0
         0         0         0
```

Si A es de $M \times N$ y B tiene N columnas entonces B/A da la solución por cuadrados mínimos de ese sistema.

```
>> A=[1 4 7 3 6.4;3.3 6 9 6 5;2 5 8 7.7 9];
>> B=[4 8 9 4 3;5 1 4 5 97; 6 3 41 3 5];

>> x=B/A
```

x =

```
0.3058    1.5521   -0.7751
11.8145  -16.8338   10.4888
5.2911    3.6153   -4.8820
```

>> x*A-B

ans =

```
-0.1226   -1.3401    0.9080    0.2612   -0.2589
-27.7595  -2.3008   11.1077   10.2044  -11.1570
1.4574   15.4458  -10.4812   -3.0266    3.0011
```



El resultado de la división en Octave y Scilab depende de los operandos, por lo que resulta confuso. Se recomienda evitar el uso de la división usando otras funciones. Por ejemplo, “lsqlin” en Octave o “lsq” en Scilab para realizar un análisis por cuadrados mínimos.

También se puede usar la operación $A \setminus B$. Esto implica resolver el sistema $A \cdot x = B$. Si A es un escalar, esta operación implica dividir cada componente de B por A. Esto es igual al caso anterior. Además, si A es una matriz de $N \times N$ y B tiene N filas entonces $A \setminus B$ es equivalente a resolver el sistema $A \cdot x = B$. Y como A es cuadrada de $N \times N$ entonces se cumple que $A \setminus B$ es igual a $\text{inv}(A) \cdot B$.

```
>> A=[1 3.3 2;4 6 5;7 9 8];
>> B=[1 1.5 14;3 3 2;-2 0 4];
>> x=A\B
```

x =

```
-34.667   -23.000    48.000
-23.333   -15.000    46.667
56.333    37.000   -94.000
```

```
>> inv(A)*B
```

ans =

```
-34.667   -23.000    48.000
-23.333   -15.000    46.667
56.333    37.000   -94.000
```

```
>> A*x-B
```

ans =

```
0 0 0
7.1054e-15 0 0
0 0 -1.1369e-13
```

Si A es de $M \times N$ y B tiene N filas entonces $A \setminus B$ da la solución por cuadrados mínimos de ese sistema. El cálculo se presenta a continuación.

```
>> A=[1 3.3 2;4 6 5;7 9 8;3 6 7.7;6.4 5 9];
>> B=[4 5 6;8 1 3;9 4 41;4 5 3;3 97 5];
```

```
>> x=A\B
x =

    0.3058    11.8145    5.2911
    1.5521   -16.8338    3.6153
   -0.7751    10.4888   -4.8820
>> A*x-B
ans =

   -0.1226   -27.7595    1.4574
   -1.3401    -2.3008   15.4458
    0.9080    11.1077  -10.4812
    0.2612    10.2044   -3.0266
   -0.2589   -11.1570    3.0011
```

Finalmente, queda solo ver las operaciones “./” y “.\”, las cuales se realizan entre dos matrices A y B de iguales dimensiones. La operación A./B equivale a dividir cada componente de A por cada componente de B. En cambio, A.\B implica dividir a cada componente de B por cada componente de A. esto se presenta en la siguiente imagen.

```
>> A=[1 2;3 4];
>> B=[2 4;6 8];
>> x1=A.\B
x1 =

    2    2
    2    2

>> x2=A./B
x2 =

    0.5000    0.5000
    0.5000    0.5000
```

5.4 Funciones especiales propias de matrices y vectores

En esta sección se analizan algunas funciones aplicables a vectores y matrices. La función “norm” es la norma de un vector.

```
>> norm([1 1 1])
ans = 1.7321
```

También se puede calcular la norma de una matriz. La norma de una matriz es el máximo valor singular de esta. Los valores singulares se calculan con la función “svd”. Notese que en el siguiente ejemplo el máximo valor singular es igual a la norma de la matriz.

```
>> A=[1 3.3;4 6;7 9]
A =

    1.0000    3.3000
    4.0000    6.0000
    7.0000    9.0000

>> norm(A)
```

```
ans = 13.871
```

```
>> svd(A)
ans =

    13.8708
     1.2206
```

Pero la función “norm” también acepta un segundo input, el cual nos permite determinar que norma podemos usar¹. En este ejemplo vamos a usar la norma infinito (que equivale a tomar el máximo valor de la suma de cada fila), por lo que se introduce Inf como input. El resultado se presenta a continuación:

```
>> norm(A, Inf)
ans = 16
```

Notar que en Scilab hay que introducir Inf entre comillas.

Si aplico el comando length a un vector se obtiene la dimensión del mismo, y si se aplica a una matriz de MxN, en Octave se obtiene la mayor de las dos dimensiones y en Scilab se obtiene M*N.

```
>> length([1 1 1])
ans = 3
>> A=[1:5;6:10;11:15]
A =

     1     2     3     4     5
     6     7     8     9    10
    11    12    13    14    15

>> length(A)
ans = 5
```

La función “size” da las dimensiones de una matriz. De acá se deduce que para un vector una de las dimensiones es 1.

```
>> size(A)
ans =

     1     5
```

El comando sum en Octave lo que hace es sumar los coeficientes de cada columna de una matriz. En cambio, para Scilab se obtiene la suma de todos los coeficientes de la matriz. Para vectores, todos los softwares se obtiene la suma de las componentes.

```
>> sum(A)
ans =

    18    21    24    27    30
```

¹ Hay que recordar que en álgebra no hay una única norma sino que hay muchas. Por ejemplo, la norma 1 de un vector equivale a sumar el valor absoluto de todos sus componente, mientras que la norma dos es la norma euclídea (o pitagórica). Para más información, introducir “help norm” en la ventana de comandos.

En Octave, el comando max busca el máximo de cada columna y da como resultado un vector con estos valores. En cambio, en Scilab, max te devuelve el máximo de todos los coeficientes. Si en Octave se introduce [a,b]=max(A), se obtiene en “a” el mencionado vector y en cada “b” en qué posición de cada columna se encuentra. En cambio, en Scilab se obtiene el máximo de la matriz y en qué fila y columna se encuentra. Como es de esperar, la función “min” es análoga a max en Octave y Scilab pero buscando el mínimo en lugar del máximo.

```
>> max(A)
ans =

    11    12    13    14    15
>> [a,b]=max(A)
a =

    11    12    13    14    15
b =

     3     3     3     3     3
```

Se pueden crear vectores y matrices de ceros y unos con los comandos “zeros” y “ones”, usando como inputs las dimensiones que se desea que tengas las matrices.

```
>> ones(2,3)
ans =
```

```
 1  1  1
 1  1  1
```

Si se desea crear una matriz que tenga, por ejemplo, ochos en lugar de unos se puede realizar lo siguiente:

```
>> 8*ones(2,3)
ans =
```

```
 8  8  8
 8  8  8
```

También se pueden crear matrices identidad. Por ejemplo, eye(3). Notese que Scilab requiere que en eye se introduzca 3 dos veces para que se genere la matriz identidad de 3x3, mientras que en Octave con uno solo alcanza. eye(3) no da un 1.

```
>> eye(3)
ans =
```

Diagonal Matrix

```
 1  0  0
 0  1  0
 0  0  1
```

```
>> eye(3,3)
ans =
```

Diagonal Matrix


```

1    0    0
0    1    0
0    0    1

```

Se pueden crear matrices diagonales usando la función “diag”, como en el siguiente ejemplo. Es una matriz de 3x3 con esos tres números en la diagonal. También se puede desplazar la diagonal introduciendo un segundo input.

```

>> diag([1 2 3])
ans =

```

Diagonal Matrix

```

1    0    0
0    2    0
0    0    3

```

```

>> diag([1 2 3],1)
ans =

```

```

0    1    0    0
0    0    2    0
0    0    0    3
0    0    0    0

```

```

>> diag([1 2 3],-1)
ans =

```

```

0    0    0    0
1    0    0    0
0    2    0    0
0    0    3    0

```

El comando sort (**llamado gsort en Scilab**) ordena los coeficientes de una matriz o vector sin cambiar las dimensiones. El orden es ascendente en cada columna para Octave **y descendente en Scilab**. Esta función se presenta en el siguiente ejemplo. Además, cuenta con distintas opciones, que pueden encontrarse escribiendo “help sort” o “help gsort” en la ventana de comandos.

```

>> A=[1 3.3 2;4 6 5;7 9 8];
>> sort(A)
ans =

```

```

1.0000    3.3000    2.0000
4.0000    6.0000    5.0000
7.0000    9.0000    8.0000

```

```

--> A=[1 3.3 2;4 6 5;7 9 8];

```

```

--> gsort(A)
ans =

```

```

9.    6.    3.3
8.    5.    2.
7.    4.    1.

```

En Octave se pueden calcular los autovalores y autovectores de una matriz usando el comando eig. En cambio, en Scilab se debe usar la función “spec”.

```
>> B=[1 2 3;4 5 6;7 8 9];
>> eig(B)
ans =

    1.6117e+01
   -1.1168e+00
   -1.3037e-15
```

```
--> B=[1 2 3;4 5 6;7 8 9];
```

```
--> spec(B)
ans =

    16.116844 + 0.i
   -1.116844 + 0.i
   -1.046D-15 + 0.i
```

También se pueden calcular los autovectores usando las funciones “eig” y “spec”. Esto se realiza en el siguiente ejemplo para Octave, pero en Scilab solo habría escribir lo mismo, solo reemplazando “eig” por “spec”. En el cálculo de $x*y*x^{-1}$, donde x e y son las matrices de autovectores y autovalores, respectivamente, se puede observar que se obtiene la matriz B, tal como ocurre en el álgebra de matrices.

```
>> [x,y]=eig(B)
x =

   -0.231971   -0.785830    0.408248
   -0.525322   -0.086751   -0.816497
   -0.818673    0.612328    0.408248

y =

Diagonal Matrix

    1.6117e+01    0    0
         0   -1.1168e+00    0
         0         0   -1.3037e-15

>> x*y*x^-1
ans =

    1    2    3
    4    5    6
    7    8    9
```

El comando reshape cambia las dimensiones de la matriz sin cambiar los coeficientes. Esto puede observarse en el ejemplo siguiente, donde se convierte una matriz de 3x3 en un vector de 9x1 y de 1x9. Notar que en Scilab se usa la función “matrix” en lugar de “reshape”, con los mismos inputs.

```
>> B
B =
```

```
1  2  3
4  5  6
7  8  9
```

```
>> reshape(B,1,9)
ans =
```

```
1  4  7  2  5  8  3  6  9
```

```
>> reshape(B,9,1)
ans =
```

```
1
4
7
2
5
8
3
6
9
```

```
--> B
B =
```

```
1.  2.  3.
4.  5.  6.
7.  8.  9.
```

```
--> matrix(B,1,9)
ans =
```

```
1.  4.  7.  2.  5.  8.  3.  6.  9.
```

```
--> matrix(B,9,1)
ans =
```

```
1.
4.
7.
2.
5.
8.
3.
6.
9.
```

Capítulo 6

Precedencia



OCTAVE



6.1 Precedencia:

La precedencia define en que orden se realizan las operaciones matemáticas. Por ejemplo, que la multiplicación se realiza antes que la suma, como se vio en matemáticas. Este tema parece complicado pero cuando lo empieza a analizar sigue la lógica de los cálculos combinados de las matemáticas. Es decir, es a lo que estamos acostumbrados.

1. Parentesis. Es lo que se utiliza para que el programa realice esa operación antes que cualquier otra.
2. Transposición (.'), potencia (.^), transposición conjugada compleja ('), potencia de matriz (^). En matrices complejas al transponerlas (en realidad le decimos matriz hermitica) se transpone y cambian los componentes imaginarios por sus conjugados. En Octave y Scilab, la transposición punto (.') es la transposición de una matriz o vector sin aplicar el conjugado, mientras que la transposición conjugada compleja (') es transponer y poner el conjugado. Se observa en el siguiente ejemplo.

```
>> A=[1 16+i;3+i 4]
```

```
A =
```

```
1 + 0i    16 + 1i
3 + 1i     4 + 0i
```

```
>> A'
```

```
ans =
```

```
1 - 0i    3 - 1i
16 - 1i   4 - 0i
```

```
>> A.'
```

```
ans =
```

```
1 + 0i    3 + 1i
16 + 1i   4 + 0i
```

3. Potencia con menos unario (.^-), potencia con más unario (.^+) o negación lógica (.^~) además de una potencia de matriz con menos unario (^-), potencia de matriz con más unario (^+) o negación lógica (^~). Las operaciones unarias son las que involucran un solo número. Por ejemplo, -3, que el usuario lo ve como -3 pero en realidad es un 3 al que se le aplica un menos y da -3. Es decir, hay una diferencia entre lo que ve el usuario, que sabe matemáticas, y lo que ven Octave y Scilab, que solo sigue ordenes. La potencia con menos unario, por ejemplo, es que si se tiene una potencia y el exponente es ^-3 entonces primero se cambia el 3 en -3 y después se eleva la base por el exponente -3.
4. Más unario (+), menos unario (-), negación unaria(~). Esto es lo mismo que si hay un - y un 3 y pensarlo como -3. Esta sería la operación “menos unario”. La operación “más unario” es tomar a +3 como 3. La negación lógica es que el 1 se convierte en 0 y al 0 en 1. Esto se puede hacer en matrices o vectores de valores lógicos

5. Multiplicación (*), división derecha(/), división izquierda (.\), multiplicación de matrices (*), división derecha de matrices (/), división izquierda de matrices (\). Todas estas ya fueron vistas en secciones anteriores.
6. Adición (+), sustracción (-). Ya fueron analizadas
7. Operador de dos puntos (:). Esto quiere decir que si se tiene la operación 1+1:4 primero se calcula 1+1=2 y después se aplica el operador colon (:), por lo que se tendría 2:4 y esto da como resultado el vector [2 3 4]. Si se introdujese un paréntesis en 1:4 primero se realiza la operación con el operador “:” y daría como resultado [2 3 4 5].

```
>> 1+1:4
ans =
```

```
2    3    4
```

```
>> 1+(1:4)
ans =
```

```
2    3    4    5
```

8. Menor que (<), menor o igual que (<=), mayor que (>), mayor que o igual a (>=), igual a (==), no es igual a (~=). En este punto aparecen las operaciones lógicas, que se estudian en el capítulo 3. Si se calcula 1+1>=1 se obtiene el 1 lógico ya que primero suma y resulta en 2>1 y esto es cierto, por lo que el resultado es un 1 lógico.

```
>> 1+1>1
ans = 1
```

Después, en orden de precedencia siguen los comandos and y or y estos son los últimos. Los de los puntos 9 y 10 son operaciones en que se realizan todas las comparaciones. Por ejemplo, 1>1 & 2>3 se realizan todas independientemente de si alguna de ellas da 0 (entonces ya conozco el resultado de la operación “and” porque si una resulta 0 entonces no es necesario hacer el resto, porque se sabe que el resultado es 0). Con cortocircuito significa que si hay un símbolo “&&” y de una de las comparaciones es 0 entonces el resto no las realizo porque se sabe que la operación “&&” va a dar 0. Para “or” es el mismo razonamiento pero si uno da 1 entonces se que el “or” va a dar 1.

9. AND elemento por elemento (&)
10. OR elemento por elemento (|)
11. AND con cortocircuito (&&)
12. OR con cortocircuito (||)

Capítulo 7

Graficación



OCTAVE



7.1 Introducción a la graficación

Hay distintos tipos de graficos. Plot sirve para graficar en 2D. Los pares ordenados se dan como vectores de igual número de componentes. Hay que notar que el primer input son las coordenadas x y el segundo, las coordenadas y. A continuación se presenta un ejemplo con seis pares ordenados.

```
>> v=5:10
```

```
v =
```

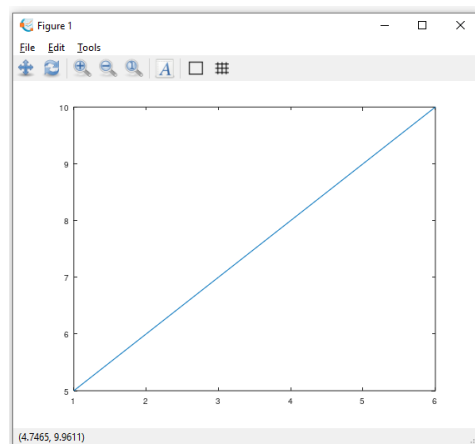
```
5    6    7    8    9   10
```

```
>> x=1:6
```

```
x =
```

```
1    2    3    4    5    6
```

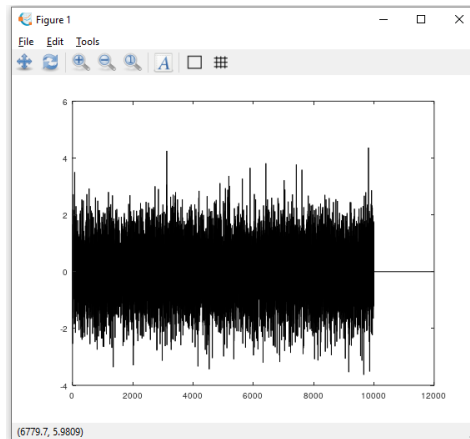
```
>> plot(x,y)
```



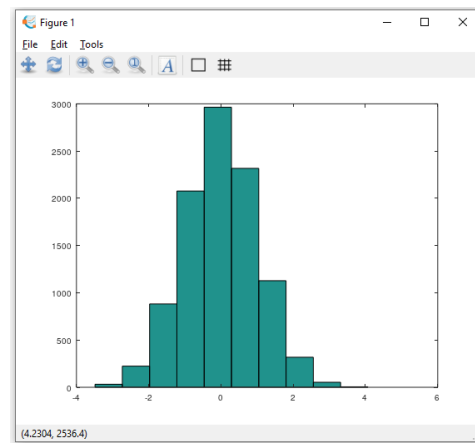
También se puede crear un gráfico de barras o un histograma usando las funciones “bar” y “hist”. Para mostrar un ejemplo en Octave de graficos de estos se utilizará la función “randn”, la cual permite generar muestras de la variable normal estándar (esta función se estudia en el capítulo 9 de estadística). Notese que el histograma se parece al gráfico de una variable normal.

```
>> v=randn(10000,1);
```

```
>> bar(v)
```

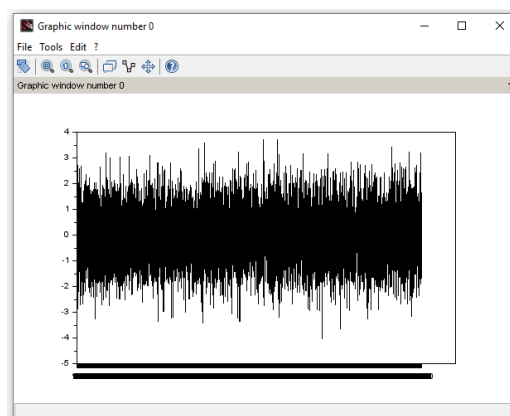



```
>> v=randn(10000,1);
>> hist(v)
```



En Scilab se realiza de otro modo. Primero se arma el vector v y luego se utiliza la función `bar` y la función “`plthist`”. En esta se debe introducir como primer input el número de intervalos que se desea. En este ejemplo es 20.

```
--> v=rand(10000,1,"n");
--> bar(v)
```



```
--> v=rand(10000,1,"n");
--> histplot(20,v)
ans =
```

```

column 1 to 9

0.0002714    0.0027144    0.0059717    0.0143865    0.0342018
0.0787183    0.1294781    0.210368    0.2950581

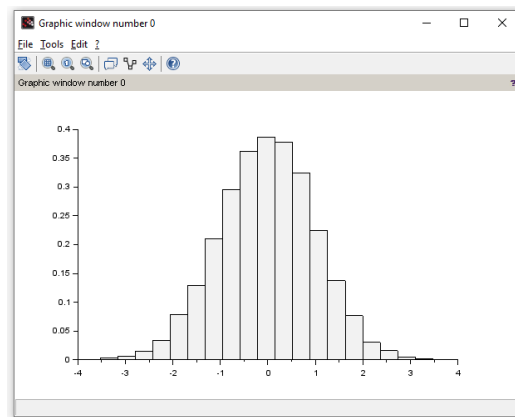
column 10 to 18

0.3615615    0.3865342    0.3778481    0.3241024    0.2247545
0.1373499    0.0768183    0.030673    0.0160151

column 19 to 20

0.0054289    0.0021715

```

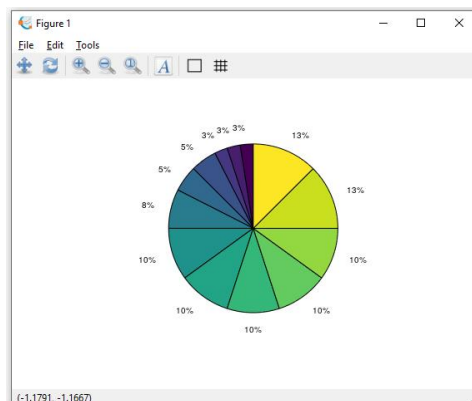


Pie permite realizar un gráfico de torta. Para ellos primer se inventarán unos valores y luego se los graficará usando la función “pie”. Esta función procede a calcular la suma de todos los coeficientes de “x” y después a cada componente del vector la divide por esa suma y el porcentaje restante lo introduce en la torta. Por ejemplo, las dos primeras tajadas son 13%, ya que $5/40=13\%$. Obsérvese que 40 es la suma de los componentes y hay dos cincos en el vector.

```

>> x=[1 1 1 2 2 3 4 4 4 4 4 5 5];
>> pie(x)

```

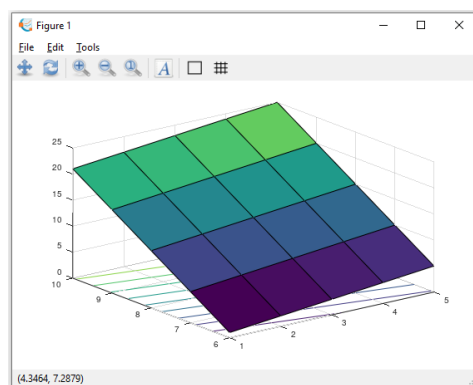


Octave y Scilab también permiten graficar superficies. Para eso se utiliza la función “surf”, la cual tiene que tener como inputs dos vectores (una para x e y) y una matriz A. Si los vectores “x” e “y” tienen dimensiones N y M, respectivamente, la matriz debe ser de N x M. A continuación, se presenta un ejemplo:

```
>> X=[1:5];
>> Y=[6:10];
>> A=[1:5;6:10;11:15;16:20;21:25]
A =
```

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

```
>> surfc(X,Y,A)
```



También se puede hacer lo mismo aplicando la función “surfc”. Esto da el mismo grafico pero agrega las curvas de abajo, que son superficies de nivel. Es decir que moviéndose por ellas se puede mover por donde Z es constante. En Scilab, las superficies de nivel se tienen que agregar aparte. Luego de aplicar el comando “surf” (y sin cerrar el gráfico) se usa el comando “contour”.

```
--> X=[1:5]
X =

    1.    2.    3.    4.    5.

--> Y=[6:10]
Y =

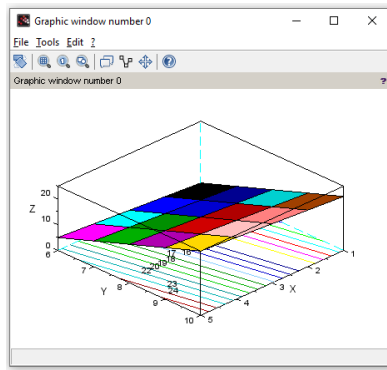
    6.    7.    8.    9.   10.

--> A=[1:5;6:10;11:15;16:20;21:25]
A =

    1.    2.    3.    4.    5.
    6.    7.    8.    9.   10.
   11.   12.   13.   14.   15.
   16.   17.   18.   19.   20.
   21.   22.   23.   24.   25.

--> surf(X,Y,A)

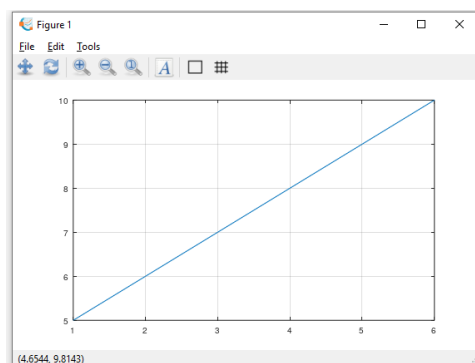
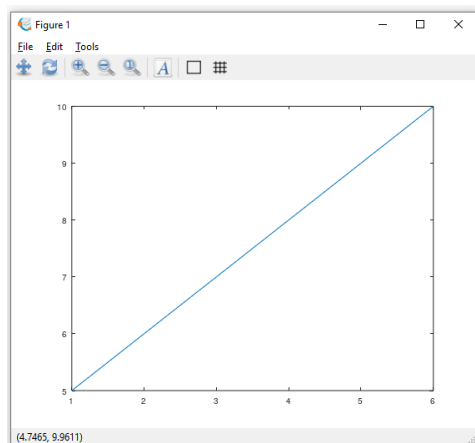
--> contour(X,Y,A,20,flag=[1 2 4])
```



A los gráficos se les puede agregar grilla, ejes, una leyenda, un título y etiquetas a los ejes. Es decir, se los puede personalizar. Esta metodología que se presentará a continuación se aplicará en los gráficos tipo “plot” pero para cualquier otro tipo de gráfico el procedimiento de personalización del gráfico es análogo.

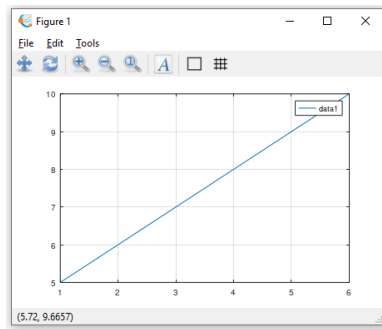
En Octave se puede agregar una grilla a un gráfico introduciendo el comando “grid”. Así, al gráfico siguiente se le agrega una grilla, como se observa a continuación. **En Scilab el comando es “xgrid”.**

```
>> x=1:6;
>> v=5:10;
>> plot(x,v)
>> grid
```



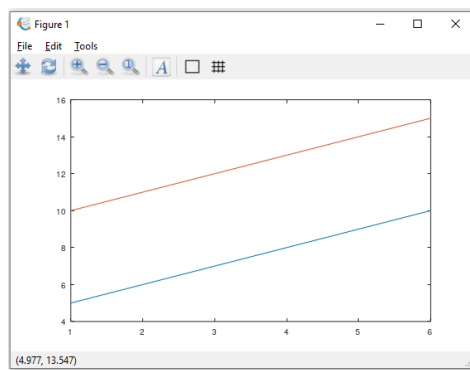
Para agregar una leyenda se utiliza el comando “legend”.

```
>> legend
```



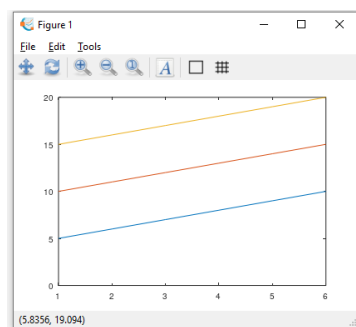
Si se busca agregarle otra serie de puntos al grafico de Octave se debe usar el comando “hold on”, y así se obtienen dos curvas en el mismo grafico. En Scilab el comando es “`mtlb_hold on`”.

```
>> x=1:6;
>> v=5:10;
>> plot(x,v)
>> hold on
>> w=[10:15];
>> plot(x,w)
```



Incluso, se puede seguir agregando.

```
>> z=[15:20];
>> plot(x,z)
```

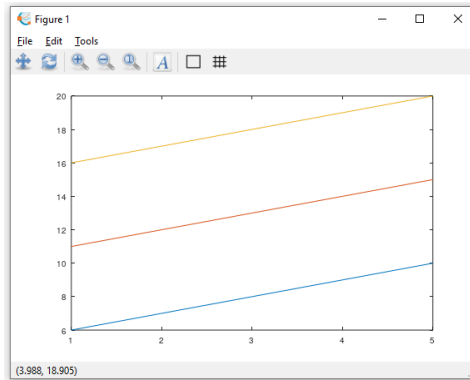


Si ya no hace falta agregar más curvas se utiliza el comando “hold off” de Octave (en Scilab es `mtlb_hold off`).

```
>> hold off
```

Finalmente, se pueden agregar varias curvas usando una sola vez el comando plot.

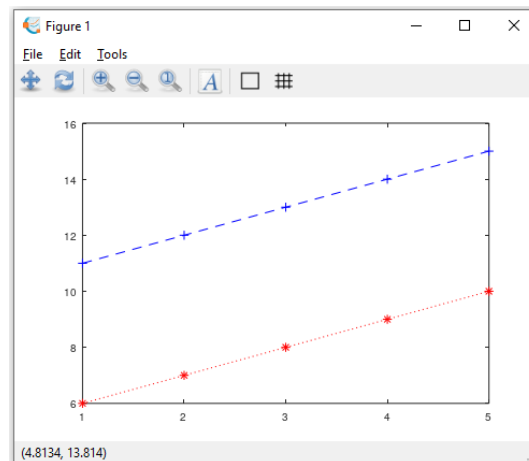
```
>> t=1:5;
>> x=6:10;
>> y=11:15;
>> z=16:20;
>> plot(t,x,t,y,t,z)
```



7.2 Personalizado de curvas

Cada vez que se introduce una nueva curva en un gráfico, Octave y Scilab permiten personalizarla. Por ejemplo, se puede introducir como una línea llena, de color azul y cuyos marcadores sean cuadrado. O, por ejemplo, que la curva sea punteada, de color verde y con marcadores circulares. Esto se logra introduciendo un nuevo input al llamar la función plot. En este nuevo input se introducen símbolos y letras, que tienen codificado el tipo de curva, el color de la misma y como serán los marcadores. A continuación, se presenta un ejemplo y la tabla con el significado de estos símbolos. Obsérvese que en este ejemplo la curva es punteada, roja y sus marcadores son asteriscos, tal cual indica la última tabla.

```
>> t=1:5;
>> x=6:10;
>> y=11:15;
>> plot(t,x,"*r",t,y,"--+b")
```



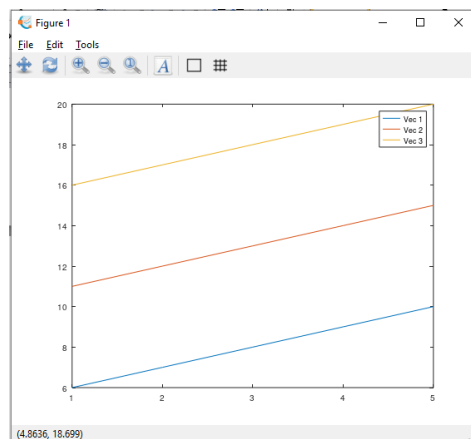
Tipo de línea	Símbolo	Marcador	Símbolo	Color	Símbolo
Continua	-	Más	+	Negro	k
Discontinua	--	Círculo	o	Rojo	r
Punteada	:	Asterisco	*	Verde	g
Discontinua punteada	-.	Punto	.	Azul	b

-	-	Cruz	x	Amarillo	y
-	-	Línea vertical		Magenta	m
-	-	Menos	-	Celeste	c
-	-	Cuadrado	s	Blanco	w
-	-	Diamante	d	-	-
-	-	Triángulos	^	-	-
			<		
			>		
-	-	Pentágono	p	-	-
-	-	Hexágono	h	-	-

7.3 Personalizado de la leyenda, el título y las etiquetas de los ejes

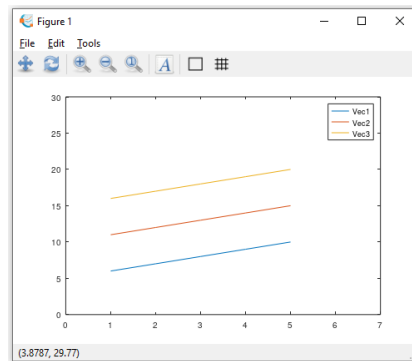
Octave y Scilab permiten personalizar la leyenda, el título y las etiquetas de los ejes. En la leyenda se pueden poner los nombres deseados.

```
>> t=1:5;
>> x=6:10;
>> y=11:15;
>> z=16:20;
>> plot(t,x,t,y,t,z)
>> legend("Vec 1", "Vec 2", "Vec 3")
```



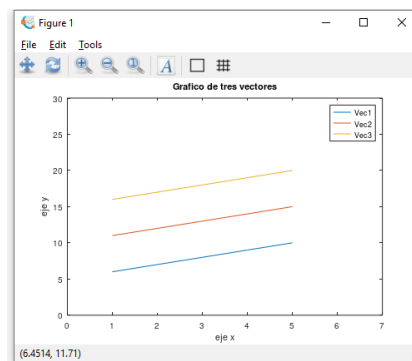
Por otro lado es posible definir los límites de los ejes. En el ejemplo siguiente, se usa la función “axis” para establecer los límites de los ejes. El eje “x” va de 0 a 7 y el “y” de 0 a 30. En Scilab se usa el comando `mtlb_axis`. El input se introduce del mismo modo, como un vector de cuatro componentes.

```
>> axis([0 7 0 30])
```



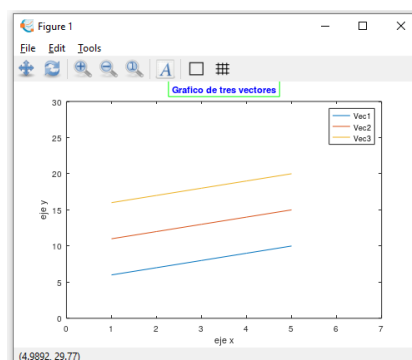
También se puede introducir un título y etiquetas a los eje.

```
>> title("Grafico de tres vectores")
>> xlabel("eje x")
>> ylabel("eje y")
```



Además de esto, la leyenda, el título y las etiquetas se las puede personalizar en cuanto a color, tipo de letra, etc. La personalización de cada uno de ellos se realiza introduciendo inputs adicionales, como con la función “plot”. Los inputs se presentan como en la sección anterior, por medio de la tabla que viene a continuación. Para ayudar a entender la operatoria se presenta el siguiente ejemplo. El comando “title('gráfico de prueba', 'color', 'blue', 'edgecolor', 'green')” nos permite introducir un título que es “gráfico de prueba”, escrito en color azul y en una caja verde. El resultado se encuentra a continuación:

```
>> title("Grafico de tres vectores","color","blue","edgecolor","green")
```



Vale la pena aclarar que en las columnas “input” se presenta uno de todos los inputs que pueden dársele a cada propiedad. Por ejemplo, en este caso se le dio el input “blue” a la propiedad “color” del título (title). Sin embargo, se podría haber elegido cualquier otro color, como “red” (rojo) o “yellow” (amarillo). Por ejemplo, introduciendo “title(' Grafico de tres vectores', 'color', 'magenta', 'edgecolor', 'cyan')”. Se puede obtener más información sobre las distintas opciones de customización escribiendo “help title”, “help xlabel”, “help ylabel” y “help legend”.

title		xlabel / ylabel		legend	
Propiedad	input	Propiedad	input	Propiedad	input
'color'	'blue'	'fontsize'	3	'color'	'blue'
'edgecolor'	'red'	'fontname'	'comic sans'	'edgecolor'	'red'
'rotation'	0	'color'	'blue'	'box' / -	'on' / %f
'position'	[6 25]	'position'	[1 1]	'location' / -	'south' / [5 20]
'fontsize'	10	'rotation'	90	'numcolumns'	2
'backgroundcolor' / 'background'	'green'	- / 'box'	'on'	'orientation'	'horizontal'
-	-	'edgecolor'	'red'	'textcolor'	'green'
-	-	'backgroundcolor' / 'background'	'green'	'fontsize'	3
-	-	-	-	'fontname'	'helvetica'



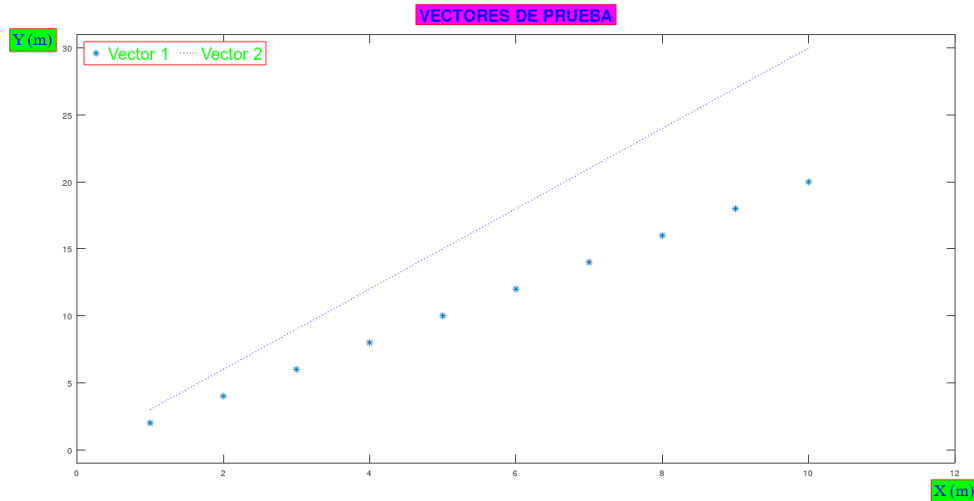
En las propiedades “box” y “location”, a Octave le corresponden los inputs que se encuentran a la izquierda de la barra (/), como “on” o “south” mientras que “%f” y “[5,20]” corresponde a Scilab.

Todas estas propiedades de la tabla están disponibles en Octave y Scilab para el título y las etiquetas. **Por otro lado, Scilab solo dispone para la leyenda las dos propiedades resaltadas en rojo.** En cambio, Octave permite utilizar todas las de la columna legend.

A continuación se presentan dos scripts en los que se realiza la personalización de un gráfico para Octave y Scilab. Ayudándose con las dos tablas anteriores y leyendo los scripts el lector podrá constatar que los gráficos finales responden a las propiedades de las tablas.

También es posible copiar y pegar este texto en un script para cambiar las propiedades y probar distintos inputs, analizando los cambios que se producen en el gráfico.

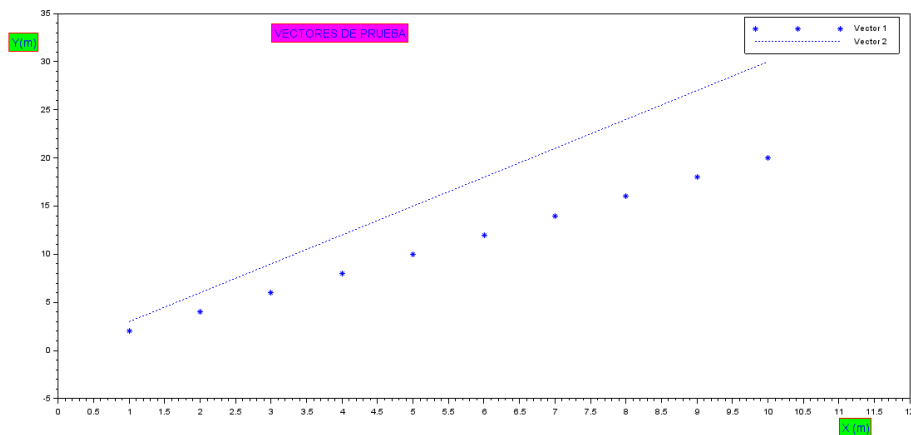
```
plot(1:10,2:2:20,'*');
hold on
plot(1:10,3:3:30,':b');
title('VECTORES DE PRUEBA', 'color','blue','edgecolor','red',...
'rotation',0,'position',[6 32],'fontsize',20,...
'backgroundcolor','magenta')
axis([0,12,-1,31])
legend('Vector 1', 'Vector 2','color','white','edgecolor','red',
'fontsize',20,'location','northwest','numcolumns',2,'box','on',...
'fontname','helvetica','textcolor','green','box','on')
xlabel('X (m)', 'color','blue','edgecolor','red',...
'rotation',0,'position',[12 -2.5],'fontsize',20,...
'fontname','times new roman','backgroundcolor','green');
ylabel('Y (m)','color','blue','edgecolor','red','rotation',0,...
'position',[-0.6 30],'fontsize',20,'fontname','times new roman',...
'backgroundcolor','green');
hold off
```



```

plot(1:10,2:2:20,'*');
mtlb_hold on
plot(1:10,3:3:30,':b');
title('VECTORES DE PRUEBA','color','blue','edgecolor','red',...
'rotation',0,'position',[3 32],'fontsize',3,...
'backgroundcolor','magenta')
mtlb_axis([0,12,-1,31])
legend('Vector 1', 'Vector 2', 'color','white','edgecolor',...
'red','fontsize',3,'location','northwest','numcolumns',2,...
'box','on','fontname','helvetica','textcolor','green','box','on')
xlabel('X (m)', 'color','blue','edgecolor','red','rotation',0,
'position',[11 -9],'fontsize',3,'fontname','times',...
'backgroundcolor','green');
ylabel('Y(m)', 'color','blue','edgecolor','red','rotation',0,...
'position',[-0.7 31],'fontsize',3,'fontname','times',...
'backgroundcolor','green');
mtlb_hold off

```



7.4 Creación de varios gráficos

Se pueden crear varios gráficos (o figuras) uno atrás de otro. Esto se explicará por medio del siguiente ejemplo. El mismo primero se desarrolla como si el usuario usase Octave y luego se presentará para Scilab.

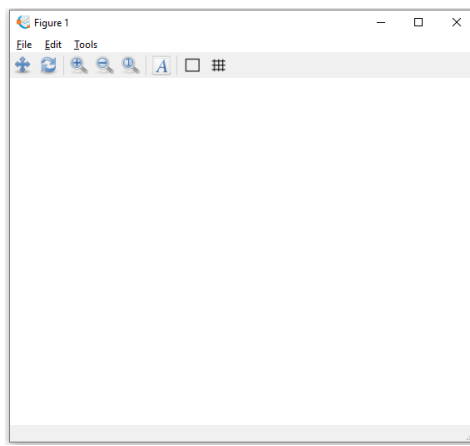
El usuario necesita crear dos gráficos. En el primero se debe graficar la posición de una partícula en función del tiempo y en el otro se debe graficar la temperatura de la misma respecto al tiempo. A continuación se definen los tres vectores.

```
>> t=[0:10];  
>> posicion=[0 11.5 20.1 24.7 32.4 38.1 49.7 58.3 64.0 71.5 80.1];  
>> temperatura=[273.15 280.4 284.14 291.4 296.7 300.7 310.3 318.9  
325.1 330.1 337.9];
```

Para crear los dos graficos el usuario realiza lo siguiente:

```
>> Figure 1
```

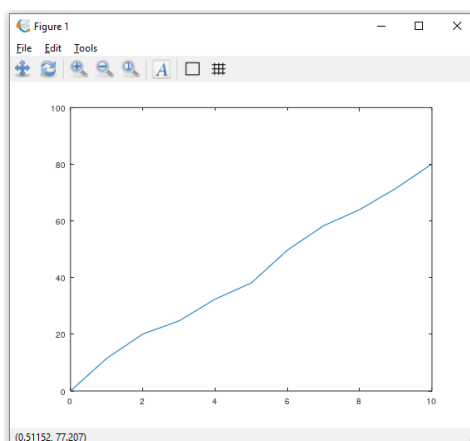
De este modo se genera la figura 1, la cual se presenta a continuación. Como puede verse, está vacía.



Luego de esto, ya se puede empezar a volcar información en dicho gráfico. El usuario introduce lo siguiente para tener la posición respecto al tiempo.

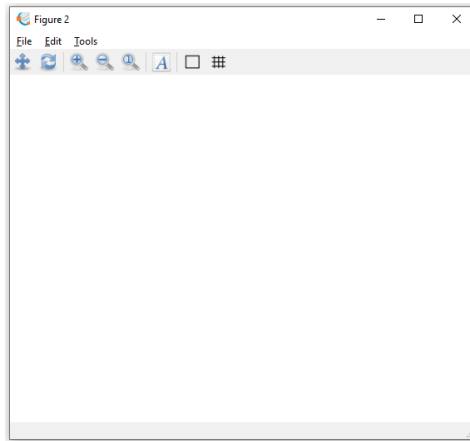
```
>> plot(t,posicion)
```

La figura 1 queda como sigue:



Ahora solo queda graficar la figura 2. Por eso, se utiliza el siguiente comando, el cual creará una figura 2 vacía.

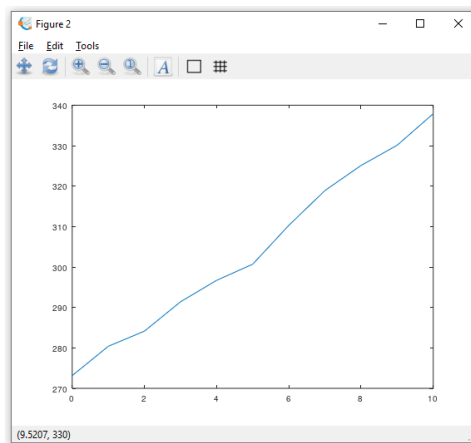
```
>> Figure 2
```



En este punto el usuario ya tiene creada dos figuras, pero la que el programa considera como seleccionada es la figura 2 (porque es la última creada). Esto significa que si se vuelve la función “plot” o alguna otra como “title” o “legend”, todo esto se vuelca en la figura 2. El usuario sigue trabajando con dicha figura:

```
>> plot(t,temperatura)
```

Con los dos últimos comandos, la figura 2 quedó así:

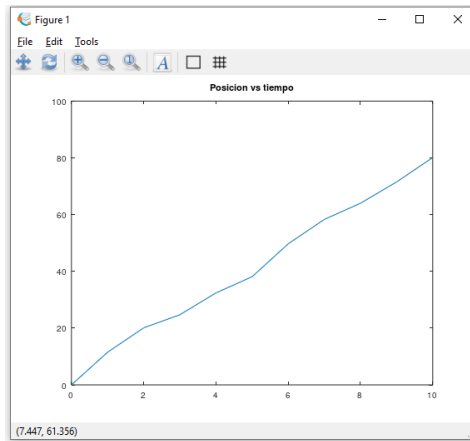


Finalmente, se puede suponer que el usuario se dio cuenta que tenia que agregarle un título a la figura 1, pero se encuentra con el problema de que la que se encuentra seleccionada actualmente es la otra, por lo que debe corregir eso. Por eso, procede del siguiente modo:

```
>> Figure 1
```

Así, el usuario seleccionó la figura 1. Finalmente, solo le queda agregar el título con la función “title”. La figura 1 queda del siguiente modo:

```
>> title("Posicion vs tiempo")
```



De un modo similar se opera en Scilab. La única diferencia importante es que en Octave el comando “figure” permite crear una figura y, si esta ya existe, seleccionarla para seguir introduciendo información o personalizarla. En cambio, en Scilab, el comando “figure” solo sirve para crear una figura, pero si la misma ya existe (como pasa con la figura 1 después de crear la figura 2), el comando no tiene efecto. Si el usuario tiene más de dos figuras creadas y desea seleccionar una de ellas (como cuando se le desea agregar el título a la figura 1 pero la que se encuentra seleccionada es la figura 2) solo se debe utilizar el comando “scf”. A continuación se presenta como debe proceder el usuario en scilab para obtener lo mismo que en Octave.

```
--> t=[0:10];

--> posicion=[0 11.5 20.1 24.7 32.4 38.1 49.7 58.3 64.0 71.5 80.1];

--> temperatura=[273.15 280.4 284.14 291.4 296.7 300.7 310.3 318.9
325.1 330.1 337.9];

--> figure(1)

--> plot(t,posicion)

--> figure(2)

--> plot(t,temperatura)

--> scf(1) //selecciona la figura cuyo número tiene como input, la 1

--> title("Posicion vs tiempo")
```



En cualquiera de los tres softwares, al crearse una figura, por default la misma queda como la que se encuentra seleccionada. Si se usa la función “plot”, por ejemplo, la nueva información se agrega a la nueva figura.

Capítulo 8

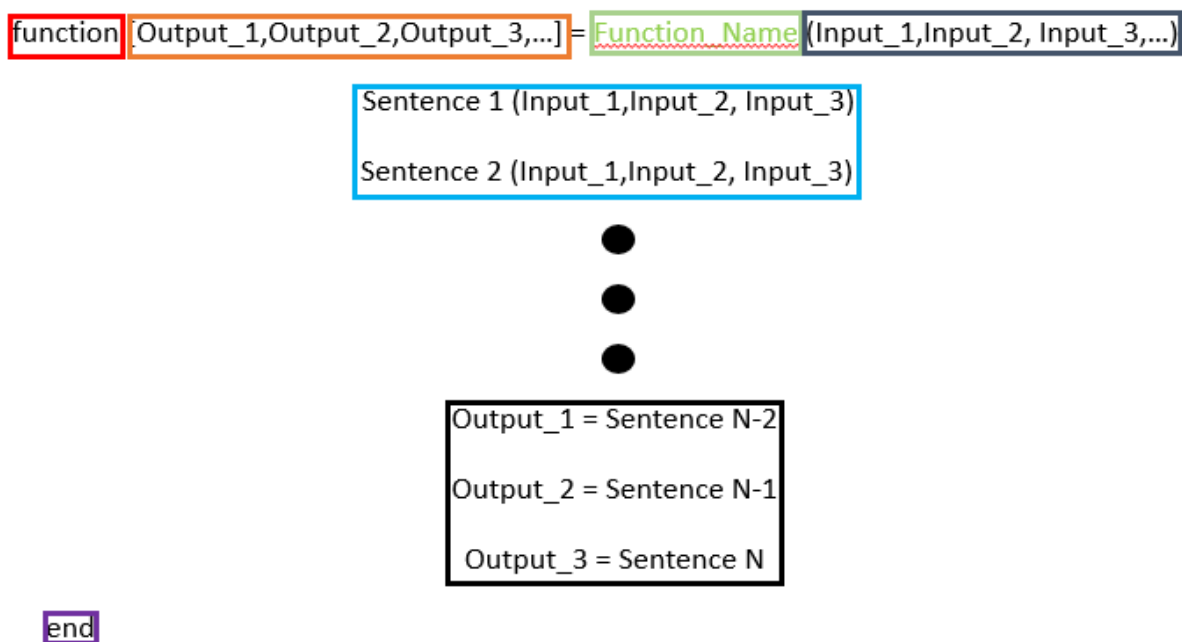
Funciones creadas por el usuario



8.1 Funciones creadas por el usuario

Antes de empezar esta sección es preferible pensar qué es una función de Octave o Scilab. Una función es un comando que cuando se lo llama, el usuario le da los inputs que necesita, se ejecutan una serie de sentencias que la función tiene cargada y finalmente da unos outputs. Por ejemplo, rem tiene dos entradas (el divisor y dividendo) y de él se obtiene una salida, el resto.

Octave y Scilab tienen funciones creadas por los desarrolladores de cada software, pero también los usuarios pueden crear las suyas. Esto es útil si el usuario sabe que va a hacer un determinado cálculo muchas veces en un mismo script, lo que se puede hacer es definir una función que realice este cálculo y llamarla cada vez que se necesite. Así se ahorran varias líneas de código. Es decir, se puede crear una función que tome determinadas entradas, aplique determinadas sentencias y luego presente las salidas. Las funciones creadas por el usuario se deben guardar en un script. Para crear una se debe iniciar un script en blanco e introducir la función nueva que se desea crear. Toda función tiene la estructura que se presenta a continuación. En este punto se presentará el significado de cada componente de la estructura.



Rojo: Function quiere decir se está creando una función propia.

Naranja: Este es el vector de outputs (salidas). Indica a la función cuantos outputs tiene. En el dictado de sentencias se debe incluir el mismo número de outputs que en este vector y con los mismos nombres.

Verde: Cada función tiene que tener un nombre. Es recomendable guardar este script con el nombre de la función.

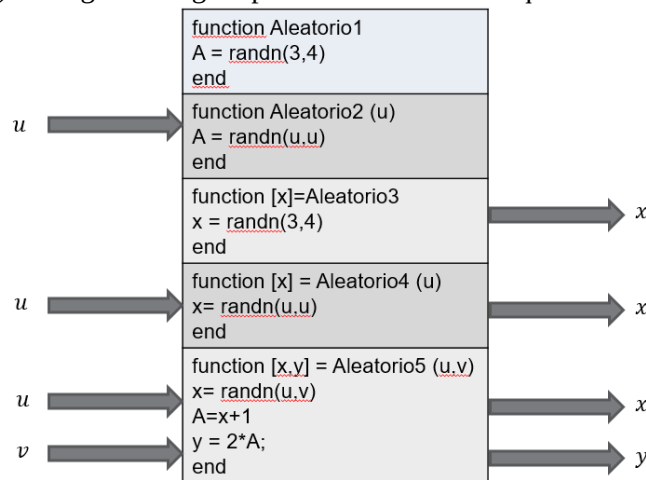
Azul: Los inputs son para que la función sepa cuantas variables de entrada tienen que recibir cuando se realiza el llamado.

Celeste: Aplicación de las sentencias, usando los inputs o usando variables locales (creadas dentro de la función)

Negro: Carga de los resultados en los outputs.

Violeta: Toda función creada por el usuario termina con un “end”.

Las funciones tienen inputs y outputs, y variables locales. A continuación se verán algunas funciones y se identificarán los inputs y outputs y las variables locales y globales (las cuales son las que se usan en la ventana de comandos). La siguiente figura presenta las funciones que se van a usar como ejemplo.



La función “Aleatorio1” cuando es llamada crea una variable A que tiene doce muestras de la variable normal en una matriz de 3x4. Notese que no tiene ni inputs ni outputs ya que en el primer renglón faltan el vector de outputs y el paréntesis de inputs. La variable “A”, al no ser introducida en ningún output, queda “atrapada” dentro de la función y su contenido no puede ser usado. Algo similar ocurre cuando se llama la función “rem”, en la cual se crean variables intermedias pero el usuario no puede verlas.

La función Aleatorio2 no tiene outputs pero tiene un input, que es “u”, ya que tiene el paréntesis de las entradas. Notar que en el primer renglón aparece a la derecha (en el lugar de los inputs) una “u” entre paréntesis y se usa en el segundo renglón para crear la matriz A. Por eso en la función Aleatorio2 hay una flecha llamada u que ingresa en la función, simulando la entrada de “u”.

La función Aleatorio3 no tiene inputs pero tiene un output, que es “x”. Notar que aparece esta variable en el vector de salidas, después de la palabra “function”. A la variable “x” se le carga los valores de muestra de la variable normal de 3x4. Por eso esta función tiene una flecha de salida que se llama “x”. En la función Aleatorio4 hay un input y un output. Se usa “u” para crear la matriz de muestras normales y se usa “x” para cargar está en la salida de la función. Por eso, esta función tiene una flecha de entrada que se llama “u” y una flecha de salida que se llama “x”.

Finalmente está la función Aleatorio5. Aquí se tienen dos entradas (“u” y “v”) y se usan para crear tres variables. Una es “x”, que es una variable de salida, otra es “A”, que es una variable intermedia (o local), y finalmente se usa “A” para crear “y”. Por eso se tienen dos flechas de entrada “u” e “v” y dos de salida “x” e “y”. Se ve, además, que el vector de outputs tiene a “x” e “y” y entre paréntesis a “u” y “v”.

Las funciones creadas por el usuario se crean como cualquier otra, pero tiene que tenerse en cuenta la cantidad de entradas y salidas que se pida. En el siguiente ejemplo se llama a las cinco funciones. Se puede ver como son llamadas. Además, notar que la cantidad de inputs y outputs en cada uno de los cinco llamados coinciden con la cantidad de inputs y outputs de la figura anterior. Para este ejemplo se tomó u=1 y v=2.

8.2 Variables globales y locales

Como ya fue explicado anteriormente, las variables de las funciones pueden clasificarse en globales y locales. Las variables globales están definidas en todos lados y están disponibles para ser usadas como entradas y salidas de cualquier función. En particular, están definidas dentro de la ventana de comandos y pueden ser llamadas en ella. Además, se pueden encontrar dentro del workspace (que es donde se

encuentran todas las variables definidas como globales, ver capítulo 0). En cambio, las locales son las que se encuentran definidas dentro de una función y no pueden salir de ellas. Noten que una variable global está disponible dentro de una función si es usada como entrada o salida. En este punto se usarán las cinco funciones definidas anteriormente para explicar detalladamente la diferencia entre variables globales y locales.

En Aleatorio1 hay una función que no tiene inputs y outputs, así que no involucra variables globales. En cambio, se crea la variable “A”. Al ser creada dentro de la función y no ser input o output, entonces es variable local. Si se la llama desde la ventana de comandos se obtendrá un mensaje que informa que la misma no está definida. Lo mismo ocurre si la llama alguna otra función. Hay que tener en cuenta, además, que una vez finalizada la ejecución de la función “Aleatorio1” (cuando ya se ejecutaron todas las sentencias de esta función) se descarta esta variable. Es por todo lo descrito que, como se observa en la siguiente imagen, “A” no se encuentra definida en la ventana de comandos ni antes ni después de ejecutar “Aleatorio1”.

```
>> A %Llamado A, variable interna de Aleatorio1. No existe en
ventana de comandos.
```

```
error: 'A' undefined near line 1, column 1
```

```
>> Aleatorio1
```

```
A = 1.0202
```

```
>> A %Llamado A, variable interna de Aleatorio1. No existe en
ventana de comandos.
```

```
error: 'A' undefined near line 1, column 1
```

Aleatorio2 requiere un input. Este es global porque primero hay que definirlo en la ventana de comandos y luego llamar la función, introduciendo esta como input. Pero en Aleatorio2 también se crea la variable “A”, que no es output, por lo que sigue siendo local. Es por eso que, al igual que el caso anterior, “A” no está disponible en la ventana de comandos ni antes ni después de correr la función “Aleatorio2”.

```
>> x=1 %Definición de x, que será input de Aleatorio2
```

```
x = 1
```

```
>> A %Se llama A, que es una variable local de Aleatorio2
```

```
%Al ser local, no existe en la ventana de comandos
```

```
error: 'A' undefined near line 1, column 1
```

```
>> Aleatorio2(x) %Llamado de Aleatorio2
```

```
A = 3.1101
```

```
>> A %Se llama A, que es una variable local de Aleatorio2
```

```
%Al ser local, no existe en la ventana de comandos
```

```
error: 'A' undefined near line 1, column 1
```

Aleatorio 3 tiene un output. Si se la llama, ese output pasa a ser variable global. Como se observa en la siguiente figura, “x” se encuentra definida dentro de la ventana de comandos luego de llamar la función, pero no antes.

```
>> x %Llamado de x, output de Aleatorio3. Todavía no se  
corrió, no existe
```

```
error: 'x' undefined near line 1, column 1
```

```
>> [x]=Aleatorio3
```

```
x =
```

```
7.8756    6.2022    9.4647    6.2288  
4.6521    2.6414    3.9040    7.4241  
3.0846    3.9248    3.4875    3.5855
```

```
%Resultado de x
```

```
>> x %Llamado de x
```

```
x =
```

```
7.8756    6.2022    9.4647    6.2288  
4.6521    2.6414    3.9040    7.4241  
3.0846    3.9248    3.4875    3.5855
```



La razón por la que “x” aparece dos veces en la ventana de comandos al ejecutar “Aleatorio3” es porque ni en la primera sentencia de esta función ni en su ejecución desde la ventana de comandos se incluyó el símbolo “;”. Esto mismo se repite en “Aleatorio4” y “Aleatorio5”. Esto mismo resulta confuso por lo que se recomienda al usuario utilizar “;” siempre que sea posible.

Aleatorio4 tiene un input llamado “u”. El usuario la define en la ventana de comandos y esto la hace una variable global. Por otro lado, al llamar la función se crea una variable global “x”. Por eso, esta variable no existe antes de llamar la función pero sí después de hacerlo, como se ve en el siguiente ejemplo.

```
>> u=3 %Definición de u, que será input de Aleatorio4
```

```
u = 3
```

```
>> x %Llamado de x, output de Aleatorio4. Todavía no se  
corrió, no existe
```

```
error: 'x' undefined near line 1, column 1
```

```
>> [x]=Aleatorio4(u) %Llamado de Aleatorio4
```

```
x =
```

```
7.2804    6.9937    9.3527
2.2712    1.6866    5.0091
8.5646    3.2275    3.8347
```

```
%Resultado de x
```

```
x =
```

```
7.2804    6.9937    9.3527
2.2712    1.6866    5.0091
8.5646    3.2275    3.8347
```

```
%Resultado de x
```

```
>> x %Llamado de x. Ahora existe en la ventana de comandos porque se corrió Aleatorio4
```

```
x =
```

```
7.2804    6.9937    9.3527
2.2712    1.6866    5.0091
8.5646    3.2275    3.8347
```

Finalmente, Solo queda analizar Aleatorio5. Se observa que “u” y “v” son variables globales definidas dentro de la ventana de comandos. Por otro lado, “x” e “y” son creadas, como en el caso anterior, en el llamado de Aleatorio5. Al estar disponibles en la ventana de comandos queda claro que son variables globales. Por otro lado, si analizamos la estructura de Aleatorio5, se observará una variable “A”. Esta es una variable local, ya que es creada dentro de la función, pero no está definida como salida. Por eso, al llamarla desde la ventana de comandos, no está ni antes ni después de llamar a Aleatorio5.

```
>> u=3 %Se define la variable u, que será un input de Aleatorio5
```

```
u = 3
```

```
>> v=4 %Se define la variable v, que será un input de Aleatorio5
```

```
v = 4
```

```
>> A %Se llama A, que es una variable local de Aleatorio5
```

```
%Al ser local, no existe en la ventana de comandos
```

```
error: 'A' undefined near line 1, column 1
```

```
>> x %Se llama x, que será un output de Aleatorio5
```

```
%da error porque no existe, al no correrse todavia
Aleatorio5
```

```
error: 'x' undefined near line 1, column 1
```

```
>> y %Se llama y, que será un output de Aleatorio5
```

```
%da error porque no existe, al no correrse todavia
Aleatorio5
```

```
error: 'y' undefined near line 1, column 1
```

```
>> [x,y]=Aleatorio5(u,v) %Llamado de Aleatorio5
```

```
x =
```

```
8.6096    9.0113    8.3606    2.7427
4.9389    4.0148    1.5193    8.8395
2.0465    2.9739    5.5564    5.6674
```

```
% Resultado de x
```

```
y =
```

```
9.6096   10.0113    9.3606    3.7427
5.9389    5.0148    2.5193    9.8395
3.0465    3.9739    6.5564    6.6674
```

```
% Resultado de y
```

```
>> x % Llamado de x, como es output está en la ventana de
comandos
```

```
x =
```

```
8.6096    9.0113    8.3606    2.7427
4.9389    4.0148    1.5193    8.8395
2.0465    2.9739    5.5564    5.6674
```

```
>> y % Llamado de y, como es output está en la ventana de
comandos
```

```
y =
```

```
9.6096   10.0113    9.3606    3.7427
5.9389    5.0148    2.5193    9.8395
```

3.0465 3.9739 6.5564 6.6674

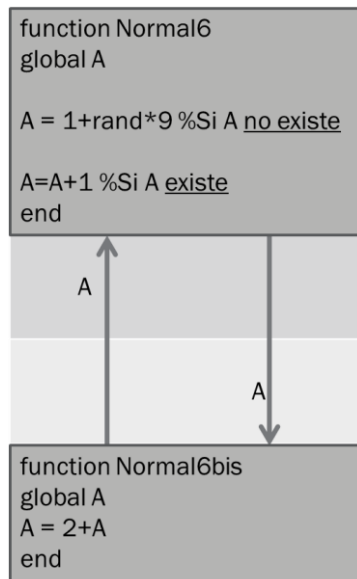
```
>> A      %A es una variable local de Aleatorio5
        %por lo que sigue sin estar en ventana de comandos
error: 'A' undefined near line 1, column 1
```

La siguiente tabla resume lo descripto en esta sección.

Variables globales	Variables locales	Definición de la función	Llamado
-	A	<pre>function Aleatorio1 A = 1+rand(3,4)*9 end</pre>	Aleatorio1
u	A	<pre>function Aleatorio2 (u) A = 1+rand(u,u)*9 end</pre>	Aleatorio2(u)
x	-	<pre>function [x]=Aleatorio3 x = 1+rand(3,4)*9 end</pre>	[x]=Aleatorio3
u x	-	<pre>function [x] = Aleatorio4 (u) x= 1+rand(u,u)*9 end</pre>	[x]=Aleatorio4(u)
x y u v	A	<pre>function [x,v] = Aleatorio5 (u,v) x= 1+rand(u,v)*9 A=x+1 y = 2*A; end</pre>	[x,v]=Aleatorio5(u,v)

8.3 Definir una variable como global dentro de una función

Dentro de una función se puede definir una variable local y que esta pueda ser también usada en otras funciones. Por ejemplo, se puede llamar una función Normal6 que cree una variable local “A” y luego otra función Normal6bis la puede levantar y usar. Así, la variable “A” está disponible tanto en una función como en la otra. En el siguiente esquema se tiene las dos funciones (Normal6 y Normal6bis).



Normal6 crea una variable “A” entre 1 y 9. Después, si se la vuelve a llamar, toma a “A” y le suma 1. Por otro lado, la función Normal6bis busca la variable “A” y le suma 2. Para poder hacer este intercambio entre Normal6 y Normal 6bis se debe escribir “global A” al principio de la definición de cada función. Notar que “A” es variable local de ambas funciones ya que no es input ni output y se crea dentro de la función Normal6 (Normal6bis solo tiene la orden de levantarla, no de crearla).

Si el usuario crease estas dos funciones en su computadora (al final de esta sección se encuentran las dos definiciones de las funciones) notaría que no puede llamar a “A” desde la ventana de comandos. A continuación, se presentan varios llamados de las funciones Normal6 y Normal6bis y se observa cómo cambia “A” después de cada llamado. Nótese que, además, al llamar a “A” desde la ventana de comandos, Octave y Scilab informan un error.

```

>> Normal6
A = 8.4678
>> Normal6
A = 9.4678
>> Normal6
A = 10.468
>> Normal6bis
A = 12.468
>> Normal6bis
A = 14.468
>> Normal6bis
A = 16.468
>> Normal6
A = 17.468
>> Normal6
A = 18.468
>> Normal6bis
A = 20.468
>> Normal6bis
A = 22.468
>> A
error: 'A' undefined near line 1, column 1

```

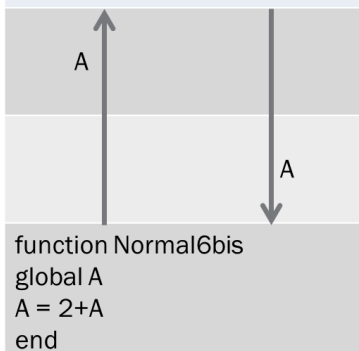
Finalmente, si el usuario deseara poder llamar a “A” desde la ventana de comandos solo tendría que escribir el comando “global A” en ella, tal como se observa abajo. Esto aplica a cualquier otra función donde quiera usarse a “A”.

```
>> global A
>> A
A = 22.468
>> A=A+1 %Este es un llamado desde la ventana de comandos
A = 23.468
>> Normal6
A = 24.468
>> Normal6bis
A = 26.468
>> Normal6bis
A = 28.468
>> A=A+3 %Este es un llamado desde la ventana de comandos
A = 31.468
>> Normal6
A = 32.468
```

A continuación se presenta las definiciones de Normal6 y Normal6bis. Se observa que en ellas aparecen sentencias que no estaban en el esquema de dichas funciones. Estas líneas no se muestran en el esquema para simplificar la comprensión del tema, pero si el lector quisiese implementar estas dos funciones en su computadora debería incluir estas líneas. Sin embargo, cabe remarcar que no hacen al concepto del comando “global”.

```
function Normal6
global A
if exist('A')==0 || isempty(A)
A = 1+rand*9 %Si A no existe
else
A=A+1 %Si A existe
end
```

```
function Normal6bis
global A
A = 2+A
end
```



8.4 Variables persistentes

Como fue estudiado en la sección anterior, en Octave y Scilab las variables pueden ser locales o globales. Las primeras son variables intermedias de la función y, al completarse el número de sentencias, se desechan. Esto quiere decir que se pierde el valor que tenían y ya no podrá recuperarse. Sin embargo, a dichas variables locales se las puede definir como persistentes. Estas tienen la particularidad de que se definen dentro de una función y que después de cada llamado retienen el valor que tenían.

Para crear una variable persistente en Octave se debe introducir el comando “persistent” y el nombre de la variable en la definición de la función. A continuación se presentan dos funciones casi idénticas, Normal7Per y Normal7NoPer, cuya única diferencia es que en la primera la variable “a” se encuentra definida como persistente y en la segunda no. Estas dos funciones crean una variable a=0 y le suman 1, siempre que exista y no sea una variable vacía.

function Normal7Per	function Normal7NoPer
<code>persistent A</code>	
<code>if exist('a')==0 %Crea a si no existe</code>	<code>if exist('a')==0 %Crea a si no existe</code>
<code> a=0;</code>	<code> a=0;</code>
<code>end</code>	<code>end</code>
<code>if isempty(a)==1</code>	<code>if isempty(a)==1</code>
<code> a=0;</code>	<code> a=0;</code>
<code>else</code>	<code>else</code>
<code> a=a+1</code>	<code> a=a+1</code>
<code>end</code>	<code>end</code>
<code>end</code>	<code>end</code>

En Normal7Per, la primera vez que se llama a la variable se obtiene que es igual a cero, como se observa en el ejemplo siguiente. En el segundo llamado de Normal7Per se le suma 1 a “a”. Y así sucesivamente.

```
>> Normal7Per
>> Normal7Per
a = 1
>> Normal7Per
a = 2
>> Normal7Per
a = 3
>> Normal7Per
a = 4
>> Normal7Per
a = 5
>> a %El llamado de a desde la ventana de comandos da error
      %Esto es porque a es persistente, no global
error: 'a' undefined near line 1, column 1
```



Observar que “a”, a pesar de ser persistente, no es global.

En cambio, al llamar a la función Normal7NoPer, donde “a” no es persistente se ve que se crea a “a” como 0 y luego se le suma 1. Al finalizar el llamado, “a” deja de existir, por lo que en el próximo llamado tendrá que volver a ser creada como “a=0” y luego se le sumará 1. Esto ocurre porque “a” no es persistente.

```
>> Normal7NoPer
a = 1
>> Normal7NoPer
a = 1
>> Normal7NoPer
a = 1
>> Normal7NoPer
a = 1
```



```
>> Normal7NoPer
a = 1
>> Normal7NoPer
a = 1
>> Normal7NoPer
a = 1
>> a %El llamado de a desde la ventana de comandos da error
      %Esto es porque a no es global (ni persistente)
error: 'a' undefined near line 1, column 1
```

Cabe aclarar que las variables persistentes son variables locales y no están disponibles fuera de la función. Por eso, no están disponibles en la ventana de comandos, como se observa a continuación. Lamentablemente, en Scilab no pude encontrar una función que defina variables persistentes. Estas no parecen estar disponibles.

Capítulo 9

Estadística y números complejos



OCTAVE



9.1. Generación de números aleatorios

Octave y Scilab ofrecen distintas funciones para generar números aleatorios. En Octave, las principales funciones son: “rand” (números aleatorios entre 0 y 1), “randi” (genera números enteros) y “randn” (números obtenidos de la distribución normal). Hay que tener en cuenta que en Scilab la función análoga a “randi” es “grand” y que en lugar de “randn” se usa “rand” con el argumento “normal”. A continuación se estudiará cada una de ellas.

“rand” permite crear números aleatorios entre 0 y 1. Como Octave y Scilab generan una matriz con números aleatorios en cada componente, el usuario debe introducir como inputs las dimensiones de dicha matriz.

```
>> rand(3,4)
ans =

    0.3245    0.7155    0.7520    0.6904
    0.9351    0.7731    0.1136    0.9445
    0.5588    0.7851    0.7086    0.7709
```

Se debe notar que si en Octave se introduce “rand(3)”, por ejemplo, se obtiene una matriz de 3x3, mientras que Scilab, al recibir solo una de las dos dimensiones, solo presenta un solo número.

En cambio, “randi” permite generar número enteros aleatorios entre 1 y N, donde N es especificado por el usuario. Esta función, además, permite obtener varios números con un solo llamado, usando una matriz. Por ejemplo, se pueden pedir números aleatorios enteros entre 1 y 30 en una matriz de 5x4 escribiendo lo siguiente. Debe notarse que esta función toma siempre el 1 como el número aleatorio más bajo.

```
>> randi(30,5,4)
ans =

     3    13    21    15
    26     6    11    18
     2    16     4    14
    11     8    17     9
    23    16     2     4
```

Por su parte, en Scilab la función para esto es “grand”. Esta es una función que puede usarse para generar números aleatorios de muchas más distribuciones (para más información escribir “help grand”) por lo que requiere que el usuario especifique que distribución hay que usar. Por ejemplo, para generar una matriz de 5x4 de números aleatorios entre 1 y 30 se escribir lo siguiente. Notar que en este caso 1 no tiene porqué ser el número más bajo, ya que se usa como entrada.

```
--> grand(5,4,"uin",1,30)
ans =

    3.     2.    26.     1.
   13.    30.     6.    10.
   15.     6.     1.     5.
     6.    19.     7.     8.
     5.    14.    30.    17.
```

“randn” genera números aleatorios de la variable normal estándar. Permite generarlos dentro de una matriz. Por ejemplo,

```
>> randn(3,4)
ans =

    1.6637   -1.2765    2.4539    0.4976
   -0.2676    1.5471    1.2469   -2.1521
   -0.4094   -0.2295    1.5622    0.1332
```

En Scilab, en lugar de la función “randn” se usa “rand” introduciendo el argumento “normal”.

9.2. Promedio, mediana y desviación estandar

La función “mean” permite recibir un vector de muestras y devuelve el promedio. En este punto cabe recordar que, para esta función, hay tres tipos de promedios, los cuales son el aritmético, el geométrico y el armónico. En Octave se utiliza la función “mean” y dependiendo de qué inputs le demos va a presentar un promedio u otro. Si el único input es un vector, el promedio calculado es el aritmético. Si queremos el promedio geométrico se introduce “g” y si se introduce “h” se obtiene el promedio armónico.

```
>> mean(1:10)
ans = 5.5000
>> mean(1:10,"g")
ans = 4.5287
>> mean(1:10,"h")
ans = 3.4142
```

En Scilab, para calcular estas dos medias se usan otros comandos.

```
--> geomean(1:10)
ans =

    4.5287287

--> harmean(1:10)
ans =

    3.4141715
```

También se puede usar una matriz como input en lugar de un vector. Así, Octave calcula los promedios de cada columna. Esto se ve en el siguiente ejemplo. **Scilab, por su parte, me da el promedio de todos los coeficientes de la matriz. Este da 5.**

```
>> A=[1 2 3;4 5 6;7 8 9]
A =

    1    2    3
    4    5    6
    7    8    9

>> mean(A)
ans =
```

4 5 6

```
--> A=[1 2 3;4 5 6;7 8 9]
A =
```

```
1.    2.    3.
4.    5.    6.
7.    8.    9.
```

```
--> mean(A)
ans =
```

5.

Octave y Scilab permiten también calcular la mediana. La mediana es el valor que deja a una mitad de los datos de un lado y a la otra mitad del otro. Una vez más, Octave calcula la mediana de las columnas mientras que Scilab calcula la mediana teniendo en cuenta a todos los coeficientes de la matriz.

```
>> B=[1 7 7 4;3 5 9 11;5 2 3 9];
>> median(B)
ans =
```

3 5 7 9

```
--> B=[1 7 7 4;3 5 9 11;5 2 3 9];
```

```
--> median(B)
ans =
```

5.

En forma análoga a las anteriores tenemos el comando std, que calcula la desviación estándar de un vector de muestras. En Octave, con una matriz trabaja sobre los valores de cada columna. En Scilab se usa la función stdev y calcula sobre la matriz entera y no sobre las columnas.

```
>> A=[1 2 3;4 5 6;7 8 9]
A =
```

```
1    2    3
4    5    6
7    8    9
```

```
>> std(A)
ans =
```

3 3 3

```
--> A=[1 2 3;4 5 6;7 8 9]
A =
```

```

1.  2.  3.
4.  5.  6.
7.  8.  9.

```

```

--> stdev(A)
ans =

2.7386128

```

9.3. Permutación de enteros aleatorios

La función randperm da un vector que tiene enteros permutados entre 1 y N. Además, permite tomar aleatoriamente solo algunos de ellos. Por ejemplo, en el siguiente ejemplo se presenta una permutación (cambio de orden) de los números entre 1 y 10 y luego se hace lo mismo pero se queda solo con cinco de ellos.

```

>> randperm(10)
ans =

2 5 9 7 1 6 4 3 10 8

>> randperm(10,5)
ans =

10 7 8 5 3

```

Para hacer esto, Scilab usa la función “grand(,”prm”,)”. Como analogía de “randperm” se puede introducir lo siguiente. De este modo se obtiene un resultado análogo al de Octave con “randperm”.

```

--> grand(1,"prm",1:10)
ans =

10. 6. 2. 3. 5. 7. 4. 8. 9. 1.

--> ans(1:5)
ans =

10. 6. 2. 3. 5.

```

9.4. Rango, desviación absoluta promedio y covarianza

Se define al rango de la muestra como la diferencia entre el valor más alto y el más bajo de un conjunto. En Octave se puede calcularlo usando “range”. Si el input fuese una matriz, se calcularía el rango para cada columna, obteniéndose como resultado la diferencia entre el máximo y el mínimo valor para cada una de ellas.

```

>> range(1:10)
ans = 9
>> A=[1 2 3;4 5 6;7 8 9]
A =

```

1	2	3
4	5	6
7	8	9

```
>> range(A)
ans =
```

6	6	6
---	---	---

Scilab hace esto con el comando “strange”. Es exactamente igual si el input es un vector. Si fuese una matriz toma todos los valores de la misma y calcula la diferencia.

```
--> strange(1:10)
ans =
```

9.

```
--> A=[1 2 3;4 5 6;7 8 9]
A =
```

1.	2.	3.
4.	5.	6.
7.	8.	9.

```
--> strange(A)
ans =
```

8.

También se puede calcular la desviación absoluta promedio, que es la función “mad”. Esta está disponible para Octave y Scilab. Opera tanto con un vector como con una matriz. Para un vector, esta función opera del mismo modo para los tres softwares. Por otro lado, si el input es una matriz, en Octave toma los valores de las columnas y da la desviación absoluta para cada columna, **mientras que para Scilab da el resultado usando los coeficientes de toda la matriz.**

$$MAD = \frac{\sum |x_i - \bar{x}_i|}{N}$$

Donde x_i es la i-esima muestra, $\bar{x}_i$ es el promedio y N es la cantidad de muestras.

```
>> mad(1:10)
ans = 2.5000
>> A
A =
```

1	2	3
4	5	6
7	8	9

```
>> mad(A)
ans =
```

```

2      2      2

--> mad(1:10)
ans =

2.5

--> A=[1 2 3;4 5 6;7 8 9]
A =

1.    2.    3.
4.    5.    6.
7.    8.    9.

--> mad(A)
ans =

2.2222222

```

Finalmente, en Octave este comando puede usar el promedio o la mediana. Este cambio se realiza introduciendo un nuevo input que puede ser 1 o 0. En el primer llamado se usó la mediana y en el segundo el promedio. Notar que es indistinto introducir el cero como input.

```

>> mad([1 2 57 21 3],1)
ans = 2
>> mad([1 2 57 21 3],0)
ans = 17.760

```

Octave y Scilab también permiten calcular la covarianza. Para esto se usa la función “cov” y si el input es solo un vector lo que calcula es la varianza de este. En cambio, se introduzco dos vectores obtengo la covarianza entre ellos. **En Scilab funciona igual pero los vectores tienen que ser columnas. Además, informa también la varianza de los vectores de los inputs, ya que tiene como output cuatro resultados. En este caso, la varianza del primer input es 11, la del segundo es 99 y la covarianza es 33.**

```

>> cov([1:11],[1:3:31])
ans = 33

--> cov([1:11],[1:3:31]) %Los inputs son vectores fila
at line 150 of function cov ( C:\Program Files\scilab-
6.1.1\modules\statistics\macros\cov.sci line 163 )

inconsistent row/column dimensions

--> cov([1:11]',[1:3:31]') %Los inputs son vectores columna
ans =

11.    33.
33.    99.

```

Si el input es una matriz, Octave y Scilab toman cada columna como un conjunto de muestras y las combina de a pares, calculando cada covarianza. La coordenada 1,1 es la covarianza de la columna 1 consigo misma (es lo mismo que la varianza). La coordenada 1,2 es la covarianza entre la columna 1 y la 2 (la cual es la misma que la coordenada 2,1, ya que las columnas involucradas son las mismas). Y así continua.


```
>> A=[1:11;1:2:21;1:3:31] '
A =
```

```

1      1      1
2      3      4
3      5      7
4      7     10
5      9     13
6     11     16
7     13     19
8     15     22
9     17     25
10     19     28
11     21     31
```

```
>> cov(A)
ans =
```

```

11     22     33
22     44     66
33     66     99
```



La matriz A se la definió como de 11 x 3 pero luego se la transpuso ya que la función “cov” toma a los vectores columna como muestras. Si no se hubiese transpuesto la matriz, Octave y Scilab habría tomado a las filas de tres coeficientes como muestras y la matriz de covarianzas calculada hubiese dado otro resultado, sin ningún sentido.

9.5. Operaciones con números complejos

En esta sección se van a presentar operaciones con números complejos. Estos se pueden definir en escalares, vectores y matrices. Para acotar espacio se definirán números complejos como coeficientes dentro de una matriz. En Octave se utiliza la letra “i” o “j” para designar la parte imaginaria, **mientras que en Scilab se utiliza “%i”**.

```
>> A=[1 21;3 4i]
A =
```

```

1 + 0i    21 + 0i
3 + 0i     0 + 4i
```

```
>> B=[5 6; 7i -8i]
B =
```

```

5 + 0i    6 + 0i
0 + 7i     0 - 8i
```

```
--> A=[1 21;3 4*%i]
A =
```

```

1. + 0.i    21. + 0.i
3. + 0.i     0.  + 4.i
```

```
--> B=[5 6; 7*%i -8*%i]
B =
```

```
5. + 0.i    6. + 0.i
0. + 7.i    0. - 8.i
```

Estos números se pueden sumar, multiplicar, dividir siguiendo las reglas del álgebra matemático.

```
>> A+B
ans =
```

```
6 + 0i    27 + 0i
3 + 7i    0 - 4i
```

```
>> A*B
ans =
```

```
5 + 147i    6 - 168i
-13 + 0i    50 + 0i
```

```
>> A/B
ans =
```

```
1.8902 + 0i    0 + 1.2073i
0.2927 + 0.3415i -0.2439 - 0.2195i
```

Se puede también calcular la norma de un número complejo. Se llama “abs” y se puede aplicar a complejos escalares, vectores y matrices con componentes imaginarios.

```
>> abs(A+B)
ans =
```

```
6.0000    27.0000
7.6158    4.0000
```

Octave también puede calcular el ángulo de un número complejo usando la función “angle”. Este calcula el ángulo de cada coeficiente complejo de una matriz, y es aplicable a escalares o vectores también. El resultado se da en radianes. **Por su parte, Scilab no tiene esta función o alguna que se le parezca, por lo que tiene que calcular en ángulo usando las funciones que ya tiene incorporadas. Esto se realiza en el siguiente ejemplo.**

```
>> angle(A+B)
ans =
```

```
0    0
1.1659 -1.5708
```

```
--> atan(imag(A),real(A))
ans =
```

```
0.    0.
0.    1.5707963
```

En Octave y Scilab se puede también calcular el conjugado usando “conj”, Además de usar “imag” y “real” para tener las partes imaginarias y complejas.

```
>> conj(A)
```

```
ans =
```

```
1 - 0i    21 - 0i  
3 - 0i    0 - 4i
```

```
>> imag(A)
```

```
ans =
```

```
0    0  
0    4
```

```
>> real(A)
```

```
ans =
```

```
1    21  
3     0
```

Capítulo 10

Strings



OCTAVE



10.1. Introducción a strings

Los strings son textos que pueden imprimirse en la ventana de comandos o incluso ser llevadas a un archivo Word o Excel. Los mismos se entienden como vectores de chars. En esta sección se presentará algunas funciones relacionadas con strings.

Para introducir un string y presentarlo en ventana de comandos, Octave usa la función “fprintf”. Al escribir esto en ventana de comandos se presenta el texto en ella.

```
>> fprintf("these are the medical test results of each patient\n")
these are the medical test results of each patient
```

El texto aparece sin paréntesis, comillas y el \n. Este último símbolo sirve para que se presente el texto y luego el cursor pase a la línea siguiente de la ventana de comandos. Si se escribe el texto sin estos símbolos se obtiene lo siguiente. Es decir que en el fprintf no solo se introduce el texto sino que también se define su formato. **Todo esto es válido en Scilab, solo que la función que hay que usar es “mfprintf” y hay que anteponer el argumento “6”, que indica que el texto se debe imprimir en la ventana de comandos.**

```
--> mfprintf(6,"These are the medical test results of each patient\n")
These are the medical test results of each patient
```

En Octave y Scilab, los strings pueden ser guardados en variables y luego ser llamados por medio de los “fprintf” o “mfprintf”. Por ejemplo, como se observa a continuación.

```
>> MyString="These are the medical test results of each patient\n";
>> fprintf(MyString)
These are the medical test results of each patient
```

También se puede guardar strings en vectores, pero la forma en que son almacenados es diferente en Octave que en Scilab.

En Octave, todos los chars se guardan como se fueran parte del mismo string. Por ejemplo,

```
>> MyString=["Hi","How are you","Have a nice day"];
>> MyString(1:10)
ans = HiHow are
```

Al llamar los índices del 1 al 10, se llama las primeras diez letras de MyString, y el resultado no distingue si cada char pertenece al primero (Hi), segundo (How are you) o tercer vector (Have a nice day).

Por su parte, en Scilab se almacena cada string en una coordenada de la matriz o el vector. Es decir que, en lugar de coeficientes numéricos hay coeficientes de letras.

```
--> MyString=["Hi","How are you","Have a nice day"];
--> MyString(1:2)
ans =

    "Hi"    "How are you"
```

La función “mfprintf” requiere como input un string. Es decir que solo puedo usar solo uno de los tres strings guardados en “MyString”. Esto se presenta a continuación. Observar que al introducir dos strings (al usar 1:2) el programa arroja error.

```
--> fprintf(6,MyString(1))
```

```
Hi
```

```
--> fprintf(6,MyString(2))
```

```
How are you
```

```
--> fprintf(6,MyString(1:2))
```

```
fprintf: Wrong type for input argument #2: A String expected.
```

Además, las dos funciones “fprintf” y “mfprintf” pueden imprimir valores de variables del workspace, intercalados por textos. Sin embargo, estas variables deben ser introducidas como inputs dentro de la función. Un ejemplo de esto se aprecia a continuación:

```
>> fprintf("Patient %d: Results: T1= %2.2f T2=%2d%% Bd1= %g Lt=%e Hb= %s\n", [0:2], 3, 4, "OK");
```

```
Patient 0: Results: T1= 1.00 T2= 2% Bd1= 3 Lt = 4.000000e+00 Hb= OK
```

Donde aparece el símbolo “%” se introduce un número o el valor de una variable, el/la cual es el/la que corresponde por orden de presentación al final del comando “fprintf”. En este ejemplo, a los primeros tres les corresponde los tres valores del vector (0, 1 y 2), el que se encuentra más a la izquierda, luego de la coma. Después, se utiliza el 3. Y así se sigue, hasta llegar al “OK”. En definitiva, el “fprintf” se encuentra con un “%” va a buscar el correspondiente número.

En Octave y Scilab se puede hacer algo similar usando variables. El ejemplo a continuación explica esto para Octave, pero se hace igual en Scilab:

```
>> A=[0:2];b=3;c=4;d="OK";
```

```
>> fprintf("Patient %d: Results: T1= %2.2f T2=%2d%% Bd1= %g Lt=%e Hb= %s\n",A,b,c,d);
```

```
Patient 0: Results: T1= 1.00 T2= 2% Bd1= 3 Lt = 4.000000e+00 Hb= OK
```

Todo esto es igual para Scilab:

```
--> fprintf(6,"Patient %d: Results: T1=%2.2f T2=%2d%% Bd1=%g Lt=%e Hb=%s\n", [0:2], 3, 4, "OK");
```

```
Patient 0: Results: T1= 1.00 T2= 2% Bd1= 3 Lt = 4.000000e+00 Hb= OK
```

```
--> A=[0:2];b=3;c=4;d="OK";
```

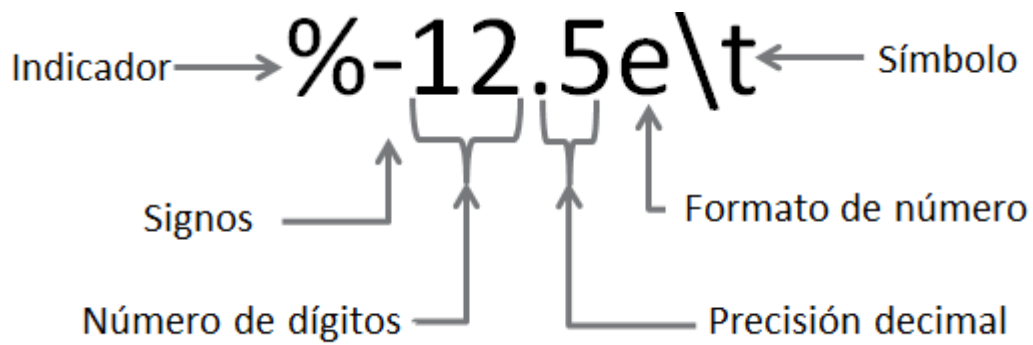
```
--> fprintf(6,"Patient %d: Results: T1=%2.2f T2=%2d%% Bd1=%g Lt=%e Hb=%s\n",A,b,c,d);
```

```
Patient 0: Results: T1=1.00 T2= 2% Bd1=3 Lt=4.000000e+00 Hb=OK
```

10.2. Formato de strings

En esta sección se presenta como darle formato a los números y variables de impresos con las funciones “fprintf” y “mfprintf”. Las opciones de formato son muchas y requeriría mucho tiempo analizarlas.

Cuando se desea imprimir un número se debe introducir el símbolo “%” seguido de los caracteres que le darán el formato a la impresión del número o variable. A continuación se presenta un ejemplo con el significado y las posibles opciones a introducir en cada carácter. Como puede observarse, las opciones para “signos”, “formato de número” y “símbolo” están presentadas en la tabla. Por otro lado, para el número de dígitos y la precisión decimal se puede introducir el número que el usuario desee.



Signos		
Caracter	Descripción	Ejemplo
-	Imprime un + o -	%-3.4d
+		%+3.4d
0	Introduce ceros en los espacios	%03.4d

Formato de Número	
Carácter	Descripción
c	Carácter simple
d	Notacion decimal con signo
e	Notacion exponencial
E	
f	Notacion de punto fijo
g	Notacion exponencial compacta
G	
o	Notacion octal (base 8)
s	Imprime strings
u	Notacion decimal sin signo
x	Notacion hexadecimal
X	

Símbolo		
Carácter		Descripción
Octave	Scilab	
\b		Borrar
\f		Salto de impresión
\n		Continuar en una línea nueva
\r		Retorno de carro 1
\t		Tabulación horizontal
\\		Barra invertida
\"	\"	Comillas
"	"	
%%		Símbolo de porcentaje

Notese que si se va a introducir un string en el “%” se debe indicar usando “%s”. Hay situaciones que el usuario debe introducir un número seguido de un %, como 3%. Entonces, en la parte de símbolos debe introducirse “%%”.

10.3. Recurrencia de impresión de strings

10.3.1 Recurrencia de impresión de strings en Octave

Supongamos que el usuario debe imprimir en la ventana de comandos varias veces el mismo string, pero cambiando las variables que imprime. Un ejemplo de esto se presenta a continuación. Estos son los resultados de distintos estudios (T1, T2, Bd1, Bd2) de distintos pacientes (paciente 2, paciente 3, paciente 4 y paciente 5).

```
Patient 2: Results: T1= 26.93 T2=3.62% Bd1= 40.37 Bd2= 2.806000e+01
Patient 3: Results: T1= 63.25 T2=15.04% Bd1= 48.18 Bd2= 5.000000e+01
Patient 4: Results: T1= 40.51 T2=25.3% Bd1= 41.48 Bd2= 4.000000e+01
Patient 5: Results: T1= 91.84 T2=48.19% Bd1= 26.39 Bd2= 7.783000e+01
```

Se puede ver que el string a utilizar es siempre el mismo (se presenta a continuación) pero cambian los valores de las variables que se imprimen. Como se debería proceder?

```
MyString = "Patient %d: Results: T1= %2.2f T2=%2d%% Bd1= %g Bd2= %e\n";
```

Para empezar, se realizará la explicación para Octave. Primero, el usuario debería guardar todas las variables en una matriz (en este caso, F). En las filas de F deben estar ordenadas por orden de impresión las variables que se van a imprimir. Se presenta esto a continuación. Cada fila, en este caso, contiene los resultados de cada estudio de cada paciente. Por ejemplo, en la fila del paciente 2 tenemos los datos de sus estudios, que son 26.93, 3.62, etc.

Patient	T1	T2	Bd1	Bd2
2	26.93	3,62	40,37	28,06
3	63.25	15,04	28,06	50
4	40.51	25,3	41,48	40
5	91,84	48,19	26,39	77,83

La misma se presenta a continuación.

$F = [2 \ 26.93 \ 3.62 \ 40.37 \ 28.06; 3 \ 63.25 \ 15.04 \ 28.06 \ 50; 4 \ 40.51 \ 25.3 \ 41.48 \ 40; 5 \ 91.84 \ 48.19 \ 26.39 \ 77.83];$

Una vez creada la matriz F, el usuario debe introducir un comando “for” que mire de a una fila por vez.

```
>> for i=1:4
    fprintf(MyString,F(i,:));
end
```

Así, Octave presentará primero la línea siguiente, la cual contiene los datos del paciente 2, tomados de la tabla. Y luego, el for bajará a la siguiente fila, usando $i=2$, la cual corresponde al paciente 2. Y así hasta terminar.

Patient 2: Results: T1= 26.93 T2=3.62% Bd1= 40.37 Bd2= 2.806000e+01

Debe tenerse en cuenta que si el resultado de los estudios es un string y no un número entonces habrá que presentar la información del string usando una matriz o vector distinto, ya que una matriz en Octave no puede contener strings y doubles o integers. Ahora se presenta un ejemplo, donde al final aparece un string inpresos como variable.

Patient 2: Results: T1= 26.93 T2=3.62% Bd1= 40.37 Bd2= 2.806000e+01 Hb= NOK
 Patient 3: Results: T1= 63.25 T2=15.04% Bd1= 48.18 Bd2= 5.000000e+01 Hb= NOK
 Patient 4: Results: T1= 40.51 T2=25.3% Bd1= 41.48 Bd2= 4.000000e+01 Hb= NOK
 Patient 5: Results: T1= 91.84 T2=48.19% Bd1= 26.39 Bd2= 7.783000e+01 Hb= OK

Primero se debe redefinir el vector “MyString” para que contenga la nueva variable, la cual se agregará al final.

```
MyString = "Patient %d: Results: T1= %2.2f T2=%2d%% Bd1= %g Bd2= %e  
Hb= %s\n";
```

En este caso, la última variable es un string, el cual puede ser “NOK” o “OK”. Se crea un vector o matriz más que contenga los strings. En este caso es Hb. A continuación se presenta Hb.

Hb
NOK
NOK
NOK
OK

Este vector se introduce en Octave del siguiente modo:

```
Hb = ["NOK"; "NOK" ; "NOK" ; "OK"];
```

Como el estudio Hb aparece al final entonces se agrega al final de la función “fprintf”. El for anterior se ve modificado para incluir este vector.

```
>> for i=1:4
    fprintf(MyString, F(i,:),Hb(i,:));
end
```

Así, el resultado impreso en la pantalla logra introducir los strings de la variable “Hb”.

10.3.2 Recurrencia de impresión de strings en Scilab

En Scilab es similar, pero la diferencia está en como se debe presentar cada variable dentro de la función “mfprintf”. Veamos el caso de que deseamos imprimir los resultados numéricos (es decir, ningún resultado de los estudios del paciente está informado como un string). En Scilab se usa el mismo “MyString”, el cual se presenta a continuación:

```
MyString = "Patient %d: Results: T1= %2.2f T2=%2d%% Bd1= %g Bd2= %e\n";
```

Además, se debe crear una matriz igual a “F”.

```
F=[2 26.93 3.62 40.37 28.09;3 63.25 15.04 28.06 50; 4 40.51 25.3 41.48
40; 5 91.84 48.19 26.39 77.83];
```

La diferencia respecto a Octave se encuentra en como se introduce la información de la matriz “F” en “mfprintf”, lo cual está a continuación. Obsérvese que esta función requiere como inputs a las columnas de F:

```
mfprintf(6,MyString, F(:,1),F(:,2),F(:,3),F(:,4),F(:,5))
```

Notese que no hace falta un comando for. Además, hay que llamar a cada una de las columnas de F individualmente. Es decir, el usuario debe introducir como inputs a “F(:,1),F(:,2),F(:,3),F(:,4),F(:,5)”. Así se obtiene el siguiente resultado.

```
Patient 2: Results: T1= 26.93 T2=3.62% Bd1= 40.37 Bd2= 2.806000e+01
Patient 3: Results: T1= 63.25 T2=15.04% Bd1= 48.18 Bd2= 5.000000e+01
Patient 4: Results: T1= 40.51 T2=25.3% Bd1= 41.48 Bd2= 4.000000e+01
Patient 5: Results: T1= 91.84 T2=48.19% Bd1= 26.39 Bd2= 7.783000e+01
```

En cambio, si se agrega una variable string al final (aparece un último estudio que es Hb y tiene resultados de string, “NOK” y “OK”), como ocurre más abajo, simplemente se debe agregar el vector de strings “Hb” al final de los inputs de “mfprintf”.

```
Patient 2: Results: T1= 26.930000 T2= 3% Bd1= 4.037000e+01 Bd2=28.06 Hb= NOK
Patient 3: Results: T1= 63.250000 T2= 15% Bd1= 4.818000e+01 Bd2=50 Hb= NOK
Patient 4: Results: T1= 40.510000 T2= 25% Bd1= 4.148000e+01 Bd2=40 Hb= NOK
Patient 5: Results: T1= 91.840000 T2= 48% Bd1= 2.639000e+01 Bd2=77.83 Hb= OK
```



También debe cambiarse “MyString” de modo que imprima el string, agregando Hb=%s al final de mismo.

Para lograr presentar lo que está inmediatamente arriba se debe cargar los resultados strings de Hb de los pacientes en un vector o matriz (en este caso se usa el mismo vector “Hb” de la sección anterior) y llamarlo luego de la última columna de F, como se presenta a continuación.

```
Hb = ["NOK"; "NOK" ;"NOK" ;"OK"];
```

```
mfprintf(6,MyString, F(:,1),F(:,2),F(:,3),F(:,4),F(:,5),Hb)
```

Como se observa arriba, se agregó el vector Hb como input de “mfprintf”.

10.4. Funciones de strings

Como ya fue explicado anteriormente se pueden imprimir strings en la ventana de comandos usando “fprintf” o “mfprintf”. También se puede almacenar strings en variables y presentarlas en ella. También se pueden almacenar en variables y cuando se llama estas variables presentan un texto.

Se puede calcular el largo de cada string usando la función “length”, como si fuese un vector.

```
>> "hola"
ans = hola
>> length("hola")
ans = 4
```

Da 4, porque este vector tiene 4 letras. En Octave, como son vectores, entonces se pueden usar índices. Usando un índice o un grupo de ellos puede presentarse una parte del string.

```
>> txt="hola"
txt = hola
>> txt(1:2)
ans = ho
>> txt(2:4)
ans = ola
```

En Scilab es un poco distinto. Cada string es una coordenada distinta.

```
--> txtt=["hola" "chicos" "perros"]
txtt =

    "hola"    "chicos"    "perros"

--> txtt(1)
ans =

    "hola"

--> txtt(2)
ans =

    "chicos"
```

```
--> txtt(3)
ans =

    "perros"

--> txtt(1:3)
ans =

    "hola"    "chicos"    "perros"
```

Incluso se puede usar estos índices para invertir un string. A continuación se presenta esto para Octave y Scilab por separado.

```
>> txt(end:-1:1)
ans = aloh

--> txtt(3:-1:1)
ans =

    "perros"    "chicos"    "hola"
```

Se puede definir si dos strings son iguales.

```
>> strcmp("Perro", "Perro")
ans = 1
>> strcmp("Perro", "Gato")
ans = 0
```

En Octave, da 1 si son iguales y 0 si son distintos. En Scilab es al revés, da 0 si son iguales y un número positivo o negativo si son distintos.

Se puede estudiar si un string se encuentra dentro de otro usando la función “strfind”. El número que obtengo del output informa en que índice del string grande se encuentra el string chico. Esto se entiende más fácil con un ejemplo, como el que se presenta a continuación. Además, si el string más chico no está incluido en el más grande se obtiene como output la matriz vacía. En Scilab es igual, pero la función a usar es `mtlb_strfind`.

```
>> strfind("hola", "la")
ans = 3
```

Como se puede observar, el resultado es 3, ya que el string “la” empieza en la tercera letra de “hola”. También se puede convertir un string en mayúsculas o minúsculas usando las funciones “lower” y “upper”. En el caso de Scilab es igual pero las funciones son `mtlb_lower(“Hola”)` y `mtlb_upper(“Hola”)`.

```
>> lower("Hola")
ans = hola
>> upper("Hola")
ans = HOLA
```

En Octave se puede sumar strings. Si se suman dos strings el resultado es un vector de números. Este vector surge de convertir ambos strings a sus correspondientes enteros usando el código ascii (al final de este capítulo se presenta un cuadro con esta equivalencia), con lo que quedan dos vectores de enteros, y

luego sumarlos. Por ejemplo, como se hizo a continuación. Debe notarse que ambos strings deben tener igual largo.

```
>> "abc"+"def"
ans =

    197    199    201
```

En Scilab la suma de strings es muy sencilla. Simplemente, cuando se suman dos strings, el software los une.

```
--> "abc"+"def"
ans =

"abcdef"
```

También se puede convertir un string de números en números double o un double en un string. En Octave, lo primero es posible usando la función “str2num” y lo segundo usando “num2str”. Se observa en el ejemplo de abajo que con “str2num” se convierte el string “123” en un double y luego se le suma 1, dando 124 (si hubiese sido un string hubiese dado como resultado un vector). Luego, en la tercera sentencia se convierte al double 123 en string y se informa su clase. Se puede constatar que la función “num2str” convirtió al double 123 en string.

```
>> str2num("123")
ans = 123
>> ans+1
ans = 124
>> num2str(123)
ans = 123
>> class(ans)
ans = char
```

En Scilab se usa el comando string() para convertir un número double en string y el strtod() para convertir un string numérico en string.

```
--> strtod("123")
ans =

    123.

--> ans+1
ans =

    124.

--> string(123)
ans =

"123"

--> typeof(ans)
ans =

"string"
```

TABLA DE CARACTERES DEL CÓDIGO ASCII

1	25	49	73	97	121	145	169	193	217	241
2	26	50	74	98	122	146	170	194	218	242
3	27	51	75	99	123	147	171	195	219	243
4	28	52	76	100	124	148	172	196	220	244
5	29	53	77	101	125	149	173	197	221	245
6	30	54	78	102	126	150	174	198	222	246
7	31	55	79	103	127	151	175	199	223	247
8	32	56	80	104	128	152	176	200	224	248
9	33	57	81	105	129	153	177	201	225	249
10	34	58	82	106	130	154	178	202	226	250
11	35	59	83	107	131	155	179	203	227	251
12	36	60	84	108	132	156	180	204	228	252
13	37	61	85	109	133	157	181	205	229	253
14	38	62	86	110	134	158	182	206	230	254
15	39	63	87	111	135	159	183	207	231	255
16	40	64	88	112	136	160	184	208	232	PRESIONA LA TECLA
17	41	65	89	113	137	161	185	209	233	Alt
18	42	66	90	114	138	162	186	210	234	MÁS EL NUMERO
19	43	67	91	115	139	163	187	211	235	CORTESÍA DE:
20	44	68	92	116	140	164	188	212	236	REDEC
21	45	69	93	117	141	165	189	213	237	desde 1976
22	46	70	94	118	142	166	190	214	238	
23	47	71	95	119	143	167	191	215	239	
24	48	72	96	120	144	168	192	216	240	

Capítulo 11

Loops for y while



OCTAVE

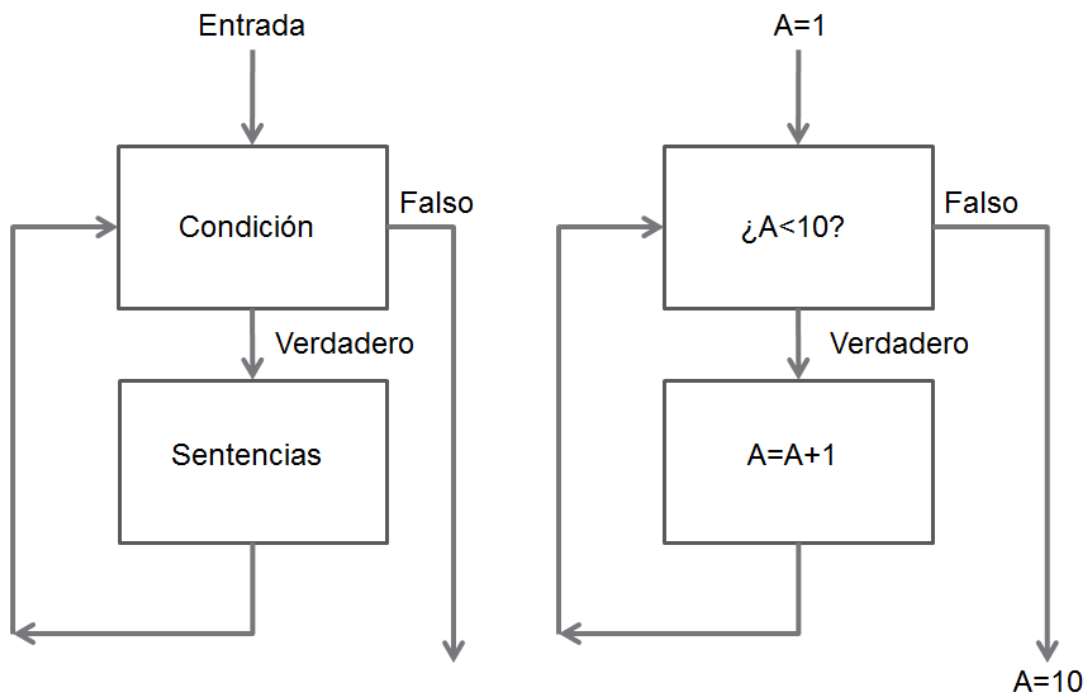


11.1. Loops while

El loop while es una de las herramientas más poderosas de Octave y Scilab. Lo que se explica en esta sección aplica tanto para Octave como para Scilab. Un loop while está formado por una entrada al loop, una condición y sus sentencias. Estos loops sirven para realizar un determinado conjunto de sentencias hasta que se cumpla una condición. A continuación se describe el proceso.

Se entra en el loop por primera vez con una entrada. Si esta entrada no cumple la condición, el loop realiza una serie de sentencias. Al cumplirlas todas, se vuelve a chequear que se cumpla la condición. Si se cumple, se sale del loop. Si no se cumple, se vuelven a correr todas las sentencias. Esto se realiza hasta que se cumpla la condición.

Por ejemplo, se tiene una entrada de $A=1$. La condición para que se siga en el loop es que A sea menor a 10. Es decir, mientras A sea menor a 10 ($A < 10$ es la “condición verdadera” y $A \geq 10$ es la “condición falsa”) el ciclo seguirá aplicando las sentencias. En este caso, las sentencias del loop consisten en sumar 1 a A . Cuando A llegue a 10 la condición será verdadera, el loop while se interrumpe y el script continuará. Se debe notar que A entró en el loop valiendo 1 y salió valiendo 10. A continuación se presenta un esquema de un loop while genérico y el del ejemplo:



También, a continuación se presenta la implementación de este loop en Octave y Scilab.

```
a=1
while a<10 %condición de entrada
    %En esta sección se introducen las sentencias del loop while
    a=a+1 % Sentencia 1

    % Acá iría la sentencia 2

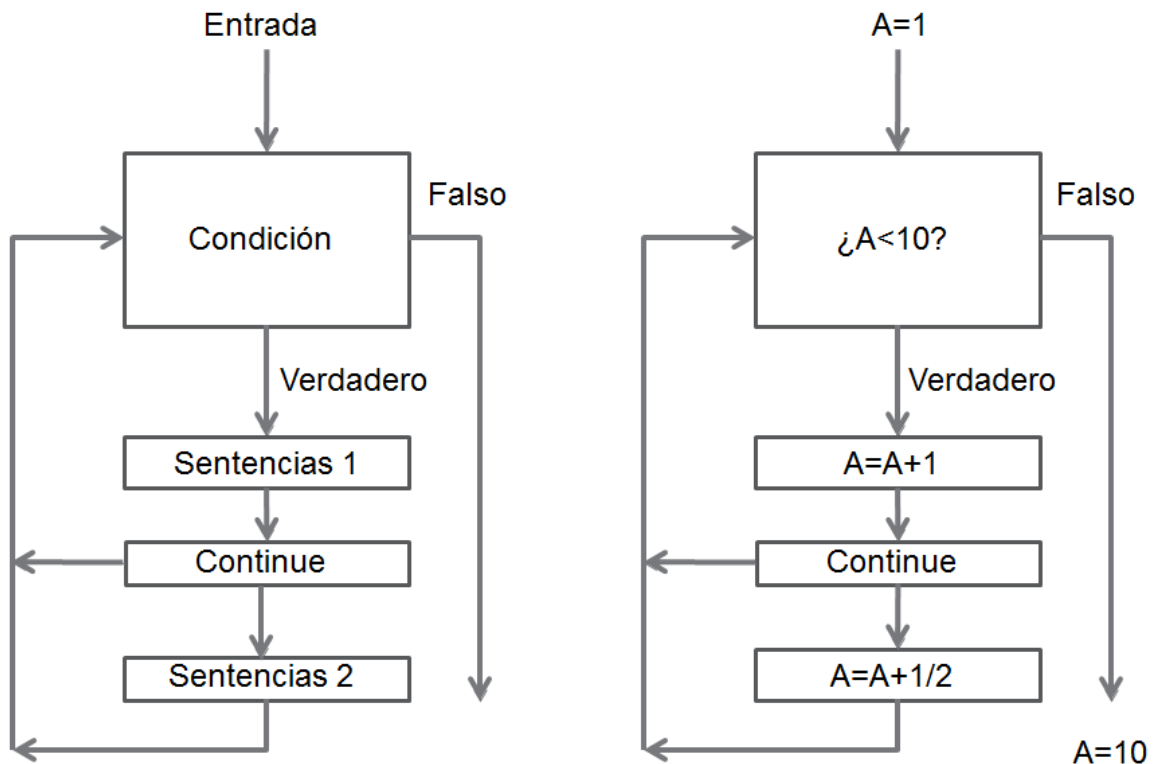
    % Acá iría la sentencia 3
end %Ejecutadas todas las sentencias, vuelve a la condición de entrada
```


11.2. El comando continue

Hay algunos comandos que se pueden usarse dentro de un loop. Por ejemplo, el comando “continue”. Este obliga a que cuando se llega a él, el programa interrumpe el loop (las sentencias debajo del “continue” no se llevan a cabo) y se vuelve a la condición.

A continuación se presenta un esquema de un loop while genérico con un “continue” y un ejemplo de este. En este ejemplo ocurre que se entra al loop con $A=1$. Luego, se le suma 1 a “A”, con lo que ahora $A=2$. Después, se llega al “continue”, por lo que la sentencia en que se suma $1/2$ a “A” no se dispara. Así, se chequea la condición con $A=2$. Como $A=2 < 10$ entonces se vuelve a disparar la primera sentencia y “A” pasa a valer 3. Se vuelve a encontrar con el “continue”, bypassando a la última sentencia. Y se vuelve a la condición. Siguiendo con este ciclo se observa que “A” nunca tiene un valor fraccionario ya que el “continue” impide que se alcance la segunda sentencia (sumar $1/2$ a “A”). Finalmente, se sale del loop con $A=10$.

Más abajo se presenta este loop implementado en un código.



```
a=1
while a<10 %condición de entrada
    %En esta sección se introducen las sentencias del loop while
    a=a+1 % Sentencia 1

    continue %Sentencia 2, el "continue" vuelve a la condición de entrada

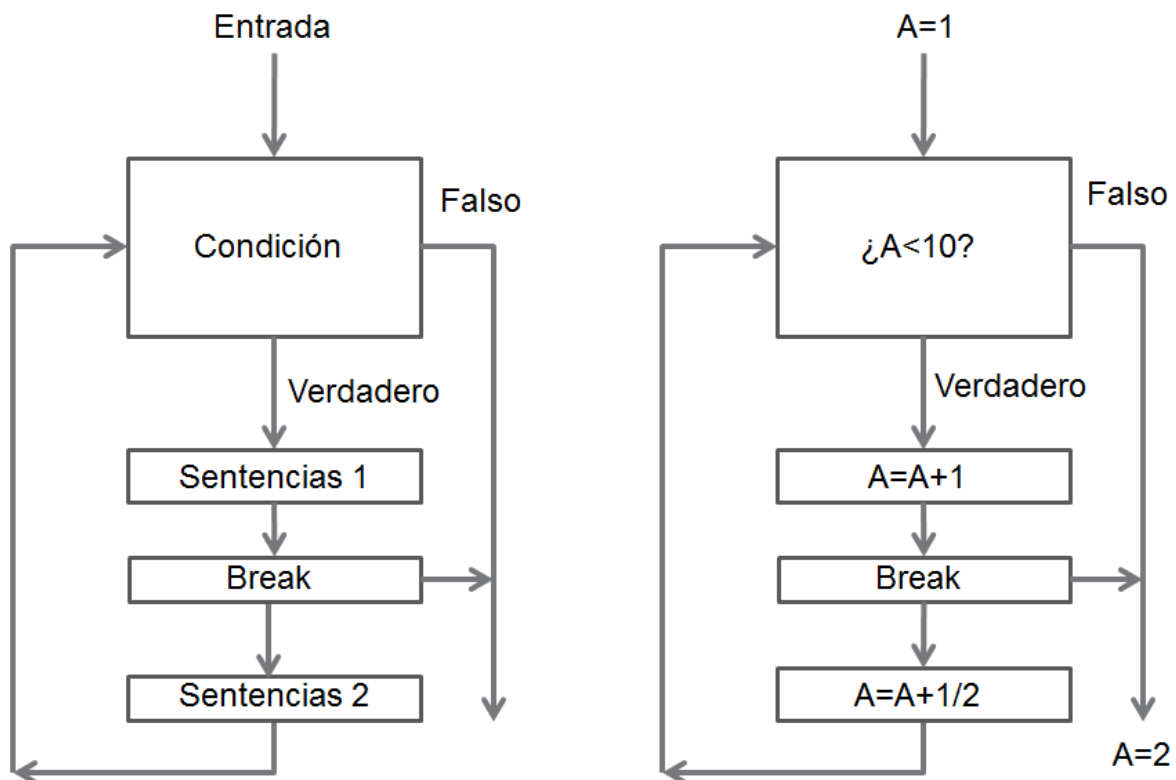
    a=a+0.5 % Sentencia 3, esta sentencia nunca se alcanza
end %Ejecutadas todas las sentencias, vuelve a la condición de entrada
```

11.3. El comando break

El comando “break” se usa en forma similar al “continue” pero su función es distinta. Este comando se usa para interrumpir el loop. Es decir, salir del mismo.

A continuación se presenta un esquema de un loop while con un comando “break”. En este esquema se puede observar que, al llegar al “break” el loop while se interrumpe. Las sentencias que no se hayan ejecutado no se ejecutarán. Además, no volverá a chequearse la condición del loop.

Esto se ve claramente en el ejemplo. Se ingresa en el loop con A=1. Luego, se chequea la condición y, como A=1 es menor a 10, entonces se ejecuta la primera sentencia. “A” pasa a valer 2. Luego, se sigue con lo que viene, que es el comando “break”. Este interrumpe el loop, por lo que se sale de este con A=2.



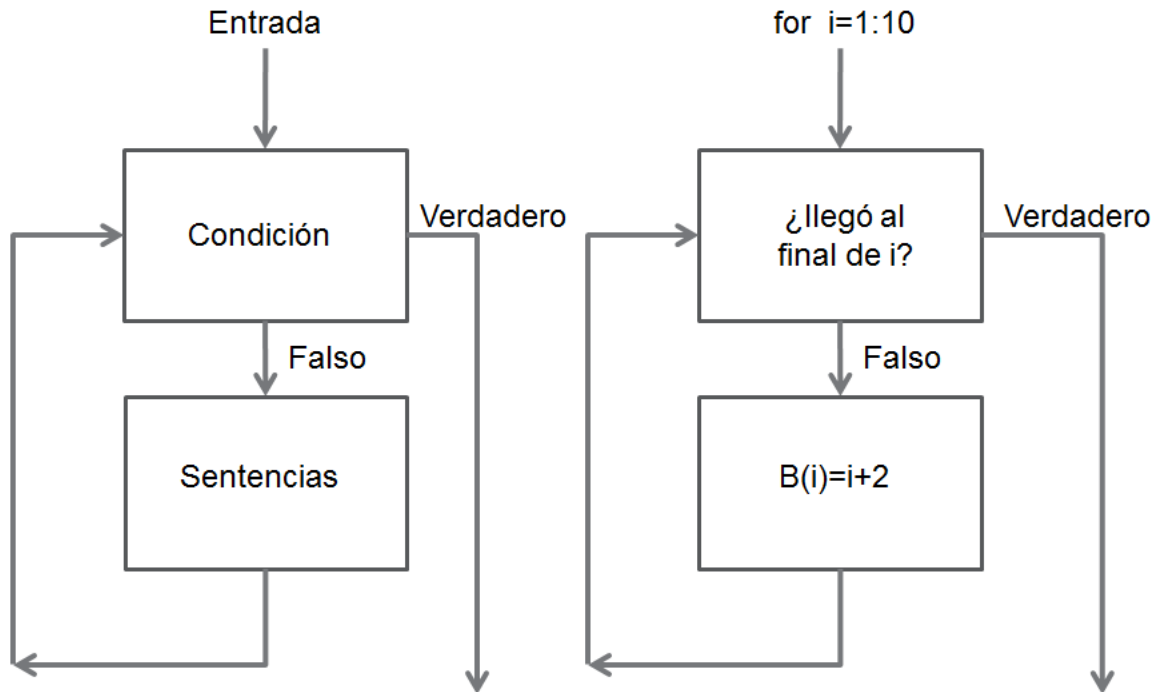
```
a=1
while a<10 %condición de entrada
    %En esta sección se introducen las sentencias del loop while
    a=a+1 % Sentencia 1

    break %Sentencia 2, el "break" sale del loop while

    a=a+0.5 % Sentencia 3, esta sentencia nunca se alcanza
end %Ejecutadas todas las sentencias, vuelve a la condición de entrada
```

11.4. El loop for

Los loops for son similares a los while pero se suelen utilizar cuando se sabe cuántas iteraciones o ciclos se deben realizar. Se pueden entender en forma análoga a los while, solo que los for tienen como condición de salida llegar a la cantidad de ciclos que el usuario especifica.



En el ejemplo se recorre el vector *i* desde 1 a 10, por lo que se realizan diez ciclos o iteraciones. En la primera, "*i*" es igual a 1, en la segunda es igual a 2, y así hasta la última, donde *i*=10. En este ejemplo se carga cada coordenada "*i*" de un vector "*B*" con el resultado de *i*+2. El resultado final que se obtiene es *B*=[3 4 5 6 7 8 9 10 11 12]. A continuación se presenta la implementación de dicho loop en el programa y sus resultados.

```
for i=1:10 %Se realizan 10 ciclos, i empieza en 1 y sigue hasta 10
    %En esta sección se introducen las sentencias del loop for
    B(i)=i+2 % Sentencia 1

    % Acá iría la sentencia 2

    % Acá iría la sentencia 3
end
```

Notar que se corre desde *i*=1 hasta *i*=10. Además, en cada una de las iteraciones se calculó una coordenada nueva para *B*.

Sin embargo, el índice del comando "for" (en el ejemplo anterior es "*i*") no tiene porqué aumentar de uno en uno. Este puede aumentar según como el usuario desee e incluso disminuir. A continuación, se presentan algunos ejemplos. En el primero el salto entre dos valores de "*i*" sucesivos es dos. En el segundo es un número fraccionario y en el tercero es un número negativo (con lo que "*i*" irá para disminuirá en cada ciclo).

```

for i=1:2:10 %Se realizan 5 ciclos, i=1,3,5,7,9
    %En esta sección se introducen las sentencias del loop for
    B(i)=i+2 % Sentencia 1

    % Acá iría la sentencia 2

    % Acá iría la sentencia 3
end

for i=10:-1:1 %Se realizan 10 ciclos, i=10,9,8,...,1
    %En esta sección se introducen las sentencias del loop for
    B(i)=i+2 % Sentencia 1

    % Acá iría la sentencia 2

    % Acá iría la sentencia 3
end

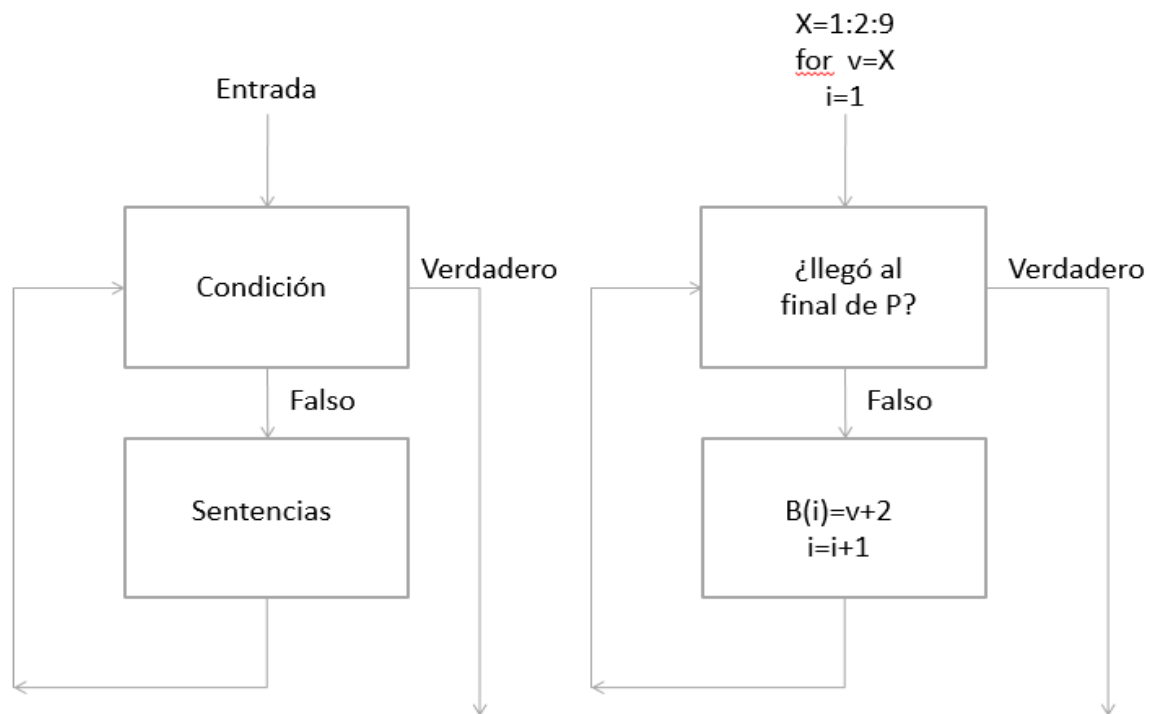
```

Debemos notar que en primer ejemplo hay ceros intercalados en B. Esto requiere una reflexión. El vector B es creado a medida que “i” cambia de valor. Sin embargo, hay algunas posiciones que no reciben ningún valor (los números pares). En ese caso, Octave y Scilab introducen en esas posiciones un cero.

Además, en el segundo ejemplo, en la sentencia 1, el índice “i” aparece multiplicado por 2. Esto se debe a que “i” toma valores fraccionales (1.5, 2.5, ..., 9.5), y estas posiciones no tienen sentido. Es decir, en los vectores no existen las posiciones fraccionales como B(1.5).

Finalmente, en el tercer ejemplo hay que notar que el salto es negativo, por lo que el operador colon toma los operandos 10 y 1, en ese orden, quedando 10-1:1. Esto quiere decir que la secuencia de números es la siguiente: 10, 9, 8, ..., 1. Además, como la primera coordenada de B que se carga es i=10 entonces inicialmente se crea el vector con nueve ceros y un 12 en i=10.

Hay otro modo de aplicar el for. En lugar de tener un índice que es el que marca cada ciclo, se puede reemplazar a este por un vector. Es decir, en el argumento del for no hay un índice sino el vector que el usuario quiere recorrer, de a una coordenada por ciclo. Esto es útil cuando hay que usar todas las coordenadas de un vector. A continuación, se ve esto en el siguiente ejemplo.



```
X=1:2:9;
i=1;
B=zeros(1,5)
for v=X %Cada ciclo se realiza con una de los coeficientes de X
    %La variable v, definida en la condición de entrada, no es el índice
    %La variable v toma los valores v=1,3,5,7,9.
    B(i)=v+2 %Sentencia 1

    i=i+1; %Sentencia 2, i aumenta en 1 para ir recorriendo el vector B

end
```

Como puede observarse, en cada ciclo se usa un valor de “X”, empezando por el primero y siguiendo hasta el último. Por otro lado, el código usa el índice “i” (el cual no se encuentra en la condición de entrada) para recorrer el vector “B”. Debe notarse que en la sentencia 2 se aumenta en uno el valor de “i” ya que, de lo contrario, dicho valor se mantendrá constante y no podrá recorrerse el vector B.



Si bien los comandos “continue” y “break” se usaron en ciclos while, cabe recordar que los mismos son también útiles en ciclos for.

Capítulo 12

Logical indexing



OCTAVE



Logical indexing es una herramienta que permite manipular los coeficientes de los vectores y matrices sin usar largos scripts. El mismo está disponible en los tres softwares y se usan del mismo modo. La única diferencia es que donde aparezcan 1 y 0 logicos en Octave hay que utilizar T y F en Scilab.

A continuación se estudiarán algunas operaciones de logical indexing.

```
>> [-1 0 2 6]>=0
ans =

    0    1    1    1
```

Esta operación consiste en comparar todos los coeficientes del vector de la izquierda con el escalar de la derecha. En este caso, consiste en tomar el primer coeficiente y chequear si es mayor o igual que el escalar de la derecha (cero). Como output se obtiene un vector del tipo logical que tiene un 1 o un 0 en la primera posición dependiendo de si se cumple la desigualdad. En este caso no se cumple, por lo que será un 0. Luego se pasa a la siguiente posición.

En este caso, como la única posición que no cumple es la primera, el vector obtenido es [0 1 1 1]. De un modo similar se pueden hacer comparaciones entre vectores, coordenada a coordenada.

```
>> [1 2 3]>[0 3 4]
ans =

    0    0    0
```

En este caso el resultado es un vector logical [1 0 0] porque solo en la primera posición se cumple la desigualdad (la cual es $1 > 0$).

El logical indexing también permite eliminar coordenadas de un vector. Esto se realiza del modo que se presenta a continuación, pero primero deben definirse dos variables. La primera es $a=[1\ 2\ 3]$ y la segunda es $c=[1\ 0\ 1]$. Esta última es una variable logical. La operación para eliminar coeficientes de un vector o matriz se presenta a continuación. Como se puede observar, se eliminan las posiciones de “a” en donde “c” es cero.

```
>> a=[1 2 3]
a =

    1    2    3

>> c=logical([1 0 1])
c =

    1    0    1

>> A=a(c)
A =

    1    3
```

Otra operación que permite el logical indexing es la de eliminar determinadas coordenadas que no cumplan con una condición. Por ejemplo, se puede pedir que se descarten todas las posiciones de una matriz que no sean mayores a cero. Esto se muestra a través del siguiente ejemplo. Primero se debe definir un vector o matriz (a este se le quitaran las coordenadas que no cumplan la condición), el cual será $X=[-1\ 3\ -2\ 4\ 5\ 6]$. Luego se realiza la operación como se presenta a continuación. Se puede ver que, como -1 y -2 no cumplen la condición, se eliminan.

```
>> X=[-1 3 -2 4 5 6];
>> X(X>0)
ans =
```

```
3 4 5 6
```

De un modo similar al anterior se puede reemplazar aquellos coeficientes que cumplan determinada condición.

```
>> X(X>3.1)=[6 7 8]
X =
```

```
-1 3 -2 6 7 8
```

El resultado obtenido es [-1 3 -2 6 7 8]. Esto es porque se reemplazó los tres coeficientes por los números de la derecha del igual. Pero debe tenerse en cuenta que coincidir la cantidad de números que se reemplazan con los de la derecha. En este ejemplo, la cantidad de números que cumplieron la condición (ser mayores a 3.1) es tres, al igual que la cantidad de valores del vector de la derecha.

El usuario también puede sumarle un determinado valor a algunas coordenadas de un vector o matriz.

```
>> X=[-1 3 -2 4 5 6];
>> X(X<0)=X(X<0)+5
X =
```

```
4 3 3 4 5 6
```

A los coeficientes de X que son menores a 0 se les suma 5 y así se obtiene un vector nuevo. En cambio, a los coeficientes menores a 0 los dejó igual.

Si bien en esta sección todos los ejemplos corresponden a vectores, estas operaciones sirven también para matrices.

Capítulo 13

Operaciones con Word y Excel



OCTAVE

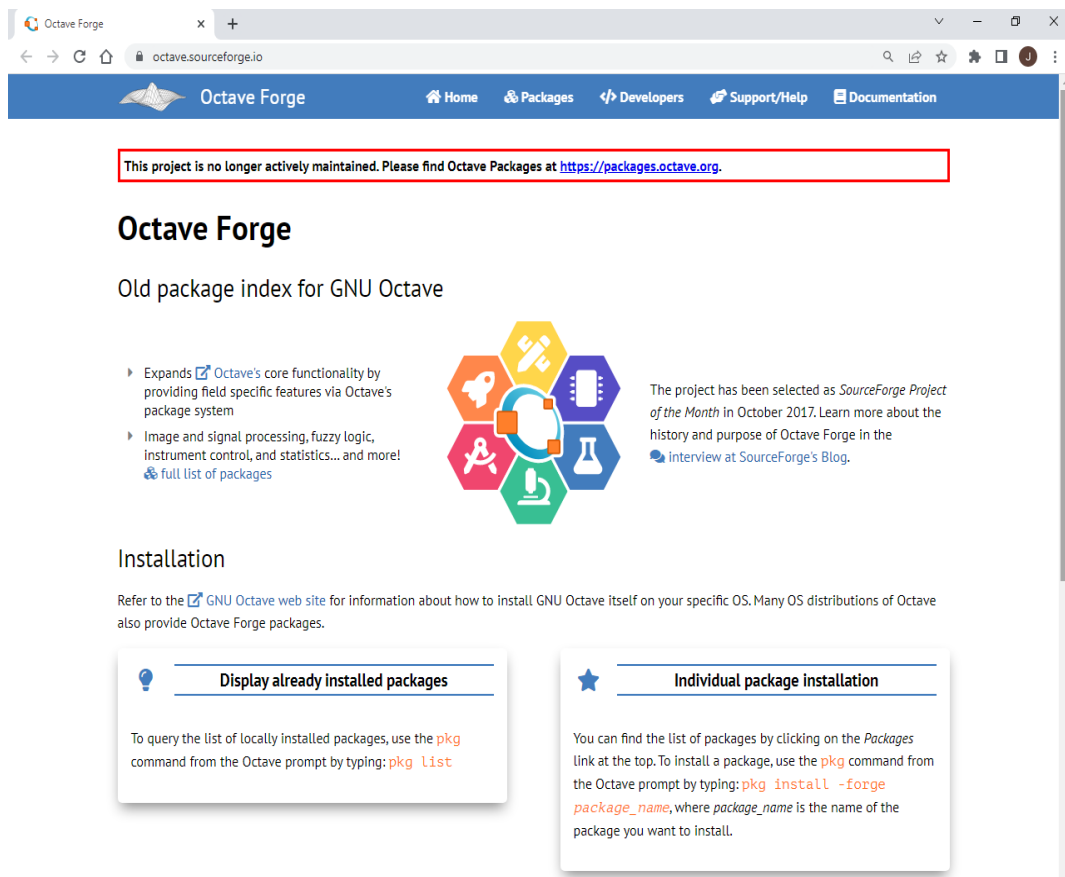


13.1. Uso de paquetes de Octave

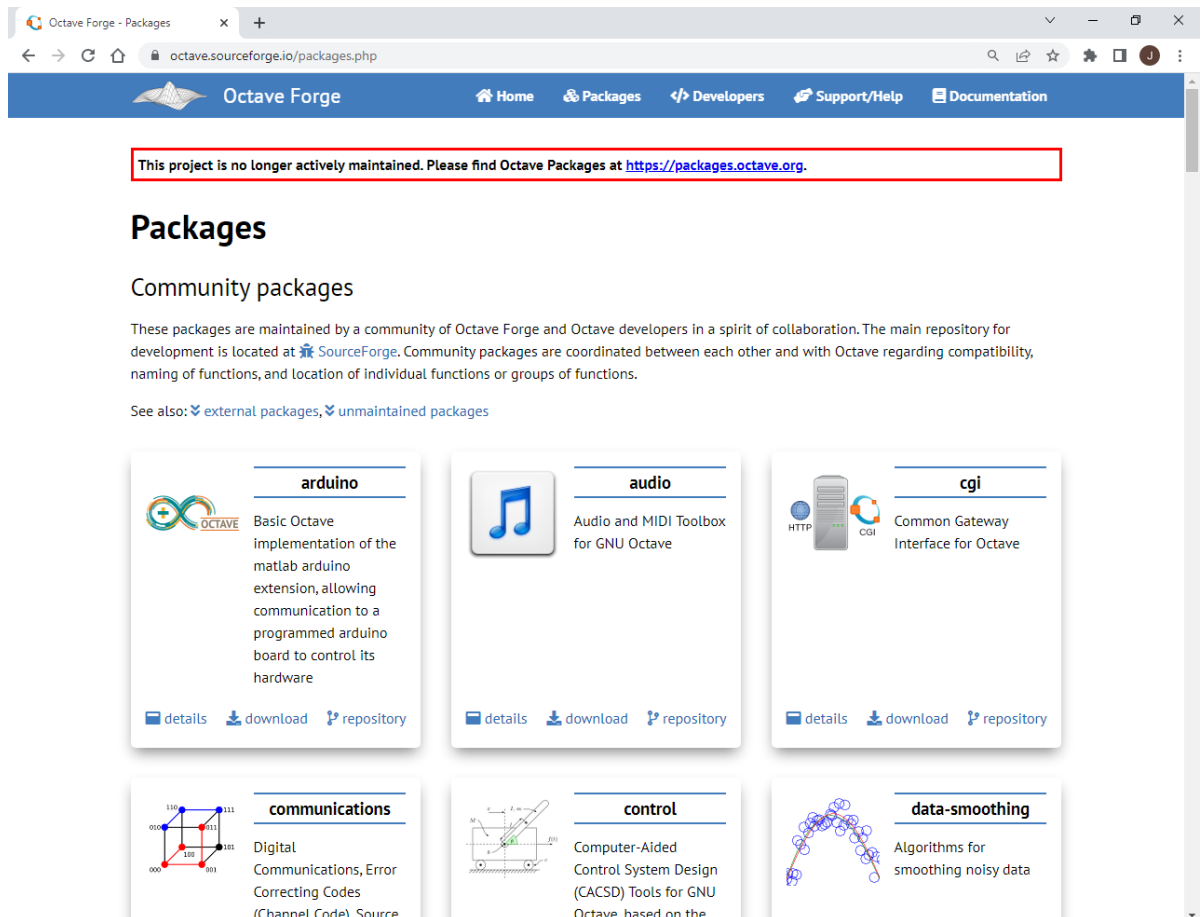
Tanto Octave como Scilab pueden interactuar con archivos de texto y hojas de cálculo. Sin embargo, la actual versión de Octave no tiene las funciones necesarias, mientras que Scilab sí (y por supuesto, lo mismo aplica a Matlab, si lo tuviesen). Esto implica que para usar estas funciones el usuario debe descargar primero el paquete necesario. Un paquete es un conjunto de funciones que fueron desarrolladas por un usuario y las subió a internet. Octave permite incorporar estas funciones al instalar el paquete que las contiene.

A continuación se desarrollan los pasos para descargar e instalar un paquete de funciones:

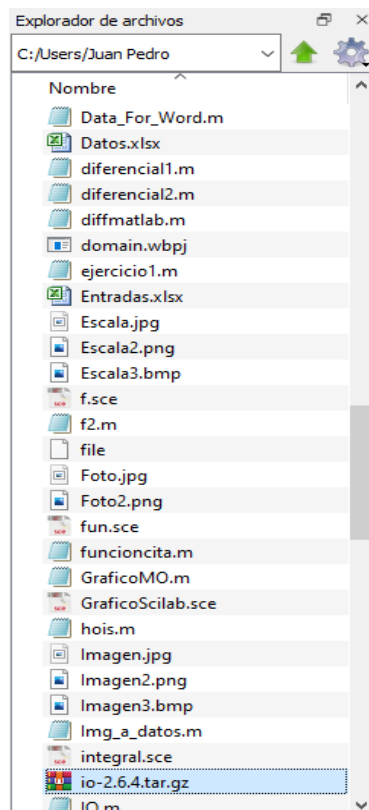
1. Primero el usuario debe saber qué funciones necesita. Eso se logra buscando en internet y viendo cuales están disponibles. Las funciones necesarias para esta lección son las que están en un paquete que se llama “io”, que son las iniciales de “input – output”.
2. Hay una página muy recomendable, la cual es ”Octave sourceforge”. Acá se encuentran muchos paquetes que son muy interesantes. La misma se presenta a continuación.



3. En esta página se debe descargar el paquete que el usuario desea. Es este caso es el paquete “io”, el cual, al momento de escribir esto está en la versión 2.6.4. Para descargar dicho paquete se debe hacer click en la solapa “Packages”, con lo que se abre la siguiente página:



- Se busca el paquete que se desea descargar y se lo guarda en el directorio en que están trabajando (“Explorador de archivos” en la interface de Octave).



5. Luego se introduce el comando `pkg install io-2.6.4.tar.gz` en la ventana de comandos. “pkg” significa “package”, paquete en ingles. “install” es porque es la acción que se va a realizar, que es instalarlo. “io” es para indicarle que paquete se desea instalar. Con un paquete se puede realizar más acciones que instalarlo. Si se desea saber qué más pueden hacer entonces se debe escribir “help pkg”. Instalar el paquete es necesario solo una vez.

```
>> pkg install io
```

6. En la ventana de comandos se debe introducir `pkg load io`. “load” es porque es la acción que se va a realizar, que es cargar las funciones de ese paquete. “io” es para indicarle que paquete debe cargar. Cada vez que se abra Octave hay que cargar el paquete “io” si se desea usarlo.

```
>> pkg load io
```

7. Ya está listo el paquete io para usar.

13.2. Importar información de Excel a Octave

En esta sección se va a estudiar como leer datos guardados en una hoja de Excel. Como ejemplo se tiene una tabla que contiene los puntajes de cinco amigos que fueron a jugar al bowling. Sus nombres son los siguientes:

	A	B	C	D	E	F	G
1	Puntajes	Intentos	Participantes				
2			Chavo	Quico	Chilindrina	Doña Florinda	Don Ramón
3		1	50	90	10	40	90
4		2	60	100	20	50	10
5		3	70	100	30	60	40

Esta tabla se encuentra en un archivo que se llama “Bowling.xlsx”. Este archivo tiene que estar en el mismo directorio en que se está trabajando. Para que Octave pueda levantar esta información hay que primero cargar el paquete io (“pkg load io”) y luego usar la función “xlsread”.

```
>> pkg load io
>> [val,text,all]=xlsread("Bowling.xlsx");
```

Observe que se usó como input un string que es el nombre del archivo de Excel, junto con su extensión. También hay tres outputs, las cuales describimos a continuación. Toda la información de la tabla que son datos numéricos queda en la variable “val”.

```
>> val
val =

     1     50     90     10     40     90
     2     60    100     20     50     10
     3     70    100     30     60     40
```

Como puede observarse, es una matriz y contiene todos los puntajes y el número de intentos. En cambio, cuando se introduce “class(text)” se obtiene lo siguiente:

```
>> class(text)
ans = cell
>> class(all)
ans = cell
```

“text” es una cell (celda). Esto es un tipo de variable que está fuera del alcance de este trabajo. Lo mismo pasa con la variable “all”. Sin embargo, a los fines de aprender como usar la función “xlsread” se puede decir que las cells serían como matrices en las que se pueden guardar números o strings. El contenido de cada una de sus posiciones es igual que en matrices, usando índices, como se observa a continuación:

```
>> text
text =
{
    [1,1] = Puntajes
    [2,1] =
    [1,2] = Intentos
    [2,2] =
    [1,3] = Participantes
    [2,3] = Chavo
    [1,4] =
    [2,4] = Quico
    [1,5] =
    [2,5] = Chilindrina
    [1,6] =
    [2,6] = Doña Florinda
    [1,7] =
    [2,7] = Don Ramón
}
```

```
>> all
all =
{
    [1,1] = Puntajes
    [2,1] = [] (0x0)
    [3,1] = [] (0x0)
    [4,1] = [] (0x0)
    [5,1] = [] (0x0)
    [1,2] = Intentos
    [2,2] = [] (0x0)
    [3,2] = 1
    [4,2] = 2
    [5,2] = 3
    [1,3] = Participantes
    [2,3] = Chavo
    [3,3] = 50
    [4,3] = 60
    [5,3] = 70
    [1,4] = [] (0x0)
    [2,4] = Quico
    [3,4] = 90
    [4,4] = 100
    [5,4] = 100
    [1,5] = [] (0x0)
    [2,5] = Chilindrina
    [3,5] = 10
    [4,5] = 20
    [5,5] = 30
    [1,6] = [] (0x0)
    [2,6] = Doña Florinda
    [3,6] = 40
}
```

```

    [4,6] = 50
    [5,6] = 60
    [1,7] = [] (0x0)
    [2,7] = Don Ramón
    [3,7] = 90
    [4,7] = 10
    [5,7] = 40
}

>> all(1,1:3)
ans =
{
    [1,1] = Puntajes
    [1,2] = Intentos
    [1,3] = Participantes
}

>> all(1:2,1:4)
ans =
{
    [1,1] = Puntajes
    [2,1] = [] (0x0)
    [1,2] = Intentos
    [2,2] = [] (0x0)
    [1,3] = Participantes
    [2,3] = Chavo
    [1,4] = [] (0x0)
    [2,4] = Quico
}

```

Puede ser que al usuario solo le interese cierta información y no toda la tabla. Por ejemplo, solo le interesan los números, entonces puede introducir la función “xlsread” como sigue, tachando los dos outputs que no le interesan:

```

>> [val,~,~]=xlsread("Bowling.xlsx")
val =

    1    50    90    10    40    90
    2    60   100    20    50    10
    3    70   100    30    60    40

```

En las últimas entradas se omitió informar a Octave de qué página tiene que obtener la información. Por eso, por default, toma los datos de la hoja 1. Se lo que se busca es obtener información de la segunda hoja, por ejemplo, debe informarse como input.

```

>> [val,text,all]=xlsread("Bowling.xlsx",2);

```

Si, además, el usuario desea información de un conjunto de celdas específicas (de la hoja de cálculo) se debe introducir dichas celdas como input. En el ejemplo que sigue a continuación se explica como levantar la información que hay entre las celdas C2 e G5.

```

>> [val,text,all]=xlsread("Bowling.xlsx",2,"C2:G5");

```



```

0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
0.
0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
0.

```

Como puede verse, la presentación de la información no es muy clara, al menos el texto. Por eso, para ordenar un poco, se pueden introducir las siguientes sentencias:

```

Maximum=max(TextInd);
for i=1:Maximum
    [x,y]=find(TextInd==i);
    text(x,y)=SST(TextInd(x,y));
end

```

Ahora la variable “text” presenta de un modo mucho más ordenado la información.

```

--> text
text =

    "Puntajes"    "Intentos"    "Participantes"    ""            ""            ""
""
    ""            ""            "Chavo"            "Quico"    "Chilindrina"
"Doña Florinda"    "Don Ramón"

```

13.4. Exportar información de Octave y Scilab a Excel

Octave y Scilab también pueden guardar el valor de sus variables en una hoja de cálculo. Solo debe especificarse que variable el usuario desea guardar y en qué archivo de Excel. Se mostrará como hacerlo a continuación.

Primero debe tenerse un conjunto de variables. Solo para fines didácticos se creará la variable A=[1:7;8:14;15:21;22:28;29:35].

```

>> A=[1:7;8:14;15:21;22:28;29:35]
A =

```

```

1     2     3     4     5     6     7
8     9    10    11    12    13    14
15    16    17    18    19    20    21
22    23    24    25    26    27    28
29    30    31    32    33    34    35

```

Luego tiene que introducirse la función “xlswrite”. Esta tiene dos inputs. Una de ellas es el nombre de la variable que se desea guardar (en este caso “A”) y el nombre del archivo Excel en que se guardará la misma.

```

>> xlswrite("Sucesion",A)
ans = 1

```

De este modo se creó un archivo hoja de cálculo que se llama “Sucesión” y contiene los valores de la variable “A”.

	A	B	C	D	E	F	G
1	1	2	3	4	5	6	7
2	8	9	10	11	12	13	14
3	15	16	17	18	19	20	21
4	22	23	24	25	26	27	28
5	29	30	31	32	33	34	35

Si se desea guardar esta información en una hoja específica, como por ejemplo la 2, se debe introducir como input el nombre de la hoja. En el ejemplo siguiente la variable “A” es guardada en la Hoja2.

```
>> xlswrite("Sucesion",A,"Hoja 2")
ans = 1
```

Finalmente, si se desea guardar esta variable en una celda (o conjunto de celdas) determinadas se debe introducir un nuevo input, el cual es un string que contiene las celdas en que se ubicarán los datos de la variable “A”.

```
>> xlswrite("Sucesion",A,"Hoja 2","G10:M14")
ans = 1
```

Por su parte, Scilab puede guardar variables en archivos csv. Para eso hay que proceder en forma similar a cuando se trabaja en Octave. A continuación, se verá cómo proceder, usando la misma variable A que en Octave.

En Scilab se guarda información en una hoja de cálculo usando la función “csvWrite”.

```
--> A=[1:7;8:14;15:21;22:28;29:35]
A =
```

```
1.    2.    3.    4.    5.    6.    7.
8.    9.   10.   11.   12.   13.   14.
15.   16.   17.   18.   19.   20.   21.
22.   23.   24.   25.   26.   27.   28.
29.   30.   31.   32.   33.   34.   35.
```

```
--> csvWrite(A,"Sucesion.csv")
```

	A	B
1	1,2,3,4,5,6,7	
2	8,9,10,11,12,13,14	
3	15,16,17,18,19,20,21	
4	22,23,24,25,26,27,28	
5	29,30,31,32,33,34,35	

Los datos de la matriz aparecen separados por comas. Generalmente es más agradable de ver cuando, en lugar de comas hay espacios. Esto se realiza introduciendo un nuevo input, el cual es un string de un carácter, el cual se usará para separar los números. A continuación, se muestra como separar los números usando un espacio, pero podría introducirse otros símbolos poniéndolos en el tercer input como strings. Por ejemplo, “~” introduce este símbolo como separador.

```
--> csvWrite(A,"Sucesion.csv"," ")
```

	A	B
1	1 2 3 4 5 6 7	
2	8 9 10 11 12 13 14	
3	15 16 17 18 19 20 21	
4	22 23 24 25 26 27 28	
5	29 30 31 32 33 34 35	

13.5. Exportar información de Octave y Scilab a un archivo de texto

La explicación que sigue a continuación es para Octave. Para Scilab es exactamente igual, solo que deben reemplazarse los nombres de las funciones según el cuadro que se presenta a continuación:

Comando	Función	
	Scilab	Octave
Abrir un archivo de texto	id=mopen(id)	id=fopen(id)
Desligarse del archivo de texto	mclose(id)	fclose(id)
Escribir un string en archivo de texto	mprintf(id,"")	fprintf(id,"")
Obtiene texto de un archivo	mgetl(id,integer)	fgets(id,integer)

En esta sección se va a estudiar cómo escribir strings en Octave y Scilab y que estos luego sean impresos en un archivo de texto en lugar de en la ventana de comandos. Antes de empezar debe informarse a Octave y Scilab en que archivo se va a escribir. Para eso se usa el comando “fopen”. Este comando va a encontrar el archivo que se le informe como input y le va a asignar un número para identificarlo. Pero además tenemos que informarle a “fopen” con qué fin se va a abrir el archivo. Esto quiere decir que se va a introducir otro input, cuyas opciones se presentan en la siguiente tabla, según querremos abrir el archivo de texto para leer o escribir.

Argumento	Descripción	Observaciones
"rt"	Abre solo para lectura	-
"wt"	Abre el archivo para escribir	Contenido previo del archivo es descartado
"at"	Abre para empezar a escribir al final del archivo	-
"r+t"	Abre un archivo para leer y escribir	-
"w+t"	Abre un archivo para leer y escribir	Contenido previo del archivo es descartado
"a+t"	Abre para empezar a escribir o leer al final del archivo	-

Si el segundo input de "fopen" es "rt" el archivo se abre para leer. No se puede escribir en él. No se escribe en él. En cambio, si el input es "wt" sí se puede escribir, pero debe tenerse en cuenta que el contenido previo es eliminado. También se puede usar "at", que es para empezar a escribir al final, y si el archivo no existe, lo crea.

Si a estas opciones le agregamos un "+", por ejemplo "w+t", todo sigue igual solo que se podrá escribir o leer al mismo tiempo (ver como se indica en la tabla de arriba).

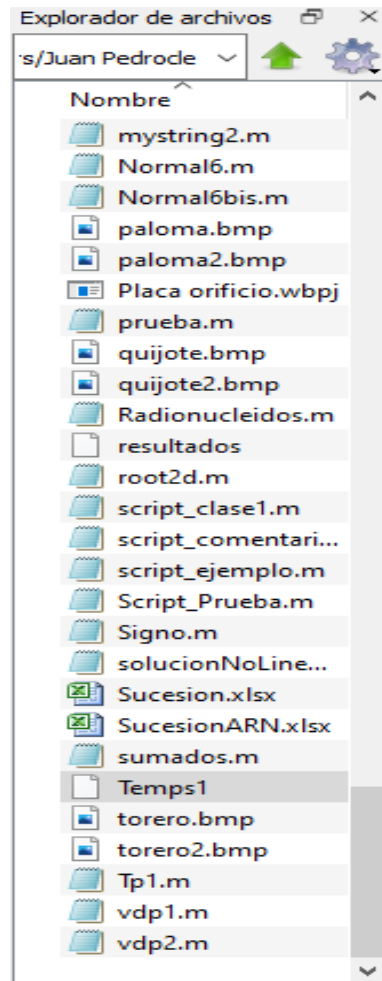
A continuación, se usará la función "fopen". Se empezará trabajando con un archivo llamado Temps1. Se va a empezar tratando de leer un archivo. Para eso se usa el comando siguiente. Si el archivo existe entonces ahora está abierto y podremos ir leyendo de a poco lo que dice, cuando usemos otro comando que se llama "fgets". La función "fopen" abre el archivo Temps1 y le asigna un número. Si el archivo existe y se encuentra dentro del directorio de trabajo, entonces "fid" debería tener un valor entero mayor a cero. Si el archivo no existe, el resultado de "fid" es -1.

```
>> fid=fopen("Temps1","rt")
```

Para fines didácticos, se supone que "Temps1" no existe, por lo que fid=-1. Entonces debe crearse el mismo. Para eso puede usarse la siguiente sentencia:

```
>> fid=fopen("Temps1","wt")
fid = 4
```

Como Temps1 es creado entonces recibe un valor entero para identificarlo. En este caso es 1. Acá debe tenerse en cuenta que si el archivo "Temps1" hubiese existido al momento de ejecutar "fopen" con el input "wt" entonces se habría sobrescrito su contenido. Pero como este no es el caso, se acaba de crear el archivo llamado "Temps1" y se puede escribir en él. Dicho archivo, además, debería estar en el directorio de trabajo.



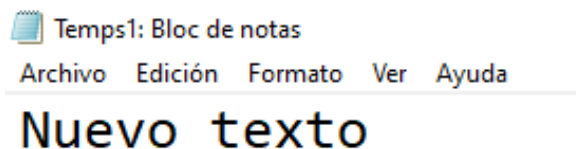
En este punto se procede a escribir en él. La línea que se introducirá es “Nuevo texto”.

```
>> fprintf(fid,"Nuevo texto")
```

En Octave se usa la función “fprintf” y tiene como argumento a “fid”, el identificador de “fopen”. Si ya se terminó de escribir en ese archivo entonces se introduce el siguiente comando

```
>> fid=fclose("Temps1")  
fid = 0
```

Así, el archivo Temps1 quedó como sigue:



Ahora se va a agregar un nuevo texto al archivo que recientemente se cerró. Recordemos que si se busca agregar texto al final del archivo sin borrar su contenido se debe usar el comando “fopen” y el input “at” en él.

```
>> fid=fopen("Temps1","at")  
fid = 3
```

```
>> fprintf(fid,"Nuevo texto 2")
>> fid=close("Temps1")
fid = 1
```

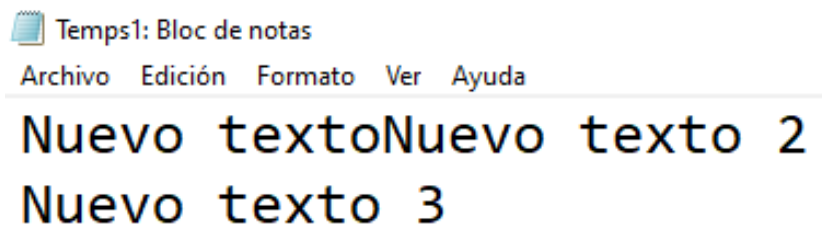
A continuación, se presenta el nuevo contenido de “Temps1”. Se observa que se agregó un nuevo texto.



Se puede observar que apareció el nuevo texto al lado del anterior. Sin embargo, hubiese quedado mejor si este se hubiese agregado en la línea de abajo. Para eso se debe indicar a Octave y Scilab que el usuario desea bajar a la línea inferior. Es decir que debería haberse agregado un “\n” antes de imprimir la última línea. Esto es lo que se realizará a continuación. Se agregará un nuevo texto en la línea de abajo y para eso se usará “\n”.

```
>> fid=fopen("Temps1","at")
fid = 5
>> fprintf(fid,"\n")
>> fprintf(fid,"Nuevo texto 3")
>> fid=close("Temps1")
fid = 1
```

El archivo de texto queda como sigue:



Con esto se considera que el alumno está en condiciones e imprimir información en un archivo de texto. Sin embargo, cabe recordar que también se pueden definir variables numéricas o strings en Octave y Scilab y plasmarlas en el texto. A continuación, se verá cómo hacerlo.

Como ejemplo se van a definir dos variables de este tipo y se las imprimirá en el archivo de texto. Las dos variables se las define en el software y son las siguientes:

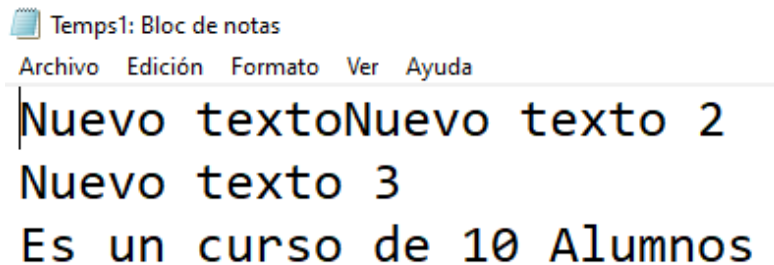
```
>> Str="Alumnos"
Str = Alumnos
>> NP=10
NP = 10
```

Como se observa, una de ellas es un string y la otra es una variable numérica. Ahora se verá como imprimirlas en el archivo de texto

```
>> fid=fopen("Temps1","at")
fid = 6
>> fprintf(fid,"\n")
>> fprintf(fid,"Es un curso de %s %d",Str,NP)
```

```
>> fid=close("Temps1")
fid = 1
```

Como puede verse, se usaron las mismas funciones vistas en esta sección y las reglas del formato vistas en el capítulo 10 de strings (solo que esta vez se imprimió en un archivo de texto y no en la ventana de comandos).



13.5. Las funciones “fgets” y “mgetl”

Para obtener una parte del texto de un archivo de texto en Octave se usa la función “fgets” **mientras que en Scilab se usa “mgets”**. Para usarlas se debe usar los comandos “fopen” (Octave) o “mopen” (Scilab) y luego usar las funciones “fgets” o “mgetl” según corresponda si lo que se busca es leer texto de un archivo. Esto se observa a continuación:

```
>> fid=fopen("Temps1","rt")
fid = 7
>> fgets(fid,8)
ans = Nuevo te

--> fid=mopen("Temps1","rt")
fid =

    2.

--> mgetl(fid,2)
ans =

    "Nuevo TextoNuevo Texto 2"
    "Nuevo Texto 3"
```

En este punto corresponde hacer unas aclaraciones:

- Cada una de las dos funciones usa el identificador “fid” como input.
- El segundo input tiene un significado distinto para “fgets” y “mgetl”. Para el primero es la cantidad de caracteres, **mientras que para el segundo es la cantidad de líneas que debe mostrar.**

Esto último se presenta en el ejemplo anterior. Mientras que fgets, con un input de 8 presentó el texto “Nuevo te” (8 caracteres), **“mgetl”, con un input de 2, presentó dos líneas.** Notese que, si se los vuelve a llamar a estos comandos, los mismos presentarán los caracteres o líneas que le sigan a los que ya mostró. **Por ejemplo, si en el primer llamado “mgetl” recibió como input un 1, mostrará “Nuevo Texto1”.** Si es vuelto a llamar con un input de 2 esta vez mostrará “Nuevo texto 2” y, debajo de esto, “Nuevo texto3”. Esto se observa a continuación.

```
>> fid=fopen("Temps1","rt")
fid = 9
>> fgets(fid,3)
ans = Nue
>> fgets(fid,5)
ans = vo te
>> fgets(fid,3)
ans = xto
```

```
--> fid=mopen("Temps1","rt")
fid =
```

```
3.
```

```
--> mgetl(fid,1)
ans =
```

```
"Nuevo TextoNuevo Texto 2"
```

```
--> mgetl(fid,2)
ans =
```

```
"Nuevo Texto 3"
""
```

```
--> mgetl(fid,2)
ans =
```

```
""
"Es un curso de 10 Alumnos"
```

Capítulo 14

Resolución de sistemas de ecuaciones diferenciales, algebraicas y cálculo diferencial e integral



OCTAVE



En esta sección se presenta como resolver integrales definidas e indefinidas. En Octave se deben usar las variables simbólicas, por lo que se empezará explicando un poco en qué consisten estas. Las

variables simbólicas son variables que representan una incógnita, por lo que no tienen un valor numérico fijo. Este tipo de variables se suelen usar para obtener expresiones, las cuales permiten obtener varios resultados sin tener que sobrescribir ninguna variable. A cambio de eso, el usuario debe indicarla a la variable simbólica que valor debe tomar en cada llamado. Cabe notar que solo Octave tiene variables simbólicas, por lo que todo lo aquí presentado respecto de ellas no se aplica en Scilab.

14.1. Las funciones simbólicas y la función “subs”

En Octave, primero debe cargarse el paquete. El mismo ya se encuentra instalado en la versión 7.1.0. El usuario debe introducir “pkg load symbolic” en la ventana de comandos. Este paso no es necesario en Matlab, si lo tuviesen.

```
>> pkg load symbolic
```

Luego de esto, el usuario ya puede resolver integrales en Octave. Primero se debe definir las variables simbólicas y una función, que es la que se va a integrar.

```
>> syms x z; r=x^3+x*z-z^2
r = (sym)
```

$$x^3 + xz - z^2$$

Con esto ya se definió las variables simbólicas “x” y “z” y una expresión algebraica. Esta última, que representa una función, puede realizar cualquier operación algebraica. Por ejemplo, pueden sumar o multiplicar. Esto se muestra a continuación:

```
>> s=2*x+z
s = (sym) 2*x + z
>> r+s
ans = (sym)
```

$$x^3 + xz + 2x - z^2 + z$$

```
>> r*s
ans = (sym)
```

$$(2x + z) \cdot (x^3 + xz - z^2)$$

Además, las funciones con variables simbólicas se pueden evaluar usando la función “subs”.

```
>> sol=subs(r,z,2)
sol = (sym)
```

$$x^3 + 2x - 4$$

De este modo, se evaluó la función “r” usando “z=2”. Sin embargo, “x” sigue teniendo un valor simbólico. Procediendo de un modo similar se puede evaluar esta nueva expresión con “x=3”.

```
>> sol2=subs(sol,x,3)
sol2 = (sym) 29
```

Con estos dos valores se observa que se tiene un resultado numérico de “r”.

14.2. Resolución de integrales

En esta sección se estudia como resolver integrales definidas e indefinidas. Hay que tener en cuenta que en Octave se pueden resolver los dos tipos de integrales, **mientras que en Scilab solo el primero**. En Octave para eso se deben usar las variables simbólicas y la función “int” **mientras que en Scilab se va a usar la función “intg”**.

14.2.1 Resolución de integrales indefinidas en Octave

Para calcular una integral indefinida en Octave se debe primero definir la variable simbólica y la función matemática que se va a integrar, y luego aplicar el mencionado comando.

```
>> syms x;  
>> f=x^3  
f = (sym)
```

```
      3  
      x  
>> int(f,x)  
ans = (sym)
```

```
      4  
      x  
      --  
      4
```

De un modo similar, se puede calcular integrales dobles, esta vez definiendo dos variables simbólicas en lugar de una. Si el usuario desea integrar primero respecto a “x” y luego respecto de “y”, debe proceder como sigue a continuación. Este procedimiento es muy intuitivo y permite calcular cambiar el orden de integración e incluso integrar funciones de más de dos variables.

```
>> syms x y, f=x*y  
f = (sym) x*y  
>> f1=int(f,x)  
f1 = (sym)
```

```
      2  
      x *y  
      ----  
      2
```

```
>> f2=int(f1,y)  
f2 = (sym)
```

```
      2  2  
      x *y  
      ----  
      4
```



Notar que “f1”, input de la última sentencia, es el output de la anteúltima sentencia.

14.2.1 Resolución de integrales definidas

Para calcular una integral definida en Octave se debe, como antes, definir las variables simbólicas de las mismas y la función que se va a integrar. Una vez hecho esto se debe usar la función “int”, cuyos inputs serán la función matemática, la variable de integración y sus límites de integración, en ese orden.

```
>> syms x y, fun=x+y
fun = (sym) x + y
>> q1=int(fun,x,0,1)
q1 = (sym) y + 1/2
>> q2=int(q1,y,2,3)
q2 = (sym) 3
```



Este comando representa la siguiente integral: $\int_2^3 \int_0^1 (x + y) dx dy$.

En este caso se definió las variables simbólicas “x” e “y” para luego definir la función “fun”. Entonces se integro respecto de “x” entre 0 y 1 y finalmente “y” entre 2 y 3.

Octave también permite calcular integrales con constantes en el integrando. En este ejemplo se definió a “a” como variable simbólica pero no se la integró. Así, queda como si fuese una constante.

```
>> syms x y a,
>> f=a*x*y
f = (sym) a*x*y
>> int(f,x,y)
ans = (sym)
```

$$- \frac{a^2 x^2 y}{2} + \frac{a^2 y^3}{2}$$

También, si se tiene que resolver la misma integral varias veces para distintos límites, se puede definir a los mismos con variables simbólicas y luego reemplazarlos por distintos valores. En el ejemplo que sigue a continuación se definen dichos límites con las variables simbólicas “a” y “b”.

```
>> syms x a b, F=x^2
F = (sym)
```

$$x^2$$

```
>> Int1=int(F,x,a,b)
Int1 = (sym)
```

$$- \frac{a^3}{3} + \frac{b^3}{3}$$

Luego puedo sustituir “a” y “b” por valores numéricos.

```
>> Res1=subs(Int1,a,2)
Res1 = (sym)
```

$$\frac{b^3}{3} - \frac{8}{3}$$

```
>> Res2=subs(Res1,b,1)
Res2 = (sym) -7/3
```

En Scilab se pueden resolver integrales simples de una variable usando el comando `intg`. Se empieza definiendo la función que se busca integrar dentro de un script.

```
function y=fun(x)
    y=2*x+1
endfunction
```

Luego de escribir el script debe guardarse con el mismo nombre de la función (en este caso “fun”). Con eso la función ya se encuentra creada. Luego se apreta el botón de “play” en el menú de opciones del script. Finalmente se debe usar la función “`intg`” para integrar.

```
--> intg(0,2,fun,0.0001)
ans =

    6.
```

Los dos primeros inputs son los límites de integración, “fun” es el nombre de la función (observar que este es el nombre que tiene en el script) y 10^{-3} es el error o tolerancia.

También se puede definir la función como se hace en Python. Para esto se usa el comando “`deff`”. Este requiere de dos inputs, ambos strings. En el primero se introducen las variables dependientes e independientes. En el segundo se introduce la función propiamente dicha. Finalmente se aplica la función “`intg`” exactamente como se realizó arriba. A continuación se muestra como usar el comando “`deff`”:

```
--> deff("z=fun(x)","z=2*x+1")
```

De este modo se informa a Scilab que se creó la función “fun”, la cual tiene como variable dependiente a “z” e independiente a “x”. La función es “`z=2*x+1`”. Una vez creada se introduce la función “`intg`”.

```
--> intg(0,2,fun,0.0001)
ans =

    6.
```

Además, Scilab puede calcular integrales dobles (también integrales triples, pero su cálculo es más complejo y por eso no es incluyó en esta versión del trabajo). El comando a utilizar es “`int2d`”. Primero hay que definir la función. Para eso se usa, nuevamente, el comando “`deff`” y luego se aplica la función “`int2d`”. “I” es el resultado y “err” es el error del método. Además, [0,1] son los límites de integración en “x” y [-1,2] los límites de integración en “y”. Debe notarse que estos resultados son numéricos y no se obtuvieron usando variables simbólicas.

```
--> deff("z=f(x,y)","z=cos(x+y)")

--> [I,error]=int2d(0,1,-1,2,f)
I =

    1.0335434
error =

    6.348D-11
```

14.3 Cálculo de derivadas

A continuación se estudiará cómo resolver derivadas. Estas también en Octave se calculan usando variables simbólicas. Deben estar definidas dentro de una función a la que se busca calcular la derivada y luego aplicar el comando “diff”. Más abajo se observa como se definió una variable simbólica “x” y la función f, para luego calcular la derivada de esta en x=1 (este último aparece como input en “diff”).

```
>> syms x, f=x^3
f = (sym)

    3
    x

>> diff(f,1)
ans = (sym)

    2
    3*x
```

Se observa que se calculó la derivada primera. Luego, se procede a calcular también la segunda.

```
>> diff(f,2)
ans = (sym) 6*x
```

Como se ve, Octave permite derivar una función matemática una y otra vez. También se puede derivar funciones de dos o más variables en el orden que se desee.

```
>> syms x y
>> f=x^2*y+y^2*x
f = (sym)

    2      2
    x *y + x*y

>> dfy=diff(f,y)
dfy = (sym)

    2
    x  + 2*x*y

>> dfyx=diff(dfy,x)
dfyx = (sym) 2*x + 2*y
>> dfx=diff(f,x)
dfx = (sym)
```

$$2x^2y + y^2$$

```
>> dfxy=diff(dfx,y)
dfxy = (sym) 2*x + 2*y
```

Sin embargo, debe notarse que entre la primera y la segunda llamada “diff” se debe guardar la respuesta. Si el usuario no desea hacerlo se puede usar una sola línea, la cual puede ser engorrosa.

```
>> diff(diff(f,x),y)
ans = (sym) 2*x + 2*y
```

El comando “diff”, además, se puede aplicarse a vectores. Son para calcular las “diferencias de orden N”. Hay una fórmula para cada orden, por lo que a continuación solo se presentan los tres primeros. Un vector v de dimensión N con el coeficiente v_i en la coordenada i -ésima, al aplicársele el comando “diff” dará como resultado otro vector donde en cada coordenada tendrá los siguientes resultados:

$$\text{Orden 1: } v_{i+1} - v_i$$

$$\text{Orden 2: } v_{i+1} - 2.v_i + v_{i-1}$$

$$\text{Orden 3: } v_{i+1} - 3.v_i + 3.v_{i-1} - v_{i-2}$$

Estas fórmulas se prueban en el siguiente ejemplo:

```
>> X=[1 2 3 4 5.6 6.7 1.2 8.9 4.7];
>> diff(X,1)
ans =

    1.0000    1.0000    1.0000    1.6000    1.1000   -5.5000    7.7000   -
    4.2000

>> diff(X,2)
ans =

         0         0    0.6000   -0.5000   -6.6000   13.2000  -11.9000

>> diff(X,3)
ans =

         0    0.6000   -1.1000   -6.1000   19.8000  -25.1000
```

En Scilab la derivada se calcula numéricamente. Se debe definir una función en un script aparte y luego calcular su derivada usando el comando “numderivative”.

```
function y=f(x)
    y=x(1)^3+x(2)^3+x(3)^3
endfunction
```

Con esto se creó la función f , la cual debe estar definida en su propio script, como ocurre con cualquier función (ver capítulo 8 de funciones creadas por el usuario). En este caso se tienen una variable independiente “x” y tiene tres coordenadas. Por eso, matemáticamente, la función es de tres variables, a pesar de que en Scilab se vea como si fuese una sola, llamada “x”. Además, “y”, la salida de la función,

tiene una sola coordenada. Esto quiere decir que la función, matemáticamente, va a R^3 a R y tiene tres variables independientes y una dependiente.

Para calcular la derivada se debe elegir el punto en que se la va a calcular y usar la función “numderivative”. A continuación se obtiene el resultado.

```
--> x=[1 2 3];
--> numderivative(f,x)
ans =

    3.    12.    27.
```



Se obtiene un vector de tres coordenadas ya que es una función de tres variables. El primer valor es la derivada de la función respecto a la primera variable “x(1)” en el punto x. Los otros dos valores del resultado se obtienen en forma análoga.

También se puede calcular la derivada de una función más compleja, donde el espacio de llegada es de más de una dimensión. En el caso que sigue la función va de R^3 a R^2 . Es decir que la función tiene tres variables independientes y dos dependientes.

```
function y=f(x)
    f1=sin(x(1)*x(2))+exp(x(2)*x(3))+x(1)
    f2=x(1)^3+x(2)^3+x(3)^3
    y=[f1;f2]
endfunction
```

La entrada de la función “f” tiene tres variables (“x(1)”, “x(2)” y “x(3)”) y la salida dos (“f1” y “f2”). A continuación se calcula la derivada “J” en el mismo punto que antes y el hessiano “H”.

```
--> J=numderivative(f,x)
J =

    1095.8009    3289.4833    2193.2663
    3.         12.         27.

--> [J,H]=numderivative(f,x)
J =

    1095.8009    3289.4833    2193.2663
    3.         12.         27.
H =

    1092.996    3287.665    2193.2665    3287.665    9868.7909    7676.4339
    2193.2665    7676.4339    4386.5334
    6.         0.         0.         0.         12.         0.
    0.         0.         18.
```



La primera fila del jacobiano representa las derivadas de la función “f1” (la que fue introducida primera en el script), la segunda fila son las derivadas de la segunda función.

En el hessiano cada fila representa las derivadas de “f1” y “f2”, respectivamente.

Además, en una fila, el primer coeficiente representa la derivada segunda respecto a “x(1)”, el segundo coeficiente representa la derivada segunda respecto a “x(1)” y “x(2)”, el tercer coeficiente representa la derivada segunda respecto a “x(1)” y “x(3)”, el cuarto coeficiente representa la derivada segunda respecto a “x(2)” y “x(1)” (como las derivadas cruzadas son iguales este coeficiente es igual al segundo), etc.

14.4 Resolución de ecuaciones diferenciales

Usando variables simbólicas, con Octave, también se pueden resolver ecuaciones diferenciales. En esta sección se verán que pasos hay que seguir para resolver una ecuación diferencial. Una ecuación diferencial es una ecuación en la que el resultado es una función y no un número. En un problema típico de ecuaciones diferenciales se tienen los siguientes datos:

- La ecuación diferencial, propiamente dicha
- El dominio en que se resuelve la ecuación (suele ser el tiempo o un espacio dado)
- Condición inicial, la cual puede ir acompañada de una condición de contorno

La condición inicial es el valor que tiene la solución de la ecuación en el instante inicial. Por ejemplo, si se busca calcular la posición de un móvil usando una ecuación diferencial, la condición inicial suele ser donde se encuentra dicho vehículo en el instante inicial del movimiento. En cambio, la condición de borde suele ser alguna restricción sobre la posición o velocidad de este. Por ejemplo, que la velocidad alcanza un máximo al final del recorrido.

Esto solo es un pequeño resumen sobre los componentes o entradas de un problema de ecuaciones diferenciales. Este libro no busca realizar un estudio detallado de este tipo de ecuaciones sino presentar una breve explicación sobre cómo resolver un problema de este tipo usando Octave o Scilab. Por eso, si existe un genuino interés en analizar estos problemas en profundidad se puede recurrir a internet, que tiene mucha información sobre esto.

En un problema de ecuaciones diferenciales en Octave (o Matlab) primero se debe definir las variables dependientes e independientes y las constantes de la ecuación diferencial.

```
>> syms y(t) a
```

Acá se está definiendo a “t” como variable independiente (por eso va entre parentesis), “y” es la variable dependiente (aquella que será la incógnita de la ecuación diferencial) y “a” es una constante genérica (después puede reemplazarse por el valor que se desee). Esto quiere decir que se va a resolver una ecuación diferencial, cuya solución será la función “y”, que tendrá como variable a “t” y “a” será tomada como una constante (después el usuario puede darle el valor que desee).



Debe tenerse en cuenta que, si el usuario desea, se puede omitir usar la constante “a” en este paso y en los subsiguientes, hasta obtener el resultado.

El segundo paso consiste en introducir la ecuación diferencial. Con el comando que sigue abajo se está creando una ecuación diferencial y es la que sale a continuación.

```
>> eqn = diff(y,t) == a*y  
eqn = (sym)
```


$$\frac{d}{dt}(y(t)) = a*y(t)$$

Finalmente, se la resuelve usando el comando `dsolve`. Abajo se presenta la solución. Debe notarse que esta depende de “a”, la cual fue introducida como una variable genérica. Actúa a modo de una constante sin un valor asignado ya que fue definida como variable simbólica. Además, apareció otra contante, la cual es desconocida, llamada “C1”, que está ahí porque no se introdujeron condiciones iniciales.

```
>> ySol(t)=dsolve(eqn)
ySol(t) = (symfun)
```

$$y(t) = C1*e^{a*t}$$

Ahora se mostrará como resolver el problema con una condición inicial. Se utilizarán los mismos comandos que en el caso anterior pero se le agregará una condición inicial, la cual consiste en tomar $y=5$ para $t=0$.

```
>> syms y(t) a
>> eqn = diff(y,t) == a*y
eqn = (sym)
```

$$\frac{d}{dt}(y(t)) = a*y(t)$$

```
>> ySol(t) = dsolve(eqn,y(0) == 5)
ySol(t) = (symfun)
```

$$y(t) = 5*e^{a*t}$$



Estos comandos son iguales a los anteriores salvo que se agregó un input a “dsolve”, que representa la condición inicial.

También se puede resolver ecuaciones que tengan una derivada de orden mayor a uno. Esto se logra introduciendo un tercer input al comando “diff”. En el ejemplo que sigue se introdujo un 2 como input, dando como resultado una derivada de orden dos. Esto implica que la derivada de la ecuación diferencial es de segundo grado.

```
>> syms y(t) a
>> eqn = diff(y,t,2) == a*y
eqn = (sym)
```

$$\frac{d^2}{dt^2}(y(t)) = a*y(t)$$

```
>> ySol(t) = dsolve(eqn)
ySol(t) = (symfun)
```

$$y(t) = C1 * e^{-\sqrt{a} * t} + C2 * e^{\sqrt{a} * t}$$

También se pueden introducir condiciones de contorno en lugar de condiciones iniciales, y es a partir de aquí que empiezan a verse diferencias en como Matlab y Octave resuelven ecuaciones diferenciales. Matlab requiere que primero se definan las condiciones de contorno y después usar el comando “dsolve”. En cambio, Octave requiere que en este se introduzcan las condiciones de contorno. A continuación se resolverá un mismo problema para los dos programas. El problema a resolver será el siguiente:

$$\frac{d^2 y}{dt^2} = a \cdot y$$

$$CI: \quad y(0) = 1$$

$$CC: \quad \left. \frac{dy}{dt} \right|_{t=0} = 1$$

Los comandos que se deben introducir en Matlab son los siguientes. El primer consiste en la definición de las variables involucradas en el problema. El segundo es la definición de la ecuación diferencial. El tercero es una definición que en el paso siguiente será usado para la condición de contorno. El cuarto es la definición de las condiciones inicial y de contorno y el último es donde se pide que se resuelva el sistema usando la función “dsolve” con la ecuación diferencial y las condiciones inicial y de contorno como inputs.

```
>> syms y(t) a
>> eqn = diff(y,t,2) == a*y
>> Dy = diff(y,t)
>> cond = [y(0)==1 , Dy(0) == 1]
>> ySol(t) = dsolve(eqn,cond)
```

En cuanto a Octave, los pasos son similares. De hecho, los dos primeros son iguales. El que cambia es el tercero (y último) ya que en Octave se deben introducir las condiciones inicial y de contorno en el comando “dsolve” sin tener que definir las previamente. Obsérvese que el segundo y tercer input corresponden a dichas condiciones.

```
>> syms y(t) a b
>> eqn = diff(y,t,2) == a*y
eqn = (sym)
```

$$\frac{d^2}{dt^2}(y(t)) = a \cdot y(t)$$

```
>> ySol(t) = dsolve(eqn,y(0) == b, diff(y)(0) == 1)
ySol(t) = (symfun)
```

$$y(t) = \frac{1}{2} \left(\frac{b}{\sqrt{a}} - \frac{1}{2\sqrt{a}} \right) e^{-\sqrt{a} * t} + \frac{1}{2} \left(\frac{b}{\sqrt{a}} + \frac{1}{2\sqrt{a}} \right) e^{\sqrt{a} * t}$$

Octave también puede calcular sistemas de ecuaciones diferenciales. La forma de resolverlos es la misma en ambos programas, pero hay una pequeña diferencia que consiste en cómo se almacenan las soluciones. Para ver esta diferencia se resolverá el siguiente sistema de ecuaciones diferenciales.

$$\frac{dy}{dt} = z$$

$$\frac{dz}{dt} = -y$$

Se empieza definiendo las variables independientes y dependientes. La variable independiente es “t” y las variables independientes son “y” y “z”. Después se introducen las ecuaciones diferenciales, y finalmente el comando “dsolve”. Esto es lo que se ve a continuación:

```
>> syms y(t) z(t)
>> eqns = [diff(y,t) == y; diff(z,t) == -z]
eqns = (sym 2x1 matrix)
```

```
[d
 [--(y(t)) = y(t) ]
 [dt
 [
 [d
 [--(z(t)) = -z(t) ]
 [dt
```

```
>> Sol = dsolve(eqns)
Sol =
{
 [1,1] =

 <class sym>

 [1,2] =

 <class sym>
}
```

Ahora tenemos la solución del problema guardado en la variable Sol. Pero acá hay una diferencia entre Matlab y Octave. Octave toma a Sol como una variable cell, por lo que para obtener la solución tenemos que escribir Sol{1} y Sol{2}. En cambio, Matlab toma a Sol como una struct, por lo que la solución se obtiene escribiendo Sol.y y Sol.z.

```
>> Sol{1}
ans = (sym)
```

```

      t
y(t) = 2*C1*e
```

```
>> Sol{2}
ans = (sym)
```

$$z(t) = -2 \cdot C_2 \cdot e^{-t}$$

14.5 Resolución numérica de ecuaciones diferenciales

En esta sección se estudiará como resolver numéricamente una ecuación diferencial usando Octave y Scilab. Sin embargo, antes conviene recordar en qué consiste la resolución numérica de ecuaciones diferenciales. Además, hay que tener en cuenta que las funciones de Octave y Scilab que van a presentarse en este capítulo usan el método de diferencias finitas.

Este método consiste en convertir la ecuación diferencial en una ecuación en diferencias, la cual es una ecuación algebraica. Esto implica que también se va a discretizar el dominio de dicha ecuación. Es decir, hay determinados puntos del dominio (que los elige el usuario) donde la ecuación en diferencias nos permite calcular los valores de la solución de la ecuación diferencial.

Debe tenerse en cuenta que este proceso tiene un error asociado, por lo que los valores calculados no son exactamente iguales a los calculados con la solución exacta. Otro tema a tener en cuenta en este proceso es que al discretizarse el dominio en puntos, cada par sucesivo de ellos está separado por un “paso”, el cual suele ser constante.

Además, se necesita una condición inicial, y cada problema puede además tener condiciones de contorno. La primera consiste en cuanto vale la función en el momento en que la variable independiente vale cero. Un ejemplo clásico de condición inicial es informar el valor de la solución al principio del dominio. En particular, si se está analizando el movimiento de una bicicleta, la condición inicial suele ser la posición o la velocidad en el instante inicial, donde $T(\text{tiempo})=0$.

En resumen, los datos de entrada de un problema de ecuaciones diferenciales son:

- Ecuación diferencial
- Dominio
- Discretización del dominio
- Condición inicial

Esto mismo es aplicable a sistemas de ecuaciones, donde se debe tener igual número de condiciones iniciales y ecuaciones. El dominio sigue siendo el mismo y, por consiguiente, la variable independiente también. Además, el problema requiere que se defina el paso de tiempo (o, lo que es lo mismo, los puntos del dominio donde se calcularán los valores de la solución). Es decir que el problema es análogo, solo que si hay dos ecuaciones, debe informarse al software, y también sus condiciones iniciales.

Finalmente, Octave permite resolver ecuaciones diferenciales de orden mayor a uno, por lo que puede haber problemas en los que haya una condición inicial y una condición de contorno (por ejemplo, en una placa que se está calentando, la condición inicial puede ser la temperatura inicial y la condición de contorno puede ser la temperatura en un extremo o la derivada de esta).

A continuación se estudiará como resolver una ecuación diferencial usando Octave. Primero se debe armar la ecuación en un script aparte. Esto se presenta en el script “diferencial”, donde se encuentra lo siguiente:

```
function dzdt=diferencial1(t,z)
    dzdt=-0.5*z
endfunction
```

Así, la ecuación que se va a resolver es la siguiente:

$$\frac{dz}{dt} = -0.5 * z$$

Notar que la ecuación diferencial se encuentra definida dentro de un script. Es decir, como una función de Octave. Esta definida como una función, porque pone “function”. Luego, tiene una variable de salida que es “dzdt”. Después está el nombre de la función, que es “diferencial1” y finalmente aparecen las variables: La independiente “t” primero y la dependiente “z” después. Pero cabe recordar que se puede usar más de una variable dependiente (y estos softwares solo permiten una independiente).

Definida la función entonces se puede usar el comando que va a resolverla: “ode45”. Para esto debe dársele play al script “diferencial1”. Luego se introduce dicho comando, como se presenta a continuación.

```
>> [t,sol]=ode45(@diferencial1,[0 20],1);
```

Con el primer input se informa dónde está la ecuación (no hay que olvidar agregar el @), en el segundo input se introduce un vector con el valor mínimo y máximo del dominio, en el último input va la condición inicial. El primer output es un vector que contiene los puntos del dominio en que se evaluó la función. Estos suelen estar más o menos espaciados dependiendo de que tanto varia la solución. Si la misma varia rápidamente (lo que equivale a una derivada de alto valor absoluto) los puntos estarán uno muy cerca de otros. La situación contraria se da si la variación de la solución es lenta.

Debe notarse que “ode45” es un método numérico y define por si mismo el paso. Esto se puede ver en el vector “t”, que tiene un paso pequeño al principio y luego es mucho mayor. Además, se puede graficar el resultado usando las herramientas básicas de graficación “plot(t,sol)”.

```
>> t'
ans =
```

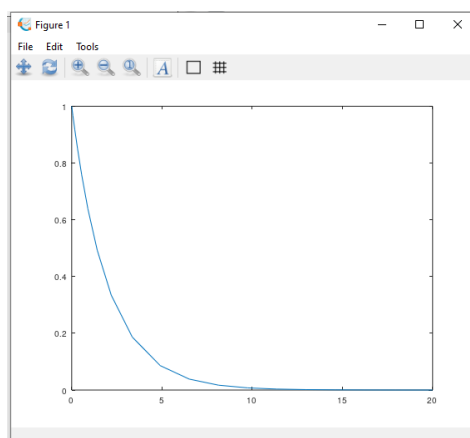
Columns 1 through 12:

```
0      0.0681    0.1703    0.3236    0.5535    0.8985    1.4158
2.1918    3.3559    4.9082    6.5178    8.1343
```

Columns 13 through 20:

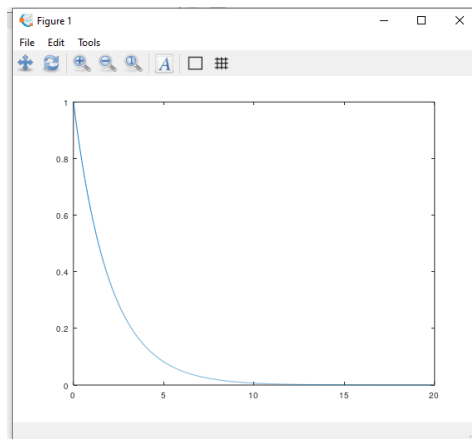
```
9.7516   11.3691   12.9865   14.6040   16.2214   17.9482   19.9393
20.0000
```

```
>> plot(t,sol)
```



A pesar de que el software elige el paso, el usuario puede determinarlo por su cuenta, si lo que desea es analizar con más detalle la solución. Esto se hace introduciendo los puntos del dominio que el usuario desea. Para eso se usará un vector “x”, el cual tendrá más puntos que “t”. El mismo se presenta a continuación, cuando a “x” como input, en lugar de informar los puntos inicial y final del intervalo. Se obtiene, ahora, una solución más definida.

```
>> x=0:0.01:20;
>> [x,sol]=ode45(@diferencial1,x,1);
>> plot(x,sol)
```



También, Octave puede resolver numéricamente sistemas de ecuaciones diferenciales, como el presentado en la sección anterior. Esto requiere que dicho sistema se defina dentro de un script. A continuación se presenta como.

```
function dzdt=diferencial2(t,z)
    dzdt = [z(2); -z(1)]
end
```

Así, el sistema que se va a resolver es el siguiente:

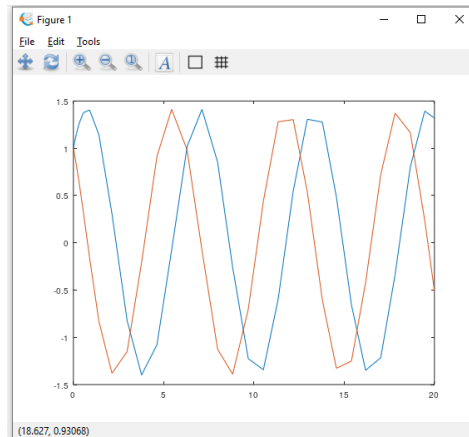
$$\frac{dy}{dt} = z$$

$$\frac{dz}{dt} = -y$$

Está definido como una función, porque pone “function”. Luego, tiene una variable de salida que es “dydt”, que es una matriz de dos filas, cada una de ellas es la evolución de una de las dos variables z1 y z2. Después está el nombre de la función, que es “diferencial2” y luego están las variables: La independiente “t” primero y la dependiente “z” después, que tiene dos coordenadas, “z1” y “z2”.

Definida la función entonces se puede usar el comando que va a resolverla: “ode45”. Para esto debe dársele play al script “diferencial2”. Luego se introduce dicho comando, como se presenta a continuación.

```
>> [t,sol]=ode45(@diferencial2,[0 20],[1 1]);
>> plot(t,sol)
```



Con el primer input se debe informar donde está la ecuación (no olvidar el @), en el segundo input va un vector con el valor mínimo y máximo del dominio y en el último se coloca la condición inicial, la cual en este caso es un vector de dos coordenadas, ya que hay dos ecuaciones.

Debe notarse que ode45 es un método numérico y define por si mismo el paso, como ocurrió en el ejemplo anterior.

```
>> t'
ans =
```

Columns 1 through 12:

```

0      0.0681    0.1703    0.3236    0.5535    0.8985    1.4158
2.1389    2.9770    3.7779    4.6299    5.4422
```

Columns 13 through 24:

```

6.3014    7.1236    7.9908    8.8229    9.6870   10.5261   11.3419
12.1776   12.9592   13.7924   14.5691   15.4044
```

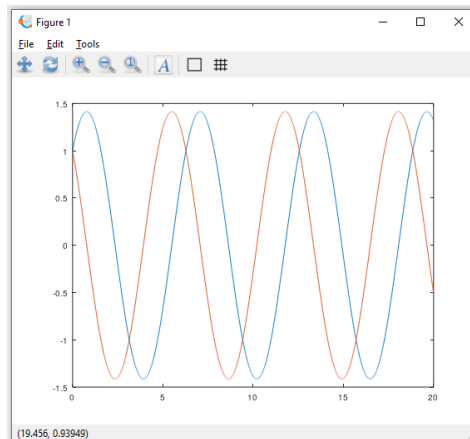
Columns 25 through 30:

```

16.1879   17.0274   17.8191   18.6638   19.4645   20.0000
```

A pesar de que el software elige el paso, el usuario, una vez más, puede determinarlo por su cuenta, si lo que desea es analizar con más detalle la solución. Esto se hace introduciendo en el segundo input los puntos del dominio que el usuario desea en lugar de los extremos del dominio. Por eso se procede como en el ejemplo anterior, usando el vector “x”.

```
>> x=0:0.01:20;
>> [x,sol]=ode45(@diferencial2,x,[1 1]);
>> plot(x,sol)
```



Una vez más, se observa que el resultado es más definido.

Algo similar se puede hacer en Scilab usando el comando ode. Este requiere que primero se defina el sistema de ecuaciones. Para eso se usa el comando deff.

```
--> deff("[w]=diferencial(t,y)", "w=y+t")
```

A continuación, se verá uno por uno el significado de cada input. En el primero está la variable “w”, la cual representa la derivada (en este caso sería dy/dt). También se encuentra el nombre del sistema de ecuaciones (“diferencial”), la variable independiente, que es “t”, y la variable dependiente, que es “y”, la cual sería el nombre de la función solución que se busca calcular. En el segundo input está la ecuación diferencial que se busca resolver numéricamente. Como ya fue explicado, “w” representa la derivada, ubicada en el lado izquierdo, mientras que “y+t” sería el lado derecho de la ecuación diferencial. Finalmente, en el último input se introduce simplemente “w”.

Así, la ecuación que se va a resolver es la siguiente:

$$\frac{dy}{dt} = y + t$$

Luego debe utilizarse la función “ode”. Esta cuenta con los siguientes inputs: El primero es la condición inicial de la función solución “y”, el segundo es el valor del dominio (“t” es este ejemplo) en el cual está la condición inicial (notar que para este ejemplo, como el primer input es 1 y el segundo es 0 entonces la condición inicial es $y_{(0)} = 1$), el tercero input es un vector que contiene todos los puntos del dominio en los cuales se calculará “y”, y finalmente, el cuarto input es el nombre de la ecuación o sistema de ecuaciones que se resolverá.

```
--> t=0:0.01:1;
```

```
--> y=ode(1,0,t,diferencial);
```

De un modo análogo se puede resolver numéricamente un sistema de ecuaciones diferenciales. El sistema de ecuaciones se define usando, una vez más, el comando “deff”.

```
--> deff("[w]=diferencial2(t,y)", ["w(1)=y(2)+t", "w(2)=-y(1)", "w"]);
```

A continuación, se verá uno por uno el significado de cada input. En el primero está la variable “w”, la cual representa la derivada de cada una de las dos ecuaciones (en este caso serían dy/dt para la primera ecuación y dz/dt para la segunda). También se encuentra el nombre del sistema de ecuaciones (“diferencial2”), la variable independiente, que es “t”, y las variables dependientes, que son “y” y “z”, las cuales serían los nombres de las funciones solución que se busca calcular. En los segundo y tercer

inputs está la ecuación diferencial que se busca resolver numéricamente. Como ya fue explicado, “w(1)” representa la derivada dy/dt, ubicada en el lado izquierdo de la primera ecuación, mientras que “y(2)+t” sería el lado derecho de dicha ecuación diferencial. Además, “w(2)” representa a dz/dt y “y(2)” representa a z. Finalmente, en el último input se introduce simplemente “w”.

Así, el sistema de ecuaciones que se va a resolver es el siguiente:

$$\frac{dy}{dt} = z + t$$

$$\frac{dz}{dt} = y$$

En resumen, “w(1)” es dy/dt, “y(2)” es z, “w(2)” es dz/dt y “y(1)” es y. Debe notarse que tanto “y” como “w” son vectores de dos filas.

Finalmente se aplica la función de Scilab “ode” para resolver el problema. Su aplicación es idéntica a la del caso anterior, con la salvedad de que en el primer argumento la condición inicial tiene que ser un vector vertical, donde en la primera coordenada esté la condición inicial de la función “y” mientras que en la segunda la de la función “z”.

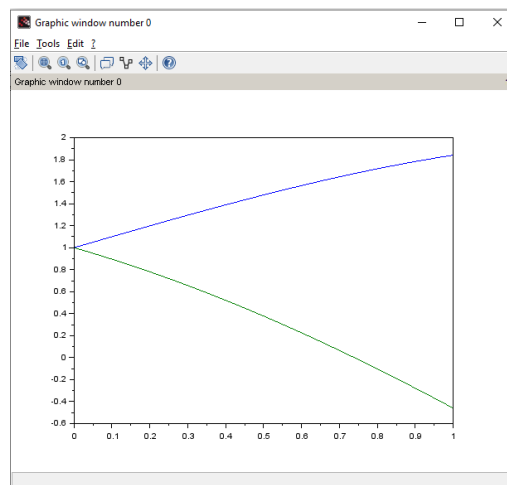
```
--> y=ode([1;1],0,t,diferencial2);
```

Cabe notar que el output de “ode” es la variable “y”. Esta es una matriz cuya primera fila son los resultados de la función solución y mientras que la segunda fila son los resultados de la función solución z. Finalmente, se pueden observar los resultados:

```
--> plot(t,y(1,:))
```

```
--> mtlb_hold on
```

```
--> plot(t,y(2,:))
```



14.6 Resolución de sistemas de ecuaciones lineales

En esta sección se estudia la resolución de sistemas de ecuaciones lineales. Es decir, matrices. Esto fue estudiado en una sección anterior, pero no se hizo foco especialmente en esto. El sistema lineal que se resolverá es $A.x=b$, que se hacía usando “A\B”. Sin embargo, en este caso se va a usar un comando alternativo con el cual se obtendrá el mismo resultado, “mldivide”. Primero se debe definir la matriz y el vector.

```
>> A=[1 0 2;1 1 0;0 0 3]
A =
```

```
1    0    2
1    1    0
0    0    3
```

```
>> b=[1;2;3]
b =
```

```
1
2
3
```

```
>> x=mldivide(A,b)
x =
```

```
-1
3
1
```

Observe que se cumple la igualdad $A \cdot x = b$

```
>> A*x-b
ans =
```

```
0
0
0
```

En Scilab se usa el comando “linsolve”, pero hay que tener en cuenta que este resuelve el sistema $A \cdot x + b = 0$. Por eso, al usar “linsolve” se debe incluir un menos al segundo argumento, de modo de resolver el sistema $A \cdot x = b$.

```
--> x=linsolve(A,-b)
x =
```

```
-1.0000000
3.0000000
1.0000000
```

```
--> A*x-b
ans =
```

```
-2.220D-16
-1.110D-15
8.882D-16
```

14.7 Resolución de sistemas de ecuaciones no lineales

Para resolver ecuaciones (o sistemas de ecuaciones) no lineales se debe definir un sistema de ecuaciones por medio de una función y luego aplicar el comando “fsolve” para encontrar sus raíces. Dado que su aplicación es muy sencilla, se presentará como usar este comando con un sistema de ecuaciones, pero el mismo puede utilizarse para ecuaciones.

En Octave primero debe crearse un script, en el cual se definirá el sistema de ecuaciones como una función de Octave. El script, guardado como “root2d” (ya que tiene que tener el mismo nombre que la función) se presenta a continuación:

```
function F=root2d(x)
    F(1)=exp(-exp(-x(1)+x(2)))-x(2)*(1+x(1)^2);
    F(2)=x(1)*cos(x(2))+x(2)*sin(x(1))-0.5;
end
```

Como se puede apreciar hay un output que es “F”, que es un vector de dimensión 2 y hay un input que es el valor de las variables independientes del sistema, “x(1)” y “x(2)”. Es decir que “x” es también un vector de dimensión 2. Se empieza dándole play al script.

Ahora, si el usuario desea se puede evaluar el sistema escribiendo “root2d([1 1])”. Se obtienen los valores de F para un x=[1 1].

```
>> root2d([1 1])
ans =

    -1.6321    0.8818
```

Si el usuario desea calcular las raíces primero debe identificar a “root2d” como un sistema de ecuaciones. Esto se realiza del siguiente modo:

```
>> fun=@root2d
fun = @root2d
```

Ahora, Octave considera a “root2d” como un sistema de ecuaciones. Luego, se puede calcular las raíces. Pero antes, se debe tener en cuenta que para calcularlas se usa un método numérico, y estos necesitan una semilla. La misma se crea a continuación como un vector de dos dimensiones, ya que “x” tiene dos coordenadas.

```
>> x0=[0 0]
x0 =

    0    0
```

Finalmente, para hallar las raíces se usa la función “fsolve” como se observa a continuación. En el primer input de este comando se introduce el sistema de ecuaciones (notar que a “root2d” lo introducimos en la variable “fun” un poco más arriba) y en el segundo el valor semilla “x0”. Además, se tiene un output llamado “x”, el cual es el valor de las raíces del mencionado sistema de ecuaciones.

```
>> x=fsolve(fun,x0)
x =

    0.3931    0.3366
```

Si, finalmente, el usuario quiere chequear que el valor de “x” es efectivamente una raíz puede hacerlo del siguiente modo. Esto consiste en simplemente evaluar “x” en el sistema de ecuaciones, por lo que debería obtener un par de ceros.

```
>> root2d(x)
ans =

   -2.0351e-11   -9.9970e-13
```

Finalmente, se presenta a continuación como sería un cálculo para una sola ecuación en lugar de dos o más. En el script debería encontrarse lo siguiente:

```
function F=root1d(x)
    F=exp(-exp(-2*x))-x*(1+x^2);
end
```

Y luego introducir los siguientes comandos:

```
>> fun=@root1d
fun = @root1d
>> x0=0
x0 = 0
>> x=fsolve(fun,x0)
x = 0.5503
```

En Scilab la resolución de sistemas de ecuaciones no lineales es distinta. Se debe definir primero el sistema de ecuaciones al que se le busca la raíz, usando el comando “deff”, y luego se usa el comando “fsolve”.

```
--> deff("[y]=sist(x)", "y=[2*x(1)^2+3*x(2)^2-32, -3*x(1)^2+4*x(2)+48]")
```

En el primer input de “deff” se define que el sistema se llama “sist” y tiene una variable de entrada “x” (que después se verá que tiene dos vectores, como se ve en el segundo input) y una variable de salida de “y” (que después se verá que tiene dos vectores, como se ve en el segundo input). En el segundo input están las dos ecuaciones del sistema, separadas por una coma. De este modo acaba de definirse el sistema de ecuaciones.

Al igual que en Octave, el usuario puede darle una entrada al sistema de ecuaciones, por ejemplo $x=[1 \ 1]$, para obtener una salida, la cual será la siguiente. Y ahí se obtiene el valor de “y” para ese “x”.

```
--> x=[1 1];
```

```
--> sist(x)
ans =

    -27.    49.
```

Ahora deben calcularse las raíces. Debe usarse el comando “fsolve”, como se presenta más abajo:

```
--> [sol,err]=fsolve([10 0],sist,10^-10)
sol =

    4.    -1.310D-16
err =

    0.    0.
```

Este tiene tres inputs y dos outputs. El primero es el valor semilla que se utiliza para hallar la raíz, el segundo es a qué sistema se le calcula la raíz y el tercero es la tolerancia del error en el cálculo. Como outputs se obtiene en “x” la raíz del sistema y en “v” el valor de “y” en la raíz (el cual debería ser lo suficientemente cercano a cero, de acuerdo a nuestra tolerancia usado en el tercer input).



Tanto en Octave como en Scilab se debe tener en cuenta que “fsolve” calcula el valor de una raíz, pero esta no tiene porqué ser la única. Esto se puede ver fácilmente con una cuadrática con dos raíces, como la que se presenta a continuación. Debe notarse que, usando dos semillas distintas, se obtuvo dos resultados distintos.

```
--> [sol1,err1]=fsolve(2,cuad,10^-10)
sol1  =

    1.6084953
err1  =

    0.

--> [sol2,err2]=fsolve(-4,cuad,10^-10)
sol2  =

   -3.1084953
err2  =

   -1.776D-15
```

Capítulo 15

Herramientas de control automático de procesos



15.1 Definición de funciones de transferencia

Octave tiene varias funciones para simular el control de procesos. Sin embargo, en Octave primero tenemos que cargar el paquete “control”, que ya viene instalado con el programa.

```
>> pkg load control
```

Octave y Scilab permiten crear las funciones de transferencia que el usuario desee. Existen dos funciones para crearlas. El primero es “tf” y pide como inputs los coeficientes del numerador y el denominador.

```
F=tf([1],[1 1])
```

Transfer function 'F' from input 'u1' to output ...

$$y1: \frac{1}{s + 1}$$

Continuous-time model.

Por alguna razón este modo de crear una función de transferencia no está implementado en Scilab. También se puede crear una función de transferencia informando polos, ceros y ganancias usando el comando “zpk”.

```
>> zpk(1,[-2 -3],10)
```

Transfer function 'ans' from input 'u1' to output ...

$$y1: \frac{10 s - 10}{s^2 + 5 s + 6}$$

Continuous-time model.

En Scilab se hace igual, pero debe agregarse otro input, dependiendo de si el usuario desea que se use variable continua o discreta.

```
--> zpk(1,[-2 -3],10,"c")  
ans =
```

$$10 \frac{(s-1)}{(s+2)(s+3)}$$

```
--> zpk(1,[-2 -3],10,"d")  
ans =
```

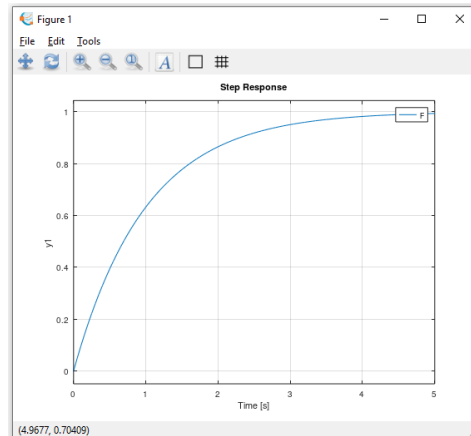
$$10 \frac{(z-1)}{(z+2)(z+3)}$$

15.2 Respuestas de lazo abierto

A continuación se presenta como calcular respuestas de lazo abierto a funciones de transferencia. Tanto Octave como Scilab permiten conocer la respuesta ante entradas (también llamadas perturbaciones) tipo salto escalón, rampa o impulso. Para esto, se usan las funciones “step”, “ramp” e “impulse” en Octave. **Para hacer esto mismo se va a utilizar la función “csim” en Scilab, como se verá más adelante.**

A las funciones de transferencia se las puede someter a un salto unitario en la entrada usando el siguiente comando, el cual dará como respuesta un gráfico.

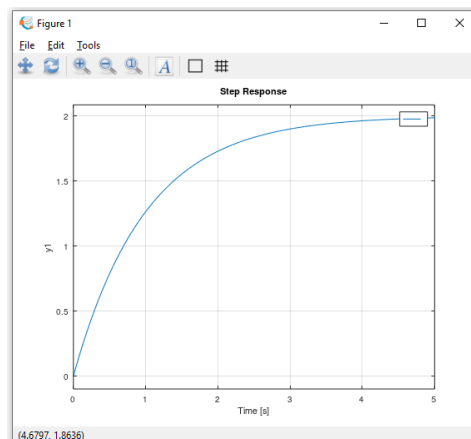
```
>> step(F)
```



De hecho, cualquiera de las funciones usadas para perturbar una planta (“step”, “ramp”, “impulse”, “lsim”, **“csim”**) dan como resultado un gráfico de como evoluciona la variable de salida.

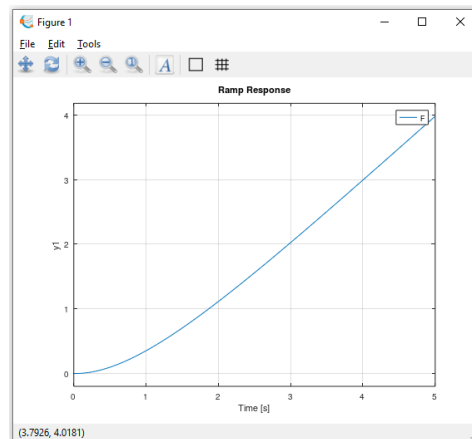
Además, este comando permite introducir saltos no unitarios utilizando “step(F*ΔS)”, donde ΔS es el cambio de valor en la entrada (si fuese igual a 1 sería el salto unitario).

```
>> step(F*2)
```

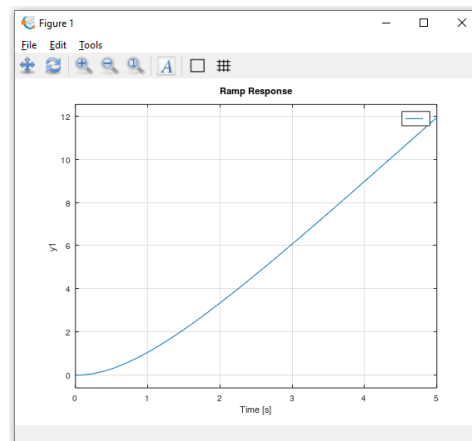


Por otro lado, Octave permite hacer lo mismo con una rampa de pendiente S. **Vale la pena aclarar que dicha función no está presente en Scilab, pero esta carencia puede ser subsanada usando la función “csim”, la cual se estudia un poco más adelante.**


```
>> ramp(F) %Rampa de pendiente 1
```

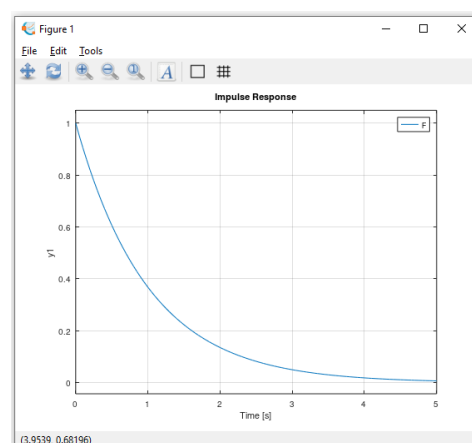


```
>> ramp(F*3) %Rampa de pendiente 3
```



Y se puede simular un impulso en la entrada usando la función “impulse” en Octave o “csim” es Scilab.

```
>> impulse(F)
```

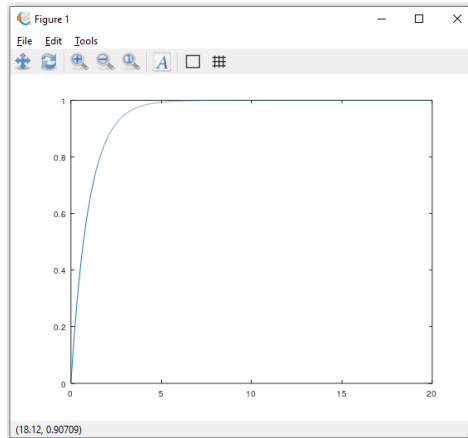


Finalmente, el usuario puede que necesite estos resultados con un paso de tiempo determinado, y que la respuesta se presente durante el tiempo que considera necesario. Para eso, primero se debe crear un vector donde se define el tiempo de la respuesta y el paso de tiempo entre cada muestreo.

```
>> t=0:0.01:20;
```

Este vector significa que la respuesta va a ser presentada hasta 20 segundos después del cambio en la entrada (que puede ser un impulso, un salto escalón o una pendiente) y su período de muestreo es de 0.1 segundos. En total, la respuesta tendrá 201 puntos. La respuesta se obtiene del siguiente modo:

```
>> R=step(F,t)
```

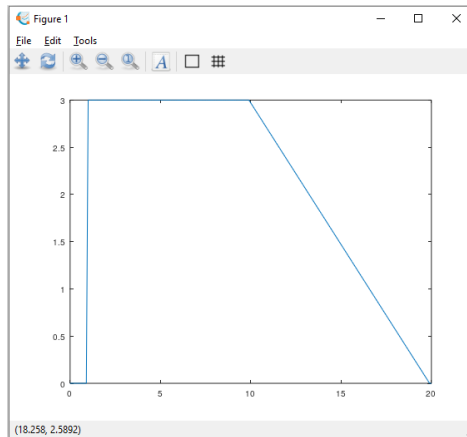


Si bien puede no estar claro en esta gráfica, hay un punto cada 0.1 segundos y la respuesta dura 20 segundos. Esto se debe a que se usó a “t” como input de “step” y que “F” tiene igual dimensión que “t” (de lo contrario el programa daría error).

El usuario también puede introducir la entrada que desee, además de un salto unitario, una rampa y un impulso, usando la función “lsim”. Es decir que el usuario obtiene la respuesta que desea para la entrada deseada.

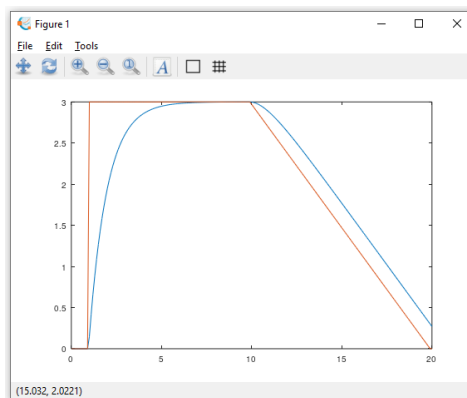
La función “lsim” funciona del siguiente modo. Esta requiere que se introduzca cómo inputs el sistema que se va a perturbar “F” y la entrada del mismo “U”, además del vector de tiempos “t”. Por otro lado, los resultados se van a guardar en un vector “R”. El vector de tiempo “t” y el vector perturbación “U” definen la entrada del sistema. Esto se presentará con un ejemplo en el que dicha entrada será un step y una rampa. Como puede verse en el siguiente gráfico, la entrada tiene la forma deseada usando las variables “t” y “U”.

```
>> t=0:0.1:20;  
>> U=3*ones(201,1);  
>> U(end/2+0.5:end-1)=2.97:-0.03:0;  
>> U(end)=0;  
>> plot(t,U)
```



Luego se aplica la función “lsim”, cuyos inputs son “F”, el sistema que se va a perturbar, “U” y “t”, que son los vectores que definen la entrada (o perturbación).

```
>> R=lsim(F,U,t);
```



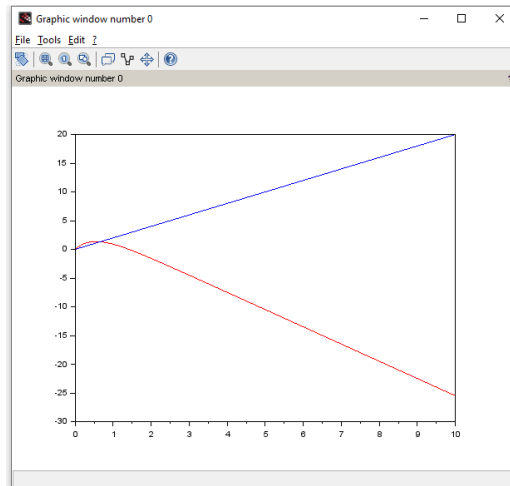
El comando análogo en Scilab es “csim”. El mismo pide tres inputs, el primero es la perturbación (análoga a “U” en el caso anterior), el segundo es el tiempo (asimilable a “t”) y el tercero es qué sistema se va a perturbar “F”.

Por ejemplo, se usará una rampa de pendiente 2 a un sistema de primer orden “F”. El resultado se presenta en la siguiente figura, donde se observa la rampa en azul y la respuesta de la planta en rojo.

```
--> F=zpk(1,-2,3,"c")
F =
```

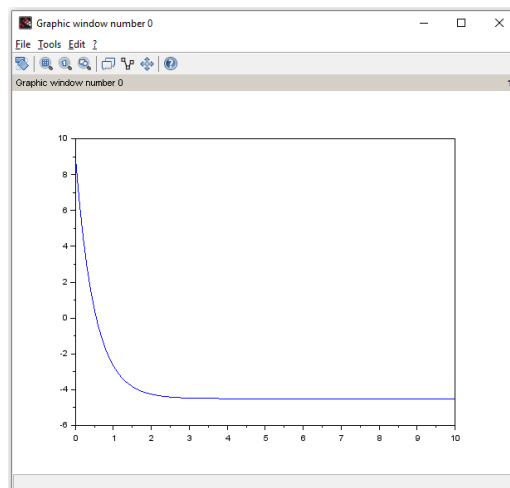
$$3 \frac{(s-1)}{(s+2)}$$

```
--> U=0:0.2:20;
--> t=0:0.1:10;
--> R=csim(U,t,F);
--> plot(t,R,"r")
--> mtlb_hold on
--> plot(t,U,"b")
```



También se le puede introducir un salto o un impulso reemplazando el primer argumento por “step” y “impulse”. Se puede cambiar la altura del salto multiplicando el último argumento (sys) por el valor del salto. Por ejemplo, para utilizar como entrada un salto de tres unidades se puede realizar lo siguiente.

```
--> R=csim("step",t,F*3);
--> plot(t,R)
```



15.3 Definición de un PID y la respuesta a lazo cerrado

Puede que al usuario le interese armar su propio PID. Para eso se utiliza la función “pid”. Esto se va a ver usando un ejemplo en el que se va a crear uno cuya la ganancia proporcional es 1, la integral es 2, la derivativa es 3 y el filtro pasabajos tiene un tiempo de 0,1 segundos. La fórmula del mismo puede verse bien escribiendo “help pid”.

```
>> C=pid(1,2,3,0.1)
```

Transfer function 'C' from input 'u1' to output ...

$$y1: \frac{3.1 s^2 + 1.2 s + 2}{0.1 s^2 + s}$$

Continuous-time model.

Esto permite analizar respuestas de sistemas en lazos cerrados. Se crea un sistema cerrado formado por una planta “F” y el PID “C” recientemente creado. Además, tiene una salida “R” y un setpoint que sufre un salto unitario a los 5 segundos, llamado “U”. Finalmente, “H” es la función de transferencia de dicho lazo cerrado.

```
>> F=tf([1],[1 1]) %Se define el proceso o planta
```

Transfer function 'F' from input 'u1' to output ...

$$y1: \frac{1}{s + 1}$$

Continuous-time model.

```
>> H=C*F/(1+C*F) %Función de transferencia del sistema
```

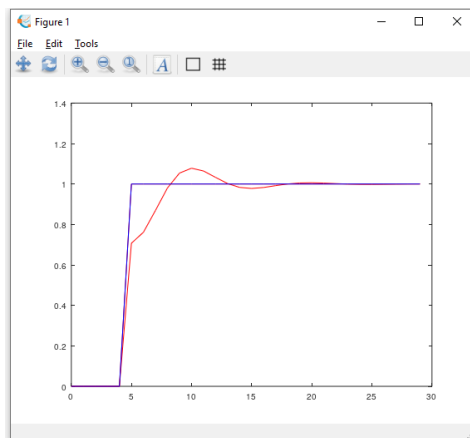
Transfer function 'H' from input 'u1' to output ...

$$y1: \frac{0.31 s^5 + 3.53 s^4 + 4.62 s^3 + 3.4 s^2 + 2 s}{0.01 s^6 + 0.53 s^5 + 4.94 s^4 + 6.82 s^3 + 4.4 s^2 + 2 s}$$

Continuous-time model.

```
>> U=ones(30,1); U(1:5)=0;t=0:1:29;
```

```
>> R=lsim(H,U,t);
```



En scilab no hay un comando para crear una función de transferencia de un PID. Sin embargo, esto no quita que no pueda armarse uno usando “zpk”.

15.4 Lugar de las raíces y diagrama de bode

En Octave también se puede analizar este sistema desde el lugar de las raíces o el diagrama de bode. Solo debe escribirse lo siguiente.

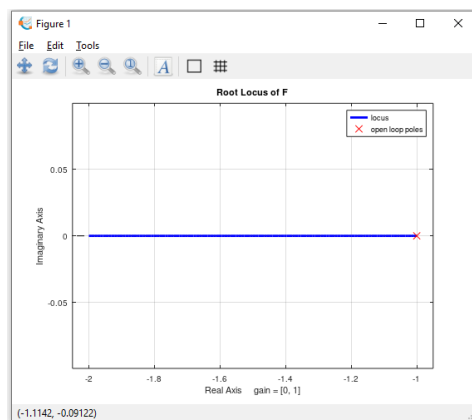
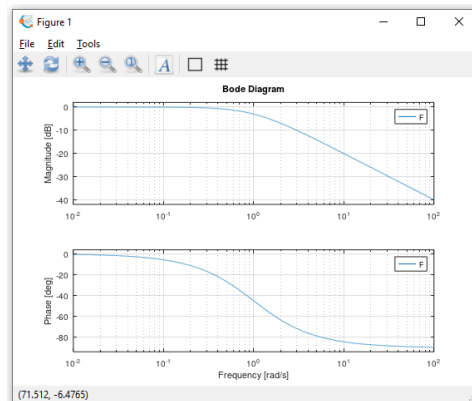
```
>> F=tf([1],[1 1])
```

Transfer function 'F' from input 'u1' to output ...

$$y1: \frac{1}{s + 1}$$

Continuous-time model.

```
>> bode(F)
>> rlocus(F)
```

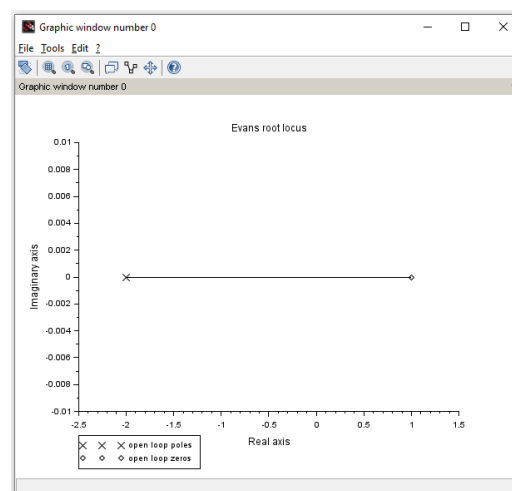
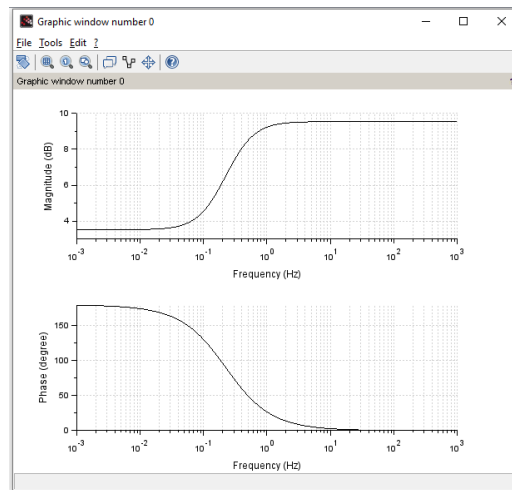


Hay, además, una función para crear el diagrama de bode en Scilab, que es igual a la de Octave. Por su parte, el comando “rlocus” es análogo a “evans” en Scilab. Sin embargo, este comando no se puede usar si se definió a la función de transferencia usando “zpk”. En realidad se debe usar primero “zpk2tf” y luego “evans”.

```
--> F=zpk(1,-2,3,"c")
F =
```

$$\frac{(s-1)}{(s+2)^3}$$

```
--> bode(F)
--> F2=zpk2tf(F)
--> evans(F2)
```



Capítulo 16

Ajuste de funciones



OCTAVE



16.1 Ajuste de funciones: Introducción

Una de las grandes utilidades de Octave y Scilab es la de ajuste de funciones. Esto significa que el usuario tiene un conjunto de pares ordenados y necesita encontrar una función que “ajuste” los mismos. Es decir, que pase (o al menos se acerque) a los pares ordenados. Este proceso funciona del siguiente modo:

1. El usuario tiene un conjunto de puntos
2. Propone una función, se sabe de qué tipo es (exponencial, cuadrática, etc.) pero no sabe sus parámetros (coeficientes de un polinomio, por ejemplo).
3. Se definen los valores iniciales (semilla) de los parámetros de dicha función.
4. El usuario utiliza la correspondiente función de ajuste en Octave o Scilab
5. Se obtiene como resultado la función que minimiza la distancia entre esta y los pares ordenados

En este punto Matlab y Octave difieren bastante en cuanto a como hacer el ajuste. Conceptualmente es similar, porque hay una función del programa que nos requiere los datos de x, los de y, un modelo de curva y un conjunto de parámetros iniciales. Sin embargo, la función que se usa (“fit” para Matlab y “leasqr” en Octave) y como introducir los inputs es distinto.

16.2 Ajuste de funciones: Octave

Antes de empezar con esta sección cabe resaltar que el paquete que contiene la función “leasqr” no está incluido en la versión 7.1.0, por lo que habría que descargarla de internet. Dicho paquete se llama “optim” y puede ser descargado en “Octave sourceforge”. Luego, deberá seguirse los pasos presentados en la sección 13.1, reemplazando la palabra “io” por “optim”.

```
>> pkg load optim
```

Primero se definen los pares ordenados que corresponden a este modelo. Los valores de x de estos pares ordenados están en “t” y los de y están en “data”. Así, “t” y “data” forman los pares ordenados.

```
>> t=[1:10:91];  
>> data=[0.90984 0.33610 0.13072 0.053245 0.022889 0.013929 0.0069914  
0.0044337 0.0021340 0.00073212];
```

Luego, se define una función. Tanto la variable x como los parámetros son introducidos como inputs de esta función, que será llamada “Leasqrfunc”. Cabe destacar que este tipo de llamados es lo que se denomina un “Function handle”.

```
%Definición de la función de ajuste y sus parámetros (p)
```

```
>> Leasqrfunc = @(x,p) p(1)*exp(-p(2)*x)
```

```
Leasqrfunc =
```

```
@(x, p) p (1) * exp (-p (2) * x)
```

También debe determinarse el valor inicial supuesto de los parámetros (el valor semilla).

```
>> p=[0.8 2];
```

Y, finalmente, se utiliza la función de Octave para ajustar.

```
>> [Sol,param]=leasqr(t,data,p,Leasqrfunc)
Sol =

Columns 1 through 9:

    9.0843e-01    3.4177e-01    1.2858e-01    4.8375e-02    1.8200e-02
    6.8472e-03    2.5760e-03    9.6916e-04    3.6462e-04

Column 10:

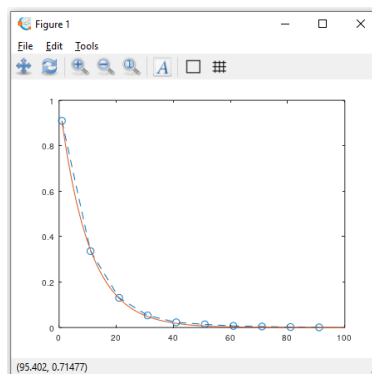
    1.3718e-04

param =

    1.001725
    0.097758
```

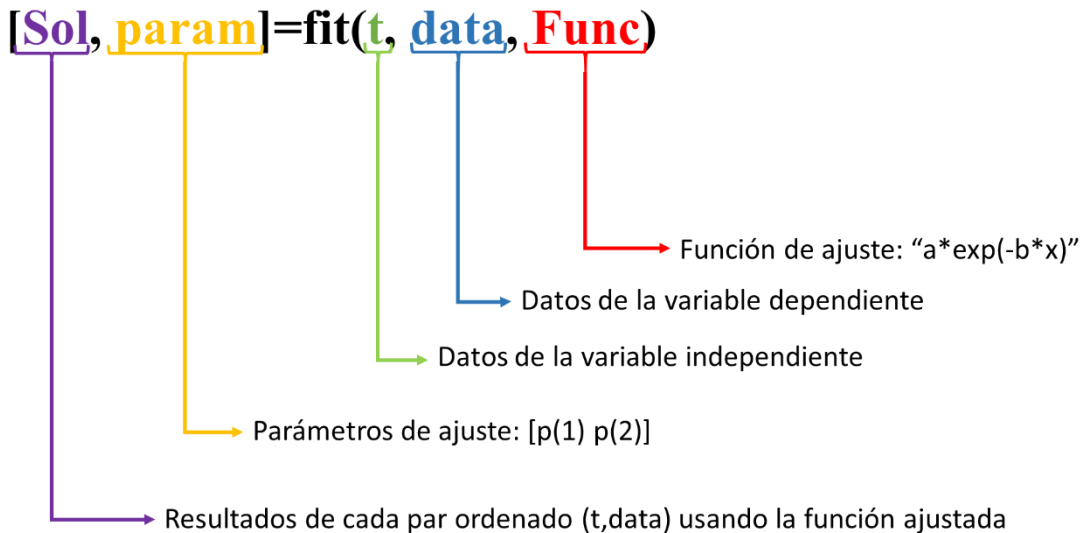
En la variable “param” se guardan los valores calculados para $p(1)$ y $p(2)$, mientras que en “Sol” se guardan los resultados para los valores del vector “t” al usar dichos parámetros en función ajustada. Ya teniendo los resultados se puede comparar los resultados.

```
>> plot(t,data,"--o")
>> hold on
>> plot(t,Sol,"r")
```

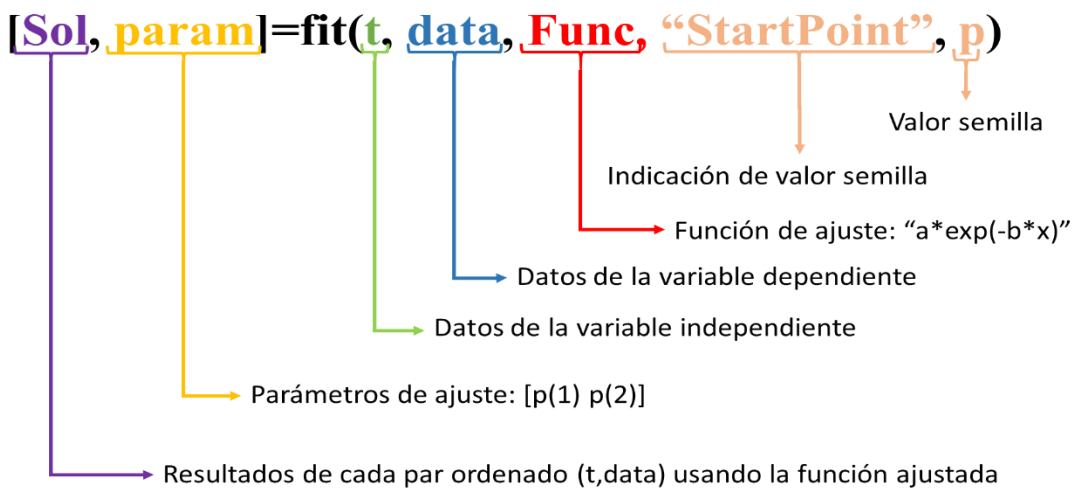


16.3 Ajuste de funciones: Matlab

En Matlab se hace de un modo similar. Para presentar esto se usarán los mismos pares ordenados que surgen de “t” y “data”. Además, ajustará con la misma función exponencial. La misma se guarda en la variable “Func” como un string “ $a \cdot \exp(-b \cdot x)$ ”. Se obtienen dos outputs, los cuales son “Sol” y “param”, que son idénticos a los obtenidos en Octave. Y Finalmente, la función de Matlab que se va a utilizar es “fit”. La imagen a continuación resume lo acá descripto:



Como en Octave, Matlab permite introducir un valor semilla. El mismo se introduce como input al final de la función "fit". La siguiente figura muestra esto mismo:



Observe que el valor de la semilla se introduce como input, anteponiéndole un string que es "StartPoint". De este modo, el software entiende que dicho input corresponde al valor con el que debe empezar la iteración.

En este punto corresponde resaltar algunas diferencias entre Matlab y Octave:

- El ajuste de funciones en Matlab no requiere un function handle, como si ocurre en Octave.
- Matlab no requiere como input un valor semilla, pero el usuario puede especificarlo, mientras que Octave es imposible realizar el ajuste in dicho valor.
- En Matlab, el usuario puede especificar cual función de ajuste desea usar (a continuación se presentan algunas de ellas) en lugar de introducir en "Func" la ecuación de la función de ajuste, mientras que en Octave es indispensable introducir dicha función como una function handle.

"Func"	Función de ajuste
'poly1'	$Y=p1*x+p2$
'poly2'	$Y=p1*x^2+p2*x+p3$
'poly3'	$Y=p1*x^3+p2*x^2+p3*x+p4$
'fourier1'	$Y = a0+a1*\cos(x*p)+b1*\sin(x*p)$
'gauss1'	$Y = a1*\exp(-((x-b1)/c1)^2)$
'power1'	$Y = a*x^b$

16.4 Ajuste de funciones: Scilab

El proceso de ajuste de una función es mucho más complejo en Scilab. Este requiere de dos funciones creadas por el usuario. En la primera es donde se encuentra definida la función que se busca ajustar, FA. La misma tiene como inputs los valores de "x" y los parámetros de la función (por ejemplo, si fuese una función lineal, estos parámetros son la pendiente y la ordenada al origen). Además, como output tiene los valores de "y". La segunda función, llamada "gap function", GF, calcula cuanto se desvia nuestra FA respecto a los valores de "y" que tenemos como datos. En otras palabras, el error de la FA. La GF tiene como inputs los valores de "x" y los valores semilla (el proceso de ajuste es iterativo, por lo que requiere de algún valor para iniciar) de los parámetros de ajuste de la FA. A continuación se presenta un ejemplo. En este se tiene un set de datos "x" e "y" y se busca ajustarlos por medio de un polinomio de grado 2. Es decir que hay tres parámetros de ajustes. A continuación se definen ambas funciones y se presentan los datos de entrada.

// FA, Función de ajuste

```
function y=FA(x, p)
    y=p(1)*(x-p(2))+p(3)*x^2
endfunction
```

// GF, Gap Function

```
function e=GF(p, Data)
    e=Data(2,:)-FA(Data(1,:),p)
endfunction
```

X	Y	X	Y	X	Y	X	Y
0.	12.55	0.9	9.86	1.8	15.19	2.7	17.69
0.1	14.90	1.	12.30	1.9	14.38	2.8	17.64
0.2	10.78	1.1	12.83	2.	13.03	2.9	19.68
0.3	12.13	1.2	13.21	2.1	16.58	3.0	20.00
0.4	13.54	1.3	10.67	2.2	13.45	3.1	20.50
0.5	13.14	1.4	12.54	2.3	14.10	3.2	20.52
0.6	14.06	1.5	11.16	2.4	15.22	3.3	23.24

0.7	13.10	1.6	11.37	2.5	15.46	3.4	23.04
0.8	13.97	1.7	11.56	2.6	17.45	3.5	26.59



Si bien los datos se presentan en una tabla de 8 x 10, por como está planteado el problema, se requiere que los datos se presenten en una matriz donde la primera columna son los datos de “X” y la segunda los datos de “Y”.

Esta tabla se presenta en la variable “Data” que se encuentra en la siguiente figura. La razón por la que “X” e “Y” aparecen transpuestas (‘) es porque las misas fueron definidas como vectores filas y para que queden como una tabla de dos columnas en “Data” se requiere transponerlas (y que queden como columnas, formando una tabla donde la primea columna es “X” y la segunda es “Y”).

Una vez definidas ambas funciones y los datos de entrada solo se requiere usar el comando “datafit”. El mismo tiene tres inputs. El primero es el nombre de la GP, el segundo es la matriz que tiene los datos de entrada y el último es un vector que contiene los valores semilla. Sobre este último debe notarse que el parámetro p(1) de la FA es la primera coordenada de dicho vector, p(2) es la segunda y así continua. En cuanto a los outputs, el comando “datafit” tiene dos. El primero son los parámetros de ajuste (es decir, el resultado del problema) y el segundo un indicador de qué tan bueno es el ajuste (la distancia residual promedio). A continuación, se presenta el llamado de la función.

```
--> Data=[X',Y'];

--> P0=[1 1 1]
P0    =

    1.    1.    1.

--> [p, dmin]=datafit(GP,Data,P0)
p      =

    0.0817805   -1.2231291    1.0693281
dmin    =

    0.0000098
```